Set-Coded Turing Machines in Isabelle/ZF

Tang Ziyi

October 14, 2026

Abstract

We construct an explicit closed set-theoretic sentence for the halting of a finite Turing machine on a finite input. For every transitive ZF set model, satisfaction of the sentence is equivalent to external execution reaching the final state. The construction uses set-coded machines, finite execution witnesses, and formulas naming finite objects. A primitive-recursive function constructs the sentence's natural-number code from independent machine-code and unary-input arguments. The library connects concrete program proofs to the existing Isabelle/ZF model interfaces. Observational tape equality and worked clients support reuse without unfolding the internal formulas.

Contents

1 Result **1**

2 Proof **1**
2.1 Finite witnesses 1
2.2 Closed sentences and codes 2

3 Use **3**
3.1 Machine semantics 3
3.2 Representation independence 3
3.3 A complete client 3

4 Sources and scope **4**

5 Set-Coded Turing Machines **4**
5.1 Machine Representation 4
5.2 Tape Semantics 6
5.3 One-Step Semantics 8
5.4 Finite Computations 10
5.5 Numeral Inputs 13
5.6 Halting 14

6 Observable Tape Contents **16**
6.1 Input Representation and Halting 19
6.2 Tape Span and Trailing Blanks 20

7 Using the Machine Interface **21**
7.1 Changing the finite representation 22

8 Machine Coding **23**
8.1 Natural Pairing 23
8.2 Natural Lists 30
8.3 Instruction Streams 33
8.4 Machine Numbers 36

9 Operations on Machine and List Codes **39**
9.1 Instruction Code Normalization 46

10 Primitive-Recursive Coding **47**
10.1 Composition and Projection 48
10.2 Arithmetic 49
10.3 Conditionals 51
10.4 Cantor Pairing 53
10.5 Cantor Projections 54
10.6 Coded Natural Lists 57

11 Primitive-Recursive Instruction Normalization **60**

12 Finite Witnesses in Transitive Models **61**
12.1 Finite object closure 62
12.2 Transitive model interpretations 63
12.3 Machine objects in a set model 63

13 Natural Membership in Transitive Models **64**

14 Internal Halting Formula **64**
14.1 Names for natural numbers 65
14.2 Model and tape updates 66
14.3 Iterated tails 68
14.4 Fetching an instruction 71
14.5 One step 74
14.6 Run formulas 76
14.7 Run correspondence 77
14.8 Halting satisfaction 79
14.9 Arities 81

15 Closed Halting Sentences **81**
15.1 Names for finite lists 81
15.2 Closed halting sentences 84
15.3 Model-theoretic consequences 85

16 Numerical Coding of Formulas **86**
16.1 Constructor codes 86
16.2 Injectivity 86

17 Primitive-Recursive Formula Constructors **87**
17.1 Tagged constructors 87
17.2 Logical connectives 88

18 Primitive-Recursive Names for Numerals **90**
18.1 Natural number names 90
18.2 Finite list names 91
18.3 Unary input names 92

19 Primitive-Recursive Machine Names **93**
19.1 Instruction names 94
19.2 Decoded list indices 95
19.3 Bounded reverse traversal 97
19.4 Complete machine names 100

20 Effective Halting Sentences **101**
20.1 Closed halting sentence code 101
20.2 Self-input instance 103

21 Using the Halting Formula Interface **103**
21.1 From execution to satisfaction 103
21.2 From a sentence code to satisfaction 104

1 Result

For a well-formed machine P and finite symbol input x, let $H(P, x)$ denote the constructed formula *halt_sentence*(P, x). It is a sentence, with no free variables. For every transitive ZF set model M,

$$M \models H(P, x) \quad \Longleftrightarrow \quad \text{halts_on}(P, x).$$

Here halting means that a finite number of external machine steps reaches the final state. The sentence is constructed from P and x alone and is the same for every model. A proof of a particular program's halting or nonhalting therefore determines the truth of its sentence in each such model. The result is conditional on the model premise and asserts no model existence.

The public contract consists of three theorems.

Formula membership	`halt_sentence_type`
No free variables	`arity_halt_sentence`
Satisfaction	`sats_halt_sentence_iff`

Their object premises are $P \in$ *tm_machine* and $x \in$ *list*(*symbol*). The predicate `transitive_zf_model` requires a transitive set satisfying the internalised ZF axiom set from `Internal_ZFC_Axioms`. This is a property of the model under consideration, distinct from the ambient Isabelle/ZF logic.

There is also an effective syntax interface. For natural e, n, let $P_e =$ *decode_machine*(e) and $x_n =$ *numeral_input*(n), the list of n ones. The certified primitive-recursive constructor F satisfies

$$F(e, n) = \ulcorner H(P_e, x_n) \urcorner.$$

Thus a program and its input determine both a closed sentence and its code. Section 20 proves membership in the bundled `prim_rec` class and the application equation. This is a certificate for numerical sentence construction. It does not supply a compiler implementing arbitrary primitive-recursive functions in the machine model.

2 Proof

2.1 Finite witnesses

A halting execution has a finite trace. If P halts on x at time n, the canonical witness is

$$r = \lambda k \in n + 1.\, \text{steps}(P, \text{initial_config}(x), k).$$

Each configuration is built from natural numbers and finite lists. The model contains these objects and their ordered pairs. Finite induction then places the entire trace in the model. The closure theorem `tm_finite_lambda_`

`closed` requires membership of each value; it does not require an external function to have been supplied internally. `transitive_zf_contains_trace` applies this argument to runs.

The explicit formula `halt_fm` binds a natural time and a witness set. It constrains the witness's value at time zero, each successive transition, and the final state. The canonical trace satisfies these conditions. Conversely, suppose an internal witness satisfies them. Induction identifies each constrained value with the corresponding external configuration. The one-step satisfaction theorem provides the induction step. At the final time, the external machine must therefore be in state zero.

These directions have different obligations. The external execution supplies a finite function that belongs to the model. An arbitrary internal witness need only satisfy the stated application constraints; its entire set need not be a finite function. No interpretation of an unbounded execution inside the model is assumed.

The shared lemmas `trans_run_steps` and `trace_trans`, in the locale `M_ZF1_trans`, give reconstruction and canonical transitions for arbitrary initial configurations and outer formula environments. They isolate the execution argument from a surrounding formula's variable layout. The halting theorem and downstream output predicates use the same lemmas. Section 14 contains the explicit formula and the proofs.

2.2 Closed sentences and codes

The formula `halt_fm` has environment $[P, x]$. Formulas naming natural numbers, instructions, and finite lists define P and x uniquely. Binding these names yields $H(P, x)$ and discharges both free variables. This construction uses the formula datatype of `ZF-Constructible.Formula` [6].

`Formula_Coding` assigns tags and paired natural-number codes to the four formula constructors. Structural induction proves injectivity. The primitive-recursive name constructors mirror the syntactic construction. A bounded reverse traversal builds the name of a decoded machine, whose list length is at most its code. The resulting application equation identifies $F(e, n)$ with the code of $H(P_e, x_n)$. Formula-code injectivity then identifies any well-formed formula bearing that code, so no separate formula decoder is needed for this contract.

The implementation of F is `pr_machine_input_sentence_code`, applied to $[e, n]$. The associated theorem, `sats_machine_input_sentence_code_iff`, derives satisfaction directly from the constructed code. Setting $n = e$ yields the unary constructor `pr_halt_sentence_code`. These certificates use the Isabelle/ZF primitive-recursive function class [7].

3 Use

3.1 Machine semantics

A machine is a finite list of instructions over the symbols 0 and 1. Each instruction contains an action and a next state. The actions write 0, write 1, move left, move right, or leave the tape unchanged. State 1 is initial and state 0 is final. The two entries for a positive state q occur at indices $2(q-1)$ and $2(q-1)+1$. Missing entries enter the final state without changing the tape. The final state is absorbing.

A configuration contains a state and two finite symbol lists. The left list starts immediately left of the head, while the right list starts at the scanned cell. Unrepresented cells are blank. `tm_step` and `steps` specify one step and finite iteration. The predicate `halts_on` includes machine and input validity. Theorem `halts_on_iff_finite_run` supplies the equivalent finite-function witness form. For nonhalting proofs, `not_halts_on_invariant` accepts an initial, step-closed predicate that excludes the final state.

3.2 Representation independence

The lists [1] and $[1, 0, 0]$ are distinct stored objects with equal observed contents. The relations `half_tape_eq`, `tape_eq`, and `config_eq` compare observations at fixed positions relative to the head. They ignore trailing stored blanks and preserve execution and halting. They do not identify head shifts. Literal storage length remains a separate property.

The theorem `sats_halt_sentence_half_tape_eq_iff` transfers internal halting truth between observationally equal inputs. The sentences may have different syntax and codes because they name different lists. Their satisfaction agrees.

3.3 A complete client

`Turing_Examples` defines a two-instruction machine that scans a unary input and writes one at the first blank. Its loop invariant gives the final configuration after $n+1$ steps. The head remains on the newly written cell. The proof derives halting and transfers the result to an input with an extra stored blank.

`Turing_Halting_Examples` continues this same example. The theorem `scan_append_sentence` obtains internal truth from the operational halting result. The theorem `scan_append_sentence_code` obtains truth of any formula bearing the constructed code. Neither proof unfolds the halting formula, the finite-object names, or the numerical constructor. The operational example can be imported without model theory or coding.

4 Sources and scope

The instruction-list and two-list tape design follows the Isabelle/HOL development of Xu, Zhang, and Urban [1] and its AFP entry *Universal Turing Machine* [3]. The representation uses the unshifted `tprog0` convention, with natural-number actions and symbols and no program-offset parameter. Here machines and finite runs are sets whose membership in a model can be proved in the same logic as the satisfaction statements.

Closure and satisfaction use the AFP model infrastructure [4]. The `Independence_CH` entry supplies the locale and full-ZF interpretation in `ZF_Trans_Interpretations` [2]. The proofs apply in the weaker locale `M_ZF1_trans`, which assumes transitivity, Zermelo set theory, and four specified replacement instances. The full-ZF results interpret this locale. No minimality of these assumptions is claimed. No CH independence result or choice assumption is used by the halting interface.

The underlying absoluteness method is classical [5, Chapter 12]. The contribution is the explicit formula, its effective closed instances, and their verified connection to concrete machine execution. The semantic interface accepts all finite symbol inputs; the numerical constructor uses unary inputs. Machine decoding normalises invalid instruction fields and is total on natural numbers. Encoding and then decoding a valid machine recovers it, while arbitrary machine codes need not be unique.

5 Set-Coded Turing Machines

theory *Turing-Machine*
  imports *ZF*
begin

5.1 Machine Representation

A machine is a finite instruction list. An instruction contains an action number and a next-state number. Actions 0 and 1 write symbols, 2 and 3 move the head, and 4 leaves the tape unchanged. State 1 is initial and state 0 is final. The named actions, states, and symbols are input abbreviations for these natural numbers. Printed propositions use numbers so that the same set, such as 0, is not given an action name when it occurs as a state or symbol.

definition *symbol* :: *i* **where**
  symbol ≡ *2*

lemma *symbol-nat*: *a* ∈ *symbol* ⟹ *a* ∈ *nat*
  unfolding *symbol-def* **by** *auto*

definition *action* :: *i* **where**

action ≡ *succ*(*succ*(*succ*(*2*)))

lemma *action-in-nat* [*TC*]:
action ∈ *nat*
unfolding *action-def* **by** *typecheck*

lemma *action-subset-nat*:
action ⊆ *nat*
using *action-in-nat Ord-nat* **by** (*blast dest*: *OrdmemD*)

abbreviation (*input*) *blank-symbol* :: *i* **where**
blank-symbol ≡ *0*

abbreviation (*input*) *one-symbol* :: *i* **where**
one-symbol ≡ *1*

abbreviation (*input*) *final-state* :: *i* **where**
final-state ≡ *0*

abbreviation (*input*) *initial-state* :: *i* **where**
initial-state ≡ *1*

abbreviation (*input*) *write-blank* :: *i* **where**
write-blank ≡ *0*

abbreviation (*input*) *write-one* :: *i* **where**
write-one ≡ *1*

abbreviation (*input*) *move-left* :: *i* **where**
move-left ≡ *2*

abbreviation (*input*) *move-right* :: *i* **where**
move-right ≡ *succ*(*2*)

abbreviation (*input*) *nop* :: *i* **where**
nop ≡ *succ*(*succ*(*2*))

definition *tm-tape* :: *i* **where**
tm-tape ≡ *list*(*symbol*) × *list*(*symbol*)

definition *instruction* :: *i* **where**
instruction ≡ *action* × *nat*

definition *tm-machine* :: *i* **where**
tm-machine ≡ *list*(*instruction*)

definition *configuration* :: *i* **where**
configuration ≡ *nat* × *tm-tape*

lemma *symbol-type* [*simp*, *TC*]:
shows *blank-symbol* ∈ *symbol*
and *one-symbol* ∈ *symbol*
unfolding *symbol-def* **by** (*simp-all add*: *succ-def*)

lemma *action-type* [*simp*, *TC*]:
shows *write-blank* ∈ *action*
and *write-one* ∈ *action*
and *move-left* ∈ *action*
and *move-right* ∈ *action*
and *nop* ∈ *action*
unfolding *action-def* **by** (*simp-all add*: *succ-def*)

lemma *tapeI* [*intro*]:
assumes *left*: *l* ∈ *list*(*symbol*)
and *right*: *r* ∈ *list*(*symbol*)
shows ⟨*l*,*r*⟩ ∈ *tm-tape*
using *left right* **unfolding** *tm-tape-def* **by** *typecheck*

lemma *tapeD*:
assumes *tm-tape*: *t* ∈ *tm-tape*
shows *fst*(*t*) ∈ *list*(*symbol*)
and *snd*(*t*) ∈ *list*(*symbol*)
using *tm-tape* **unfolding** *tm-tape-def* **by** *typecheck*+

5.2 Tape Semantics

The left list begins with the cell immediately left of the head; the right list begins with the scanned cell. Cells beyond either finite list are blank. This representation uses the Isabelle/ZF equations hd([]) = 0 and nth(n,[]) = 0, with blank symbol 0.

definition *scan* :: *i* ⇒ *i* **where**
scan(*t*) ≡ *hd*(*snd*(*t*))

lemma *hd-symbol-type*:
assumes *symbols*: *xs* ∈ *list*(*symbol*)
shows *hd*(*xs*) ∈ *symbol*
using *symbols*
by (*erule-tac a*=*xs* **in** *list.cases*) *auto*

lemma *scan-type* [*TC*]:
assumes *tm-tape*: *t* ∈ *tm-tape*
shows *scan*(*t*) ∈ *symbol*
proof −
from *tm-tape* **have** *snd*(*t*) ∈ *list*(*symbol*) **by** (*rule tapeD*)
then show *?thesis*
unfolding *scan-def* **by** (*rule hd-symbol-type*)
qed

lemma *scan-pair* [*simp*]:
scan(⟨*l*,*r*⟩) = *hd*(*r*)
unfolding *scan-def* **by** *simp*

definition *tm-update* :: [*i*,*i*] ⇒ *i* **where**
tm-update(*a*,*t*) ≡
if a = *write-blank then*
⟨*fst*(*t*),*Cons*(*blank-symbol*,*tl*(*snd*(*t*)))⟩
else if a = *write-one then*
⟨*fst*(*t*),*Cons*(*one-symbol*,*tl*(*snd*(*t*)))⟩
else if a = *move-left then*
⟨*tl*(*fst*(*t*)),*Cons*(*hd*(*fst*(*t*)),*snd*(*t*))⟩
else if a = *move-right then*
⟨*Cons*(*hd*(*snd*(*t*)),*fst*(*t*)),*tl*(*snd*(*t*))⟩
else t

lemma *tm-update-type* [*TC*]:
assumes *tm-tape*: *t* ∈ *tm-tape*
shows *tm-update*(*a*,*t*) ∈ *tm-tape*
proof −
from *tm-tape*
have *left*: *fst*(*t*) ∈ *list*(*symbol*)
and *right*: *snd*(*t*) ∈ *list*(*symbol*)
by (*rule tapeD*)+
from *left* **have** *left-head*: *hd*(*fst*(*t*)) ∈ *symbol*
by (*rule hd-symbol-type*)
from *right* **have** *right-head*: *hd*(*snd*(*t*)) ∈ *symbol*
by (*rule hd-symbol-type*)
from *left* **have** *left-tail*: *tl*(*fst*(*t*)) ∈ *list*(*symbol*)
by (*rule tl-type*)
from *right* **have** *right-tail*: *tl*(*snd*(*t*)) ∈ *list*(*symbol*)
by (*rule tl-type*)
show *?thesis*
unfolding *tm-update-def*
using *tm-tape left right left-head right-head left-tail right-tail*
by *auto*
qed

lemma *tm-update-write-blank* [*simp*]:
tm-update(*write-blank*,*t*) =
⟨*fst*(*t*),*Cons*(*blank-symbol*,*tl*(*snd*(*t*)))⟩
unfolding *tm-update-def* **by** *simp*

lemma *tm-update-write-one* [*simp*]:
tm-update(*write-one*,*t*) =
⟨*fst*(*t*),*Cons*(*one-symbol*,*tl*(*snd*(*t*)))⟩
unfolding *tm-update-def* **by** *simp*

lemma *tm-update-move-left* [*simp*]:

tm-update(*move-left*,*t*) =
⟨*tl*(*fst*(*t*)),*Cons*(*hd*(*fst*(*t*)),*snd*(*t*))⟩
unfolding *tm-update-def* **by** *simp*

lemma *tm-update-move-right* [*simp*]:
tm-update(*move-right*,*t*) =
⟨*Cons*(*hd*(*snd*(*t*)),*fst*(*t*)),*tl*(*snd*(*t*))⟩
unfolding *tm-update-def* **by** *simp*

lemma *tm-update-nop* [*simp*]:
tm-update(*nop*,*t*) = *t*
unfolding *tm-update-def* **by** *simp*

5.3 One-Step Semantics

For a nonfinal state q and scanned symbol b, the instruction slot is 2 * pred(q) + b. A missing slot enters the final state without changing the tape. The final state is absorbing.

definition *slot* :: [*i*,*i*] ⇒ *i* **where**
slot(*q*,*b*) ≡ *2* #∗ *pred*(*q*) #+ *b*

lemma *slot-type* [*TC*]:
assumes *state*: *q* ∈ *nat*
and *scanned*: *b* ∈ *symbol*
shows *slot*(*q*,*b*) ∈ *nat*
using *state scanned* **unfolding** *slot-def symbol-def* **by** *typecheck*

lemma *slot-succ* [*simp*]:
assumes *state*: *q* ∈ *nat*
shows *slot*(*succ*(*q*),*b*) = *2* #∗ *q* #+ *b*
using *state* **unfolding** *slot-def* **by** *simp*

definition *fetch* :: [*i*,*i*,*i*] ⇒ *i* **where**
fetch(*M*,*q*,*b*) ≡
if q = *final-state then* ⟨*nop*,*final-state*⟩
else if slot(*q*,*b*) < *length*(*M*) *then*
nth(*slot*(*q*,*b*),*M*)
else ⟨*nop*,*final-state*⟩

lemma *fetch-type* [*TC*]:
assumes *tm-machine*: *M* ∈ *tm-machine*
shows *fetch*(*M*,*q*,*b*) ∈ *instruction*
using *tm-machine*
unfolding *fetch-def tm-machine-def instruction-def*
by (*auto intro*: *nth-type*)

lemma *fetch-final* [*simp*]:
fetch(*M*,*final-state*,*b*) = ⟨*nop*,*final-state*⟩
unfolding *fetch-def* **by** *simp*

lemma *fetch-in-range* [*simp*]:
assumes *nonfinal*: $q \neq$ *final-state*
and *in-range*: *slot*(q,b) < *length*(M)
shows *fetch*(M,q,b) = *nth*(*slot*(q,b),M)
using *nonfinal in-range* **unfolding** *fetch-def* **by** *simp*

lemma *fetch-out-of-range* [*simp*]:
assumes *out-of-range*: ¬ *slot*(q,b) < *length*(M)
shows *fetch*(M,q,b) = ⟨*nop*,*final-state*⟩
using *out-of-range* **unfolding** *fetch-def* **by** *simp*

definition *tm-step* :: [i,i] ⇒ i **where**
tm-step(M,c) ≡
let ins = *fetch*(M,*fst*(c),*scan*(*snd*(c)))
in ⟨*snd*(*ins*),*tm-update*(*fst*(*ins*),*snd*(c))⟩

lemma *step-type* [*TC*]:
assumes *tm-machine*: $M \in$ *tm-machine*
and *configuration*: $c \in$ *configuration*
shows *tm-step*(M,c) ∈ *configuration*
proof −
have *fetched-instruction*:
fetch(M,*fst*(c),*scan*(*snd*(c))) ∈ *instruction*
using *tm-machine* **by** (*rule fetch-type*)
have *current-tape*: *snd*(c) ∈ *tm-tape*
using *configuration* **unfolding** *configuration-def* **by** *typecheck*
show *?thesis*
unfolding *tm-step-def Let-def configuration-def*
using *fetched-instruction current-tape*
unfolding *instruction-def* **by** *typecheck*
qed

lemma *step-pair* [*simp*]:
tm-step(M,⟨q,t⟩) =
⟨*snd*(*fetch*(M,q,*scan*(t))),
tm-update(*fst*(*fetch*(M,q,*scan*(t))),t)⟩
unfolding *tm-step-def Let-def* **by** *simp*

lemma *step-final* [*simp*]:
tm-step(M,⟨*final-state*,t⟩) = ⟨*final-state*,t⟩
unfolding *tm-step-def Let-def* **by** *simp*

lemma *step-final-configuration* [*simp*]:
assumes *configuration*: $c \in$ *configuration*
and *final*: *fst*(c) = *final-state*
shows *tm-step*(M,c) = c
proof −
from *configuration* **obtain** q t **where** *representation*: c = ⟨q,t⟩

unfolding *configuration-def* **by** *auto*
from *final representation* **have** $q = \textit{final-state}$ **by** *simp*
with *representation* **show** *?thesis* **by** *simp*
qed

5.4 Finite Computations

A finite run is a set-theoretic function on succ(n). The equivalence theorem identifies such a run with n iterations of the step function. This witness form supports internal descriptions of finite computations in set-theoretic models.

definition *steps* :: $[i,i,i] \Rightarrow i$ **where**
$\textit{steps}(M,c,n) \equiv (\lambda x.\ \textit{tm-step}(M,x))\hat{}n\ (c)$

lemma *steps-0* [*simp*]:
$\textit{steps}(M,c,0) = c$
unfolding *steps-def* **by** *simp*

lemma *steps-succ* [*simp*]:
$\textit{steps}(M,c,\textit{succ}(n)) = \textit{tm-step}(M,\textit{steps}(M,c,n))$
unfolding *steps-def* **by** *simp*

lemma *steps-type* [*TC*]:
assumes *tm-machine*: $M \in \textit{tm-machine}$
and *configuration*: $c \in \textit{configuration}$
and *time*: $n \in \textit{nat}$
shows $\textit{steps}(M,c,n) \in \textit{configuration}$
unfolding *steps-def*
proof (*rule iterates-type*)
show $n \in \textit{nat}$ **by** (*rule time*)
show $c \in \textit{configuration}$ **by** (*rule configuration*)
fix x
assume $x \in \textit{configuration}$
with *tm-machine* **show** $\textit{tm-step}(M,x) \in \textit{configuration}$
by (*rule step-type*)
qed

lemma *steps-add*:
assumes *m-nat*: $m \in \textit{nat}$
and *n-nat*: $n \in \textit{nat}$
shows $\textit{steps}(M,c,m\ \#+\ n) = \textit{steps}(M,\textit{steps}(M,c,m),n)$
proof −
from *n-nat* **show** *?thesis*
proof (*induct n rule*: *nat-induct*)
case *0*
with *m-nat* **show** *?case* **by** *simp*
next
case (*succ n*)
with *m-nat* **show** *?case* **by** *simp*

```
  qed
qed

lemma steps-invariant:
  assumes time: n ∈ nat and start: I(c)
    and closed: ⋀d. I(d) ⟹ I(tm-step(P,d))
  shows I(steps(P,c,n))
  using time by (induct n rule: nat-induct) (auto intro: start closed)

lemma steps-final [simp]:
  assumes time: n ∈ nat
  shows steps(M,⟨final-state,t⟩,n) =
    ⟨final-state,t⟩
  using time unfolding steps-def
  by (rule iterates-triv) simp

lemma steps-final-absorb:
  assumes machine-M: M ∈ tm-machine
    and configuration: c ∈ configuration
    and earlier: m ∈ nat
    and later: n ∈ nat
    and ordered: m ≤ n
    and final: fst(steps(M,c,m)) = final-state
  shows steps(M,c,n) = steps(M,c,m)
proof −
  let ?d = n #− m
  let ?z = steps(M,c,m)
  from steps-type[OF machine-M configuration earlier]
  have reached: ?z ∈ configuration .
  from reached final have absorbed:
    steps(M,?z,?d) = ?z
  proof −
    from reached obtain q t where representation: ?z = ⟨q,t⟩
      unfolding configuration-def by auto
    from final representation have q = final-state by simp
    moreover have ?d ∈ nat by typecheck
    ultimately show ?thesis using representation by simp
  qed
  from add-diff-inverse[OF ordered later] have decomposition:
    m #+ ?d = n .
  have difference: ?d ∈ nat by typecheck
  from steps-add[OF earlier difference, of M c] have split:
    steps(M,c,m #+ ?d) = steps(M,?z,?d) .
  from decomposition split absorbed show ?thesis by simp
qed

lemma steps-final-mono:
  assumes machine-M: M ∈ tm-machine
    and configuration: c ∈ configuration
```

```
    and earlier: m ∈ nat
    and later: n ∈ nat
    and ordered: m ≤ n
    and final: fst(steps(M,c,m)) = final-state
  shows fst(steps(M,c,n)) = final-state
  using steps-final-absorb[OF assms] final by simp

definition finite-run :: [i,i,i] ⇒ o where
  finite-run(M,n,r) ≡
    relation(r) ∧
    function(r) ∧
    domain(r) = succ(n) ∧
    (∀ k∈n. r‘succ(k) = tm-step(M,r‘k))

lemma finite-run-iff-steps:
  assumes time: n ∈ nat
  shows finite-run(M,n,r) ∧ r‘0 = c ⟷
    r = (λk∈succ(n). steps(M,c,k))
proof
  assume run: finite-run(M,n,r) ∧ r‘0 = c
  then have relational: relation(r)
    and functional: function(r)
    and domain: domain(r) = succ(n)
    and initial: r‘0 = c
    and next-step: ⋀k. k ∈ n ⟹ r‘succ(k) = tm-step(M,r‘k)
    unfolding finite-run-def by auto
  from function-imp-Pi[OF functional relational] domain
  have run-type: r ∈ succ(n) → range(r)
    by simp
  have canonical-type:
    (λk∈succ(n). steps(M,c,k))
      ∈ succ(n) → {steps(M,c,k). k ∈ succ(n)}
    by (rule lam-funtype)
  have trace-at:
    ⋀k. k ∈ succ(n) ⟹ r‘k = steps(M,c,k)
  proof −
    fix k
    assume within: k ∈ succ(n)
    with time have natural: k ∈ nat
    by (blast intro: Ord-trans)
    from natural within show r‘k = steps(M,c,k)
    proof (induct k rule: nat-induct)
      case 0
      with initial show ?case by simp
    next
      case (succ k)
      from succ.prems time have before-end: k ∈ n
        by (blast intro: Ord-trans nat-into-Ord)
      then have within: k ∈ succ(n) by auto
```

```
      from next-step[OF before-end] succ.hyps(2)[OF within]
      show ?case by simp
    qed
  qed
  show r = (λk∈succ(n). steps(M,c,k))
  proof (rule fun-extension)
    show r ∈ succ(n) → range(r)
      by (rule run-type)
    show (λk∈succ(n). steps(M,c,k))
        ∈ succ(n) → {steps(M,c,k). k ∈ succ(n)}
      by (rule canonical-type)
    fix k
    assume k ∈ succ(n)
    with trace-at show r‘k = (λk∈succ(n). steps(M,c,k))‘k
      by simp
  qed
next
  assume canonical: r = (λk∈succ(n). steps(M,c,k))
  have next-step:
    ⋀k. k ∈ n ⟹
      (λj∈succ(n). steps(M,c,j))‘succ(k) =
      tm-step(M,(λj∈succ(n). steps(M,c,j))‘k)
  proof −
    fix k
    assume before-end: k ∈ n
    then have within: k ∈ succ(n)
      by (rule succI2)
    from before-end nat-into-Ord[OF time]
    have next-within: succ(k) ∈ succ(n)
      by (simp add: Ord-succ-mem-iff)
    show (λj∈succ(n). steps(M,c,j))‘succ(k) =
      tm-step(M,(λj∈succ(n). steps(M,c,j))‘k)
      using within next-within by simp
  qed
  from time have zero-within: 0 ∈ succ(n)
    by (rule nat-0-le [THEN ltD])
  have initial: (λk∈succ(n). steps(M,c,k))‘0 = c
    using zero-within by simp
  show finite-run(M,n,r) ∧ r‘0 = c
    unfolding canonical finite-run-def
    using next-step initial
    by (auto intro: relation-lam function-lam)
qed
```

5.5 Numeral Inputs

```
definition numeral-input :: i ⇒ i where
  numeral-input(n) ≡ repeat(one-symbol,n)
```

lemma *numeral-input-type* [*TC*]:
  assumes *natural*: $n \in nat$
  shows *numeral-input*(n) $\in$ *list*(*symbol*)
  using *natural* **unfolding** *numeral-input-def* **by** *typecheck*

lemma *length-numeral-input* [*simp*]:
  assumes *natural*: $n \in nat$
  shows *length*(*numeral-input*(n)) = n
  using *natural* **unfolding** *numeral-input-def* **by** (*rule length-repeat*)

lemma *numeral-input-injective*:
  assumes *first*: $m \in nat$
    and *second*: $n \in nat$
    and *equal*: *numeral-input*(m) = *numeral-input*(n)
  shows $m = n$
proof −
  from *equal* **have** *length*(*numeral-input*(m)) = *length*(*numeral-input*(n))
    by *simp*
  with *first second* **show** *?thesis* **by** *simp*
qed

5.6 Halting

abbreviation *initial-config* :: $i \Rightarrow i$ **where**
  initial-config(x) ≡ ⟨*initial-state*,⟨[],x⟩⟩

lemma *initial-config-type* [*TC*]:
  assumes *input*: $x \in list(symbol)$
  shows *initial-config*(x) $\in$ *configuration*
  using *input* **unfolding** *configuration-def tm-tape-def* **by** *typecheck*

definition *halts-on* :: [i,i] ⇒ o **where**
  halts-on(M,x) ≡
    $M \in$ *tm-machine* ∧
    $x \in list(symbol)$ ∧
    (∃ n∈nat.
      fst(*steps*(M,*initial-config*(x),n)) = *final-state*)

lemma *halts-onI*:
  assumes *machine*: $P \in$ *tm-machine* **and** *input*: $x \in list(symbol)$
    and *time*: $n \in nat$
    and *final*: *fst*(*steps*(P,*initial-config*(x),n)) = *final-state*
  shows *halts-on*(P,x)
  using *assms* **unfolding** *halts-on-def* **by** *auto*

A nonhalting proof may give a step-closed invariant that excludes the final state. Only reachable configurations need to satisfy the invariant. The rule requires neither a running-time bound nor a complete transition table.

theorem *not-halts-on-invariant*:

assumes *start*: $I(\mathit{initial\text{-}config}(x))$
and *closed*: $\bigwedge c.\ I(c) \Longrightarrow I(\mathit{tm\text{-}step}(P,c))$
and *nonfinal*: $\bigwedge c.\ I(c) \Longrightarrow \mathit{fst}(c) \neq \mathit{final\text{-}state}$
shows $\neg\ \mathit{halts\text{-}on}(P,x)$
proof
assume *halts-on*(P,x)
then obtain n **where** *time*: $n \in \mathit{nat}$
and *final*: $\mathit{fst}(\mathit{steps}(P,\mathit{initial\text{-}config}(x),n)) = \mathit{final\text{-}state}$
unfolding *halts-on-def* **by** *auto*
from *steps-invariant*[*OF time start closed*] *nonfinal final* **show** *False* **by** *blast*
qed

theorem *halts-on-iff-finite-run*:
$\mathit{halts\text{-}on}(M,x) \longleftrightarrow$
$M \in \mathit{tm\text{-}machine} \wedge$
$x \in \mathit{list}(\mathit{symbol}) \wedge$
$(\exists\, n{\in}\mathit{nat}.\ \exists\, r.$
$\mathit{finite\text{-}run}(M,n,r) \wedge$
$r`0 = \mathit{initial\text{-}config}(x) \wedge$
$\mathit{fst}(r`n) = \mathit{final\text{-}state})$
proof
assume *halts*: *halts-on*(M,x)
then obtain n **where**
tm-machine: $M \in \mathit{tm\text{-}machine}$
and *input*: $x \in \mathit{list}(\mathit{symbol})$
and *time*: $n \in \mathit{nat}$
and *final*: $\mathit{fst}(\mathit{steps}(M,\mathit{initial\text{-}config}(x),n)) = \mathit{final\text{-}state}$
unfolding *halts-on-def* **by** *auto*
let $?r = \lambda k{\in}\mathit{succ}(n).\ \mathit{steps}(M,\mathit{initial\text{-}config}(x),k)$
have *run*: $\mathit{finite\text{-}run}(M,n,?r) \wedge\ ?r`0 = \mathit{initial\text{-}config}(x)$
using *finite-run-iff-steps*[*OF time*, *of M ?r initial-config*(x)]
by *simp*
have *endpoint*: $?r`n = \mathit{steps}(M,\mathit{initial\text{-}config}(x),n)$
using *time* **by** *simp*
have *final-run*: $\mathit{fst}(?r`n) = \mathit{final\text{-}state}$
using *endpoint final* **by** *simp*
show $M \in \mathit{tm\text{-}machine} \wedge$
$x \in \mathit{list}(\mathit{symbol}) \wedge$
$(\exists\, n{\in}\mathit{nat}.\ \exists\, r.$
$\mathit{finite\text{-}run}(M,n,r) \wedge$
$r`0 = \mathit{initial\text{-}config}(x) \wedge$
$\mathit{fst}(r`n) = \mathit{final\text{-}state})$
using *tm-machine input time run final-run* **by** *blast*
next
assume *witness*:
$M \in \mathit{tm\text{-}machine} \wedge$
$x \in \mathit{list}(\mathit{symbol}) \wedge$
$(\exists\, n{\in}\mathit{nat}.\ \exists\, r.$
$\mathit{finite\text{-}run}(M,n,r) \wedge$

```
      r‘0 = initial-config(x) ∧
      fst(r‘n) = final-state)
  then obtain n r where
    tm-machine: M ∈ tm-machine
    and input: x ∈ list(symbol)
    and time: n ∈ nat
    and run: finite-run(M,n,r)
    and initial: r‘0 = initial-config(x)
    and final: fst(r‘n) = final-state
    by auto
  from finite-run-iff-steps[OF time, of M r initial-config(x)]
    run initial
  have canonical:
    r = (λk∈succ(n). steps(M,initial-config(x),k))
    by blast
  have endpoint: r‘n = steps(M,initial-config(x),n)
    unfolding canonical using time by simp
  have final-step:
    fst(steps(M,initial-config(x),n)) = final-state
    using final endpoint by simp
  show halts-on(M,x)
    unfolding halts-on-def
    using tm-machine input time final-step by blast
qed

abbreviation halts-blank :: i ⇒ o where
  halts-blank(M) ≡ halts-on(M,[])

end
```

6 Observable Tape Contents

```
theory Turing-Tape
  imports Turing-Machine
begin
```

A finite list represents an infinite blank-ended half-tape. The Isabelle/ZF function nth returns 0 beyond the end of the list, matching the blank symbol. Equality of represented contents therefore ignores trailing blanks. The relation *half-tape-eq* compares observations only. Its uses as a tape interface state list membership separately. The relations *tape-eq* and *config-eq* include the validity of both arguments.

```
definition half-tape-eq :: [i,i] ⇒ o where
  half-tape-eq(xs,ys) ≡ ∀ n∈nat. nth(n,xs) = nth(n,ys)

lemma half-tape-eq-refl [simp]: half-tape-eq(xs,xs)
  unfolding half-tape-eq-def by simp
```

lemma *half-tape-eq-sym*: *half-tape-eq*(*xs*,*ys*) $\Longrightarrow$ *half-tape-eq*(*ys*,*xs*)
unfolding *half-tape-eq-def* **by** *auto*

lemma *half-tape-eq-trans*:
⟦*half-tape-eq*(*xs*,*ys*); *half-tape-eq*(*ys*,*zs*)⟧ $\Longrightarrow$ *half-tape-eq*(*xs*,*zs*)
unfolding *half-tape-eq-def* **by** *auto*

lemma *nth-zero-hd*:
xs $\in$ *list*(*A*) $\Longrightarrow$ *nth*(*0*,*xs*) = *hd*(*xs*)
by (*erule list.cases*) *auto*

lemma *nth-succ-tl*:
⟦*xs* $\in$ *list*(*A*); *n* $\in$ *nat*⟧ $\Longrightarrow$ *nth*(*succ*(*n*),*xs*) = *nth*(*n*,*tl*(*xs*))
by (*erule list.cases*) *auto*

lemma *half-tape-eq-hd*:
⟦*half-tape-eq*(*xs*,*ys*); *xs* $\in$ *list*(*A*); *ys* $\in$ *list*(*B*)⟧ $\Longrightarrow$ *hd*(*xs*) = *hd*(*ys*)
unfolding *half-tape-eq-def*
by (*drule-tac x=0* **in** *bspec*) (*auto simp add*: *nth-zero-hd*)

lemma *half-tape-eq-tl*:
assumes *eq*: *half-tape-eq*(*xs*,*ys*)
and *xs*: *xs* $\in$ *list*(*A*) **and** *ys*: *ys* $\in$ *list*(*B*)
shows *half-tape-eq*(*tl*(*xs*),*tl*(*ys*))
unfolding *half-tape-eq-def*
proof (*intro ballI*)
fix *n* **assume** *n*: *n* $\in$ *nat*
from *eq n* **have** *nth*(*succ*(*n*),*xs*) = *nth*(*succ*(*n*),*ys*)
unfolding *half-tape-eq-def* **by** *auto*
with *xs ys n* **show** *nth*(*n*,*tl*(*xs*)) = *nth*(*n*,*tl*(*ys*))
by (*simp add*: *nth-succ-tl*)
qed

lemma *half-tape-eq-Cons*:
assumes *eq*: *half-tape-eq*(*xs*,*ys*)
shows *half-tape-eq*(*Cons*(*a*,*xs*),*Cons*(*a*,*ys*))
unfolding *half-tape-eq-def*
proof (*intro ballI*)
fix *n* **assume** *n* $\in$ *nat*
then show *nth*(*n*,*Cons*(*a*,*xs*)) = *nth*(*n*,*Cons*(*a*,*ys*))
proof (*induct n rule*: *nat-induct*)
case *0* **then show** *?case* **by** *simp*
next
case (*succ n*)
with *eq* **show** *?case* **unfolding** *half-tape-eq-def* **by** *auto*
qed
qed

lemma *half-tape-eq-blank* [*simp*]: *half-tape-eq*([*blank-symbol*],[])

unfolding *half-tape-eq-def*
by (*auto elim*: *natE*)

definition *tape-eq* :: [*i*,*i*] ⇒ *o* **where**
tape-eq(*t*,*u*) ≡ *t* ∈ *tm-tape* ∧ *u* ∈ *tm-tape* ∧
half-tape-eq(*fst*(*t*),*fst*(*u*)) ∧ *half-tape-eq*(*snd*(*t*),*snd*(*u*))

lemma *tape-eq-refl*: *t* ∈ *tm-tape* ⟹ *tape-eq*(*t*,*t*)
unfolding *tape-eq-def* **by** *simp*

lemma *tape-eq-sym*: *tape-eq*(*t*,*u*) ⟹ *tape-eq*(*u*,*t*)
unfolding *tape-eq-def* **using** *half-tape-eq-sym* **by** *blast*

lemma *tape-eq-trans*:
⟦*tape-eq*(*t*,*u*); *tape-eq*(*u*,*v*)⟧ ⟹ *tape-eq*(*t*,*v*)
unfolding *tape-eq-def* **using** *half-tape-eq-trans* **by** *blast*

lemma *tape-eq-scan*: *tape-eq*(*t*,*u*) ⟹ *scan*(*t*) = *scan*(*u*)
unfolding *tape-eq-def scan-def*
by (*blast intro*: *half-tape-eq-hd dest*: *tapeD*)

lemma *tape-eq-update*:
assumes *eq*: *tape-eq*(*t*,*u*)
shows *tape-eq*(*tm-update*(*a*,*t*),*tm-update*(*a*,*u*))
proof −
from *eq* **have** *t*: *t* ∈ *tm-tape* **and** *u*: *u* ∈ *tm-tape*
and *left*: *half-tape-eq*(*fst*(*t*),*fst*(*u*))
and *right*: *half-tape-eq*(*snd*(*t*),*snd*(*u*))
unfolding *tape-eq-def* **by** *auto*
from *tapeD*[*OF t*] *tapeD*[*OF u*] *left right*
have *heads*: *hd*(*fst*(*t*)) = *hd*(*fst*(*u*)) *hd*(*snd*(*t*)) = *hd*(*snd*(*u*))
and *tails*: *half-tape-eq*(*tl*(*fst*(*t*)),*tl*(*fst*(*u*)))
half-tape-eq(*tl*(*snd*(*t*)),*tl*(*snd*(*u*)))
by (*auto intro*: *half-tape-eq-hd half-tape-eq-tl*)
from *tm-update-type*[*OF t*] *tm-update-type*[*OF u*]
show *?thesis*
unfolding *tape-eq-def*
using *left right heads tails*
by (*auto simp add*: *tm-update-def intro*: *half-tape-eq-Cons*)
qed

definition *config-eq* :: [*i*,*i*] ⇒ *o* **where**
config-eq(*c*,*d*) ≡ *c* ∈ *configuration* ∧ *d* ∈ *configuration* ∧
fst(*c*) = *fst*(*d*) ∧ *tape-eq*(*snd*(*c*),*snd*(*d*))

lemma *config-eq-refl*: *c* ∈ *configuration* ⟹ *config-eq*(*c*,*c*)
unfolding *config-eq-def configuration-def*
by (*auto intro*: *tape-eq-refl*)

lemma *config-eq-sym*: *config-eq*(*c*,*d*) $\Longrightarrow$ *config-eq*(*d*,*c*)
unfolding *config-eq-def tape-eq-def half-tape-eq-def* **by** *auto*

lemma *config-eq-trans*:
⟦*config-eq*(*c*,*d*); *config-eq*(*d*,*e*)⟧ $\Longrightarrow$ *config-eq*(*c*,*e*)
unfolding *config-eq-def tape-eq-def half-tape-eq-def* **by** *auto*

lemma *config-eq-pair*:
⟦*q* ∈ *nat*; *tape-eq*(*t*,*u*)⟧ $\Longrightarrow$ *config-eq*(⟨*q*,*t*⟩,⟨*q*,*u*⟩)
unfolding *config-eq-def tape-eq-def configuration-def* **by** *auto*

lemma *config-eq-observe*:
config-eq(*c*,*d*) $\Longrightarrow$ *fst*(*c*) = *fst*(*d*)
config-eq(*c*,*d*) $\Longrightarrow$ *scan*(*snd*(*c*)) = *scan*(*snd*(*d*))
unfolding *config-eq-def* **by** (*auto intro*: *tape-eq-scan*)

lemma *step-config-eq*:
assumes *M*: *M* ∈ *tm-machine* **and** *eq*: *config-eq*(*c*,*d*)
shows *config-eq*(*tm-step*(*M*,*c*),*tm-step*(*M*,*d*))
proof −
from *eq* **have** *c*: *c* ∈ *configuration* **and** *d*: *d* ∈ *configuration*
and *state*: *fst*(*c*) = *fst*(*d*) **and** *tapes*: *tape-eq*(*snd*(*c*),*snd*(*d*))
unfolding *config-eq-def* **by** *auto*
from *tape-eq-scan*[*OF tapes*] *state* **have** *fetch*:
fetch(*M*,*fst*(*c*),*scan*(*snd*(*c*))) = *fetch*(*M*,*fst*(*d*),*scan*(*snd*(*d*))) **by** *simp*
from *step-type*[*OF M c*] *step-type*[*OF M d*] *tape-eq-update*[*OF tapes*] *fetch*
show *?thesis* **unfolding** *config-eq-def tm-step-def Let-def* **by** *simp*
qed

theorem *steps-config-eq*:
assumes *M*: *M* ∈ *tm-machine* **and** *eq*: *config-eq*(*c*,*d*) **and** *n*: *n* ∈ *nat*
shows *config-eq*(*steps*(*M*,*c*,*n*),*steps*(*M*,*d*,*n*))
using *n* **by** (*induct n rule*: *nat-induct*) (*auto intro*: *eq step-config-eq*[*OF M*])

6.1 Input Representation and Halting

lemma *initial-config-eq*:
assumes *x*: *x* ∈ *list*(*symbol*) **and** *y*: *y* ∈ *list*(*symbol*)
and *eq*: *half-tape-eq*(*x*,*y*)
shows *config-eq*(*initial-config*(*x*),*initial-config*(*y*))
using *assms* **unfolding** *config-eq-def configuration-def tape-eq-def tm-tape-def*
by *auto*

theorem *halts-on-half-tape-eq-iff*:
assumes *P*: *P* ∈ *tm-machine*
and *x*: *x* ∈ *list*(*symbol*) **and** *y*: *y* ∈ *list*(*symbol*)
and *eq*: *half-tape-eq*(*x*,*y*)
shows *halts-on*(*P*,*x*) $\longleftrightarrow$ *halts-on*(*P*,*y*)
proof −

have *states*: $\bigwedge n.\ n \in nat \Longrightarrow$
fst(*steps*(*P*,*initial-config*(*x*),*n*)) = *fst*(*steps*(*P*,*initial-config*(*y*),*n*))
by (*rule config-eq-observe*(*1*), *rule steps-config-eq*[*OF P*
initial-config-eq[*OF x y eq*]])
from *P x y states* **show** *?thesis* **unfolding** *halts-on-def* **by** *simp*
qed

6.2 Tape Span and Trailing Blanks

definition *tape-span* :: $i \Rightarrow i$ **where**
tape-span(*t*) ≡ *length*(*fst*(*t*)) #+ *length*(*snd*(*t*))

lemma *tm-update-span-mono*:
assumes *t*: $t \in$ *tm-tape*
shows *tape-span*(*t*) ≤ *tape-span*(*tm-update*(*a*,*t*))
proof −
from *tapeD*[*OF t*] **have** *l*: *fst*(*t*) ∈ *list*(*symbol*) **and** *r*: *snd*(*t*) ∈ *list*(*symbol*) **.**
from *l r* **show** *?thesis*
unfolding *tape-span-def tm-update-def*
by (*erule-tac a*=*fst*(*t*) **in** *list.cases*;
erule-tac a=*snd*(*t*) **in** *list.cases*;
auto simp add: *add-commute*)
qed

lemma *step-span-mono*:
c ∈ *configuration* ⟹ *tape-span*(*snd*(*c*)) ≤ *tape-span*(*snd*(*tm-step*(*M*,*c*)))
unfolding *tm-step-def Let-def*
by (*auto intro*: *tm-update-span-mono simp add*: *configuration-def*)

lemma *steps-span-mono*:
assumes *M*: $M \in$ *tm-machine* **and** *c*: $c \in$ *configuration* **and** *n*: $n \in nat$
shows *tape-span*(*snd*(*c*)) ≤ *tape-span*(*snd*(*steps*(*M*,*c*,*n*)))
using *n*
proof (*induct n rule*: *nat-induct*)
case *0*
with *tapeD c* **show** *?case* **unfolding** *tape-span-def configuration-def* **by** *auto*
next
case (*succ n*)
from *step-span-mono*[*OF steps-type*[*OF M c succ.hyps*(*1*)]] *succ.hyps*(*2*)
show *?case* **by** (*auto intro*: *le-trans*)
qed

lemma *half-tape-eq-blank-repeat*:
assumes *n*: $n \in nat$
shows *half-tape-eq*(*repeat*(*blank-symbol*,*n*),[])
using *n*
proof (*induct n rule*: *nat-induct*)
case *0* **then show** *?case* **by** *simp*
next

case (*succ n*)
from *half-tape-eq-Cons*[*OF succ.hyps*(*2*), *of blank-symbol*]
half-tape-eq-blank
show *?case* **by** (*auto intro*: *half-tape-eq-trans*)
qed

lemma *half-tape-eq-append*:
⟦*xs* ∈ *list*(*symbol*); *half-tape-eq*(*ys*,*zs*)⟧ ⟹ *half-tape-eq*(*xs* @ *ys*,*xs* @ *zs*)
by (*induct xs rule*: *list.induct*) (*auto intro*: *half-tape-eq-Cons*)

lemma *half-tape-eq-trailing-blank*:
xs ∈ *list*(*symbol*) ⟹ *half-tape-eq*(*xs* @ [*blank-symbol*],*xs*)
using *half-tape-eq-append*[*OF* - *half-tape-eq-blank*] **by** *auto*

end

7 Using the Machine Interface

theory *Turing-Examples*
imports *Turing-Tape*
begin

The machine scans right across a unary input, writes one at the first blank cell, and halts there. Its two instructions belong to state 1. The first is selected on blank, the second on one. The result theorem specifies the complete final configuration, including the head position. It does not impose a convention for returning the head to the input origin.

definition *scan-append-machine* :: *i* **where**
scan-append-machine ≡
[⟨*write-one*,*final-state*⟩,⟨*move-right*,*initial-state*⟩]

lemma *scan-append-machine-type* [*TC*]: *scan-append-machine* ∈ *tm-machine*
unfolding *scan-append-machine-def tm-machine-def instruction-def* **by** *typecheck*

lemma *scan-append-fetch-blank* [*simp*]:
fetch(*scan-append-machine*,*initial-state*,*blank-symbol*) =
⟨*write-one*,*final-state*⟩
by (*simp add*: *scan-append-machine-def fetch-def slot-def*)

lemma *scan-append-fetch-one* [*simp*]:
fetch(*scan-append-machine*,*initial-state*,*one-symbol*) =
⟨*move-right*,*initial-state*⟩
by (*simp add*: *scan-append-machine-def fetch-def slot-def*)

The loop invariant allows an arbitrary stored left half-tape. Each step transfers one input symbol to that half. Generalising the left half before induction makes the invariant applicable after the first step.

```
lemma scan-append-loop:
  assumes n: n ∈ nat and left: l ∈ list(symbol)
  shows steps(scan-append-machine,
      ⟨initial-state,⟨l,numeral-input(n)⟩⟩,n) =
    ⟨initial-state,⟨numeral-input(n) @ l,[]⟩⟩
  using n left
proof (induct n arbitrary: l rule: nat-induct)
  case 0
  then show ?case by (simp add: numeral-input-def)
next
  case (succ n)
  have first: steps(scan-append-machine,
      ⟨initial-state,⟨l,numeral-input(succ(n))⟩⟩,1) =
    ⟨initial-state,⟨Cons(one-symbol,l),numeral-input(n)⟩⟩
    by (simp add: numeral-input-def)
  have left: Cons(one-symbol,l) ∈ list(symbol)
    using succ.prems by typecheck
  have rearrange:
    numeral-input(n) @ Cons(one-symbol,l) = numeral-input(succ(n)) @ l
    using repeat-succ-app[OF succ.hyps(1), of one-symbol]
      numeral-input-type[OF succ.hyps(1)]
    unfolding numeral-input-def by (simp add: app-assoc)
  from steps-add[OF nat-1I succ.hyps(1), of scan-append-machine
      ⟨initial-state,⟨l,numeral-input(succ(n))⟩⟩]
    first succ.hyps(2)[OF left] rearrange succ.hyps(1)
  show ?case by simp
qed

theorem scan-append-result:
  assumes n ∈ nat
  shows steps(scan-append-machine,initial-config(numeral-input(n)),succ(n)) =
    ⟨final-state,⟨numeral-input(n),[one-symbol]⟩⟩
  using scan-append-loop[OF assms, of []] numeral-input-type[OF assms]
  by simp

corollary scan-append-halts:
  assumes n ∈ nat
  shows halts-on(scan-append-machine,numeral-input(n))
  by (rule halts-onI[OF scan-append-machine-type numeral-input-type[OF assms]
        nat-succI[OF assms]]) (simp only: scan-append-result[OF assms] fst-conv)
```

7.1 Changing the finite representation

Appending a stored blank does not change the observed run. The proof transfers *scan-append-result* through configuration equivalence. It uses the observation interface without repeating the scan induction.

```
theorem scan-append-padded-result:
  assumes n: n ∈ nat
  shows config-eq(
```

```
    steps(scan-append-machine,
      initial-config(numeral-input(n) @ [blank-symbol]),succ(n)),
    ⟨final-state,⟨numeral-input(n),[one-symbol]⟩⟩)
proof −
  have input: numeral-input(n) ∈ list(symbol) by (rule numeral-input-type[OF n])
  have eq: config-eq(initial-config(numeral-input(n) @ [blank-symbol]),
      initial-config(numeral-input(n)))
    using input half-tape-eq-trailing-blank[OF input]
    unfolding config-eq-def configuration-def tape-eq-def tm-tape-def by auto
  from steps-config-eq[OF scan-append-machine-type eq nat-succI[OF n]]
    scan-append-result[OF n]
  show ?thesis by simp
qed

end
```

8 Machine Coding

```
theory Turing-Coding
  imports Turing-Machine
begin
```

8.1 Natural Pairing

The pairing code enumerates diagonals of nat times nat. The inverse follows the same diagonals by iteration; the two directions are proved inverse on natural numbers.

```
consts triangle :: i ⇒ i

primrec
  triangle(0) = 0
  triangle(succ(n)) = triangle(n) #+ succ(n)

definition pair-code :: [i,i] ⇒ i where
  pair-code(m,n) ≡ triangle(m #+ n) #+ n

definition pair-step :: i ⇒ i where
  pair-step(p) ≡
    if fst(p) = 0 then
      ⟨succ(snd(p)),0⟩
    else
      ⟨pred(fst(p)),succ(snd(p))⟩

consts pair-decode :: i ⇒ i

primrec
  pair-decode(0) = ⟨0,0⟩
  pair-decode(succ(n)) = pair-step(pair-decode(n))
```

definition *pair-number* :: $i \Rightarrow i$ **where**
  pair-number(p) $\equiv$ *pair-code*(*fst*(p),*snd*(p))

lemma *triangle-type* [*TC*]:
  assumes *natural*: $n \in nat$
  shows *triangle*(n) $\in nat$
  using *natural* **by** (*induct n rule*: *nat-induct*) *simp-all*

lemma *pair-code-type* [*TC*]:
  assumes *first*: $m \in nat$
    and *second*: $n \in nat$
  shows *pair-code*(m,n) $\in nat$
  using *first second* **unfolding** *pair-code-def* **by** *typecheck*

lemma *pair-step-type* [*TC*]:
  assumes *pair*: $p \in nat \times nat$
  shows *pair-step*(p) $\in nat \times nat$
proof −
  from *pair* **have** *first*: *fst*(p) $\in nat$
    and *second*: *snd*(p) $\in nat$
    by *typecheck+*
  show *?thesis*
    unfolding *pair-step-def* **using** *first second* **by** *typecheck*
qed

lemma *pair-decode-type* [*TC*]:
  assumes *natural*: $n \in nat$
  shows *pair-decode*(n) $\in nat \times nat$
  using *natural* **by** (*induct n rule*: *nat-induct*) (*simp-all add*: *pair-step-type*)

lemma *pair-number-type* [*TC*]:
  assumes *pair*: $p \in nat \times nat$
  shows *pair-number*(p) $\in nat$
  using *pair* **unfolding** *pair-number-def* **by** *typecheck*

lemma *pair-number-step*:
  assumes *pair*: $p \in nat \times nat$
  shows *pair-number*(*pair-step*(p)) = *succ*(*pair-number*(p))
proof −
  from *pair* **have** *first*: *fst*(p) $\in nat$
    and *second*: *snd*(p) $\in nat$
    by *typecheck+*
  from *first* **show** *?thesis*
  proof (*cases rule*: *natE*)
    case *0*
    with *second* **show** *?thesis*
      unfolding *pair-number-def pair-step-def pair-code-def*
      by *simp*

```
  next
    case (succ m)
    with second show ?thesis
      unfolding pair-number-def pair-step-def pair-code-def
      by (simp add: add-ac)
  qed
qed

lemma pair-number-decode:
  assumes natural: n ∈ nat
  shows pair-number(pair-decode(n)) = n
  using natural
proof (induct n rule: nat-induct)
  case 0
  then show ?case
    unfolding pair-number-def pair-code-def by simp
next
  case (succ n)
  from succ.hyps(1) have decoded: pair-decode(n) ∈ nat × nat
    by (rule pair-decode-type)
  have pair-number(pair-decode(succ(n))) =
      pair-number(pair-step(pair-decode(n)))
    by simp
  also from decoded have ... = succ(pair-number(pair-decode(n)))
    by (rule pair-number-step)
  also from succ.hyps(2) have ... = succ(n)
    by simp
  finally show ?case .
qed

definition pair-prev :: i ⇒ i where
  pair-prev(p) ≡
    if snd(p) = 0 then
      ⟨0,pred(fst(p))⟩
    else
      ⟨succ(fst(p)),pred(snd(p))⟩

lemma pair-prev-type [TC]:
  assumes pair: p ∈ nat × nat
  shows pair-prev(p) ∈ nat × nat
proof −
  from pair have first: fst(p) ∈ nat
    and second: snd(p) ∈ nat
    by typecheck+
  show ?thesis
    unfolding pair-prev-def using first second by typecheck
qed

lemma triangle-eq-zero:
```

```
  assumes natural: n ∈ nat
  shows triangle(n) = 0 ⟷ n = 0
  using natural by (induct n rule: nat-induct) simp-all

lemma pair-number-zero:
  assumes pair: p ∈ nat × nat
  shows pair-number(p) = 0 ⟷ p = ⟨0,0⟩
proof −
  from pair have first: fst(p) ∈ nat
    and second: snd(p) ∈ nat
    by typecheck+
  from pair have recover: ⟨fst(p),snd(p)⟩ = p
    by (rule Pair-fst-snd-eq)
  show ?thesis
    unfolding pair-number-def pair-code-def
    using first second recover triangle-eq-zero
    by auto
qed

lemma pair-step-prev:
  assumes pair: p ∈ nat × nat
    and nonzero: p ≠ ⟨0,0⟩
  shows pair-step(pair-prev(p)) = p
proof −
  from pair have first: fst(p) ∈ nat
    and second: snd(p) ∈ nat
    by typecheck+
  from pair have recover: ⟨fst(p),snd(p)⟩ = p
    by (rule Pair-fst-snd-eq)
  show ?thesis
  proof (cases snd(p) = 0)
    case True
    from first show ?thesis
    proof (cases rule: natE)
      case 0
      with True nonzero recover show ?thesis by auto
    next
      case (succ m)
      show ?thesis
        unfolding pair-prev-def pair-step-def
        using True succ recover by simp
    qed
  next
    case False
    from second show ?thesis
    proof (cases rule: natE)
      case 0
      with False show ?thesis by contradiction
    next
```

```
    case (succ n)
    show ?thesis
      unfolding pair-prev-def pair-step-def
      using False succ recover by simp
  qed
 qed
qed

lemma pair-decode-number-aux:
 assumes natural: n ∈ nat
 shows ∀ p∈nat × nat.
   pair-number(p) = n ⟶ pair-decode(n) = p
 using natural
proof (induct n rule: nat-induct)
 case 0
 show ?case
 proof (intro ballI impI)
   fix p
   assume pair: p ∈ nat × nat
     and number: pair-number(p) = 0
   from pair-number-zero[OF pair] number have p = ⟨0,0⟩
     by simp
   then show pair-decode(0) = p by simp
 qed
next
 case (succ n)
 show ?case
 proof (intro ballI impI)
   fix p
   assume pair: p ∈ nat × nat
     and number: pair-number(p) = succ(n)
   have nonzero: p ≠ ⟨0,0⟩
   proof
     assume p = ⟨0,0⟩
     with pair-number-zero[OF pair]
     have pair-number(p) = 0 by simp
     with number show False by simp
   qed
   let ?q = pair-prev(p)
   from pair have previous: ?q ∈ nat × nat
     by (rule pair-prev-type)
   from pair nonzero have step-previous: pair-step(?q) = p
     by (rule pair-step-prev)
   have previous-number: pair-number(?q) = n
   proof −
     from pair-number-step[OF previous]
     have succ(pair-number(?q)) = pair-number(pair-step(?q))
       by (rule sym)
     also from step-previous have ... = pair-number(p) by simp
```

```
    also from number have ... = succ(n) .
    finally show ?thesis by simp
  qed
  from succ.hyps(2) previous have previous-imp:
    pair-number(?q) = n ⟶ pair-decode(n) = ?q
    by (rule bspec)
  from previous-imp previous-number
  have decoded-previous: pair-decode(n) = ?q
    by (rule mp)
  have pair-decode(succ(n)) = pair-step(pair-decode(n))
    by simp
  also from decoded-previous have ... = pair-step(?q) by simp
  also from step-previous have ... = p .
  finally show pair-decode(succ(n)) = p .
 qed
qed

lemma pair-decode-number:
 assumes pair: p ∈ nat × nat
 shows pair-decode(pair-number(p)) = p
proof −
 from pair have natural: pair-number(p) ∈ nat
   by (rule pair-number-type)
 from pair-decode-number-aux[OF natural] pair show ?thesis
   by blast
qed

theorem pair-decode-code:
 assumes first: m ∈ nat
   and second: n ∈ nat
 shows pair-decode(pair-code(m,n)) = ⟨m,n⟩
proof −
 from first second have pair: ⟨m,n⟩ ∈ nat × nat
   by typecheck
 from pair-decode-number[OF pair] show ?thesis
   unfolding pair-number-def by simp
qed

theorem pair-code-decode:
 assumes natural: n ∈ nat
 shows pair-code(fst(pair-decode(n)),snd(pair-decode(n))) = n
 using pair-number-decode[OF natural]
 unfolding pair-number-def .

lemma pair-decode-iterates:
 assumes natural: n ∈ nat
 shows pair-decode(n) = pair-step^n (⟨0,0⟩)
 using natural by (induct n rule: nat-induct) simp-all
```

theorem *pair-number-bij*:
  $(\lambda p{\in}nat \times nat.\ pair\text{-}number(p))$
    $\in bij(nat \times nat, nat)$
proof (*rule lam-bijective*)
  fix p
  assume $p \in nat \times nat$
  then show $pair\text{-}number(p) \in nat$
    by (*rule pair-number-type*)
next
  fix n
  assume $n \in nat$
  then show $pair\text{-}decode(n) \in nat \times nat$
    by (*rule pair-decode-type*)
next
  fix p
  assume $p \in nat \times nat$
  then show $pair\text{-}decode(pair\text{-}number(p)) = p$
    by (*rule pair-decode-number*)
next
  fix n
  assume $n \in nat$
  then show $pair\text{-}number(pair\text{-}decode(n)) = n$
    by (*rule pair-number-decode*)
qed

lemma *pair-code-second-le*:
  assumes *first*: $m \in nat$
    and *second*: $n \in nat$
  shows $n \leq pair\text{-}code(m,n)$
  using *first second* **unfolding** *pair-code-def*
  by (*simp add*: *add-le-self2*)

lemma *pair-decode-second-le*:
  assumes *natural*: $n \in nat$
  shows $snd(pair\text{-}decode(n)) \leq n$
proof −
  from *natural* **have** *decoded*: $pair\text{-}decode(n) \in nat \times nat$
    by (*rule pair-decode-type*)
  from *decoded* **have** *first*: $fst(pair\text{-}decode(n)) \in nat$
    and *second*: $snd(pair\text{-}decode(n)) \in nat$
    by *typecheck+*
  from *pair-code-second-le*[*OF first second*]
    pair-code-decode[*OF natural*]
  show *?thesis* **by** *simp*
qed

lemma *pair-decode-second-in-succ*:
  assumes *natural*: $n \in nat$
  shows $snd(pair\text{-}decode(n)) \in succ(n)$

proof −
from *pair-decode-second-le*[*OF natural*]
have *less*: *snd*(*pair-decode*(*n*)) < *succ*(*n*) **.**
from *nat-into-Ord*[*OF natural*] **have** *ordinal*: *Ord*(*succ*(*n*))
by (*rule Ord-succ*)
from *Ord-mem-iff-lt*[*OF ordinal*] *less* **show** *?thesis*
by (*rule iffD2*)
qed

8.2 Natural Lists

consts *nat-list-encode* :: *i* ⇒ *i*

primrec
nat-list-encode([]) = *0*
nat-list-encode(*Cons*(*a*,*l*)) =
succ(*pair-code*(*a*,*nat-list-encode*(*l*)))

definition *nat-list-decode* :: *i* ⇒ *i* **where**
nat-list-decode(*n*) ≡
wfrec(*Memrel*(*nat*),*n*,
λ*k f*. *nat-case*([],
λ*m*. *Cons*(*fst*(*pair-decode*(*m*)),*f*‘*snd*(*pair-decode*(*m*))),*k*))

lemmas *nat-list-decode-unfold* =
def-wfrec [*OF nat-list-decode-def wf-Memrel*]

lemma *nat-list-encode-type* [*TC*]:
assumes *list*: *l* ∈ *list*(*nat*)
shows *nat-list-encode*(*l*) ∈ *nat*
using *list* **by** (*induct l rule*: *list.induct*) *simp-all*

lemma *nat-list-decode-zero* [*simp*]:
nat-list-decode(*0*) = []
by (*rule nat-list-decode-unfold* [*THEN trans*]) *simp*

lemma *nat-list-decode-succ* [*simp*]:
assumes *natural*: *n* ∈ *nat*
shows *nat-list-decode*(*succ*(*n*)) =
Cons(*fst*(*pair-decode*(*n*)),*nat-list-decode*(*snd*(*pair-decode*(*n*))))
proof −
from *natural* **have** *decoded*: *pair-decode*(*n*) ∈ *nat* × *nat*
by (*rule pair-decode-type*)
from *decoded* **have** *tail-natural*: *snd*(*pair-decode*(*n*)) ∈ *nat*
by *typecheck*
from *natural* **have** *tail*: *snd*(*pair-decode*(*n*)) ∈ *succ*(*n*)
by (*rule pair-decode-second-in-succ*)
show *?thesis*
apply (*rule nat-list-decode-unfold* [*THEN trans*])

```
    using natural tail tail-natural
    apply (simp add: vimage-singleton-iff)
    done
qed

lemma nat-list-decode-type [TC]:
  assumes natural: n ∈ nat
  shows nat-list-decode(n) ∈ list(nat)
  using natural
proof (induct n rule: complete-induct-rule)
  case (less n)
  note smaller = less.hyps(2)
  from less.hyps(1) show ?case
  proof (cases rule: natE)
    case 0
    then show ?thesis by simp
  next
    case (succ k)
    from succ(1) have decoded: pair-decode(k) ∈ nat × nat
      by (rule pair-decode-type)
    from decoded have head: fst(pair-decode(k)) ∈ nat
      and tail-natural: snd(pair-decode(k)) ∈ nat
      by typecheck+
    from succ(1) have tail-before:
      snd(pair-decode(k)) ∈ succ(k)
      by (rule pair-decode-second-in-succ)
    from smaller tail-before succ
    have tail: nat-list-decode(snd(pair-decode(k))) ∈ list(nat)
      by blast
    from succ head tail show ?thesis by simp
  qed
qed

theorem nat-list-decode-encode:
  assumes list: l ∈ list(nat)
  shows nat-list-decode(nat-list-encode(l)) = l
  using list
proof (induct l rule: list.induct)
  case Nil
  then show ?case by simp
next
  case (Cons a l)
  from Cons.hyps(2) have tail-code: nat-list-encode(l) ∈ nat
    by (rule nat-list-encode-type)
  from Cons.hyps tail-code show ?case
    by (simp add: pair-decode-code)
qed

theorem nat-list-encode-decode:
```

```
  assumes natural: n ∈ nat
  shows nat-list-encode(nat-list-decode(n)) = n
  using natural
proof (induct n rule: complete-induct-rule)
  case (less n)
  note smaller = less.hyps(2)
  from less.hyps(1) show ?case
  proof (cases rule: natE)
    case 0
    then show ?thesis by simp
  next
    case (succ k)
    from succ(1) have decoded: pair-decode(k) ∈ nat × nat
      by (rule pair-decode-type)
    from decoded have head: fst(pair-decode(k)) ∈ nat
      and tail-natural: snd(pair-decode(k)) ∈ nat
      by typecheck+
    from succ(1) have tail-before:
      snd(pair-decode(k)) ∈ succ(k)
      by (rule pair-decode-second-in-succ)
    from smaller tail-before succ have tail-roundtrip:
      nat-list-encode(nat-list-decode(snd(pair-decode(k)))) =
        snd(pair-decode(k))
      by blast
    from pair-code-decode[OF succ(1)] have code-roundtrip:
      pair-code(fst(pair-decode(k)),snd(pair-decode(k))) = k .
    from succ head tail-natural tail-roundtrip code-roundtrip
    show ?thesis by simp
  qed
qed

lemma length-nat-list-encode-le:
  assumes list: l ∈ list(nat)
  shows length(l) ≤ nat-list-encode(l)
  using list
proof (induct l rule: list.induct)
  case Nil
  then show ?case by simp
next
  case (Cons a l)
  from Cons.hyps(2) have tail-code: nat-list-encode(l) ∈ nat
    by (rule nat-list-encode-type)
  from pair-code-second-le[OF Cons.hyps(1) tail-code] have code-bound:
    nat-list-encode(l) ≤ pair-code(a,nat-list-encode(l)) .
  from Cons.hyps(3) code-bound have length-bound:
    length(l) ≤ pair-code(a,nat-list-encode(l))
    by (rule le-trans)
  from length-bound show ?case by simp
qed
```

lemma *length-nat-list-decode-le*:
assumes *natural*: $n \in nat$
shows $length(nat\text{-}list\text{-}decode(n)) \leq n$
proof −
from *natural* **have** *decoded*: $nat\text{-}list\text{-}decode(n) \in list(nat)$
by (*rule nat-list-decode-type*)
from *length-nat-list-encode-le*[*OF decoded*]
nat-list-encode-decode[*OF natural*]
show *?thesis* **by** *simp*
qed

theorem *nat-list-encode-bij*:
$(\lambda l \in list(nat).\ nat\text{-}list\text{-}encode(l))$
$\in bij(list(nat), nat)$
proof (*rule lam-bijective*)
fix l
assume $l \in list(nat)$
then show $nat\text{-}list\text{-}encode(l) \in nat$
by (*rule nat-list-encode-type*)
next
fix n
assume $n \in nat$
then show $nat\text{-}list\text{-}decode(n) \in list(nat)$
by (*rule nat-list-decode-type*)
next
fix l
assume $l \in list(nat)$
then show $nat\text{-}list\text{-}decode(nat\text{-}list\text{-}encode(l)) = l$
by (*rule nat-list-decode-encode*)
next
fix n
assume $n \in nat$
then show $nat\text{-}list\text{-}encode(nat\text{-}list\text{-}decode(n)) = n$
by (*rule nat-list-encode-decode*)
qed

8.3 Instruction Streams

An instruction code whose action field is outside the five valid actions decodes to a no-op instruction targeting the final state. Thus decoding is defined for every natural number. The canonical codes are identified separately.

definition *encode-instruction* :: $i \Rightarrow i$ **where**
$encode\text{-}instruction(ins) \equiv pair\text{-}code(fst(ins), snd(ins))$

definition *decode-instruction* :: $i \Rightarrow i$ **where**
$decode\text{-}instruction(n) \equiv$
if $n \in nat \wedge fst(pair\text{-}decode(n)) \in action$ *then*

```
    pair-decode(n)
  else
    ⟨nop,final-state⟩

definition instruction-code :: i where
  instruction-code ≡
    {n∈nat. fst(pair-decode(n)) ∈ action}

lemma instruction-code-subset-nat:
  instruction-code ⊆ nat
  unfolding instruction-code-def by blast

lemma encode-instruction-type [TC]:
  assumes instruction: ins ∈ instruction
  shows encode-instruction(ins) ∈ instruction-code
proof −
  from instruction have action-field: fst(ins) ∈ action
    and state-field: snd(ins) ∈ nat
    unfolding instruction-def by typecheck+
  from action-field action-subset-nat have natural-action: fst(ins) ∈ nat
    by blast
  from natural-action state-field have code-natural:
    encode-instruction(ins) ∈ nat
    unfolding encode-instruction-def by typecheck
  from pair-decode-code[OF natural-action state-field] action-field
    code-natural
  show ?thesis
    unfolding instruction-code-def encode-instruction-def by simp
qed

lemma decode-instruction-type [TC]:
  assumes natural: n ∈ nat
  shows decode-instruction(n) ∈ instruction
proof −
  from natural have decoded: pair-decode(n) ∈ nat × nat
    by (rule pair-decode-type)
  from decoded have pair-recover:
    ⟨fst(pair-decode(n)),snd(pair-decode(n))⟩ = pair-decode(n)
    by (rule Pair-fst-snd-eq)
  show ?thesis
    unfolding decode-instruction-def instruction-def
    using natural decoded pair-recover by auto
qed

theorem decode-encode-instruction:
  assumes instruction: ins ∈ instruction
  shows decode-instruction(encode-instruction(ins)) = ins
proof −
  from instruction have action-field: fst(ins) ∈ action
```

```
    and state-field: snd(ins) ∈ nat
    unfolding instruction-def by typecheck+
  from action-field action-subset-nat have natural-action: fst(ins) ∈ nat
    by blast
  from natural-action state-field have natural-code:
    pair-code(fst(ins),snd(ins)) ∈ nat
    by (rule pair-code-type)
  from instruction have recover: ⟨fst(ins),snd(ins)⟩ = ins
    unfolding instruction-def by (rule Pair-fst-snd-eq)
  show ?thesis
    unfolding decode-instruction-def encode-instruction-def
    using pair-decode-code[OF natural-action state-field]
      natural-code action-field recover
    by simp
qed

theorem encode-decode-instruction:
  assumes code: n ∈ instruction-code
  shows encode-instruction(decode-instruction(n)) = n
proof −
  from code have natural: n ∈ nat
    and action-field: fst(pair-decode(n)) ∈ action
    unfolding instruction-code-def by auto
  show ?thesis
    unfolding encode-instruction-def decode-instruction-def
    using pair-code-decode[OF natural] natural action-field by simp
qed

definition machine-code-list :: i ⇒ i where
  machine-code-list(M) ≡ map(encode-instruction,M)

definition decode-machine-list :: i ⇒ i where
  decode-machine-list(l) ≡ map(decode-instruction,l)

lemma machine-code-list-type [TC]:
  assumes machine-M: M ∈ tm-machine
  shows machine-code-list(M) ∈ list(instruction-code)
proof −
  from machine-M have instructions: M ∈ list(instruction)
    unfolding tm-machine-def .
  show ?thesis
    unfolding machine-code-list-def
    using instructions by (rule map-type) (rule encode-instruction-type)
qed

lemma machine-code-list-natural:
  assumes machine-M: M ∈ tm-machine
  shows machine-code-list(M) ∈ list(nat)
proof −
```

from *machine-code-list-type*[*OF machine-M*]
show *?thesis*
by (*rule list-mono*[*OF instruction-code-subset-nat*, *THEN subsetD*])
qed

lemma *decode-machine-list-type* [*TC*]:
assumes *codes*: $l \in list(nat)$
shows *decode-machine-list*(l) $\in$ *tm-machine*
proof −
have *map*(*decode-instruction*,l) $\in$ *list*(*instruction*)
using *codes* **by** (*rule map-type*) (*rule decode-instruction-type*)
then show *?thesis*
unfolding *decode-machine-list-def tm-machine-def* **.**
qed

theorem *decode-machine-list-code*:
assumes *machine-M*: $M \in$ *tm-machine*
shows *decode-machine-list*(*machine-code-list*(M)) = M
using *machine-M*
unfolding *tm-machine-def machine-code-list-def decode-machine-list-def*
proof (*induct M rule*: *list.induct*)
case *Nil*
then show *?case* **by** *simp*
next
case (*Cons ins M*)
then show *?case* **by** (*simp add*: *decode-encode-instruction*)
qed

theorem *machine-code-list-decode*:
assumes *codes*: $l \in list(instruction\text{-}code)$
shows *machine-code-list*(*decode-machine-list*(l)) = l
using *codes* **unfolding** *machine-code-list-def decode-machine-list-def*
proof (*induct l rule*: *list.induct*)
case *Nil*
then show *?case* **by** *simp*
next
case (*Cons n l*)
from *Cons* **show** *?case* **by** (*simp add*: *encode-decode-instruction*)
qed

8.4 Machine Numbers

Machines are encoded as finite lists of instruction numbers. Canonical codes give a bijection with machines, while the total decoder also accepts non-canonical natural numbers. The latter convention lets the self-halting predicate quantify over all natural numbers.

definition *encode-machine* :: $i \Rightarrow i$ **where**
encode-machine(M) $\equiv$ *nat-list-encode*(*machine-code-list*(M))

definition *decode-machine* :: $i \Rightarrow i$ **where**
  decode-machine$(n) \equiv$
    decode-machine-list(*nat-list-decode*(n))

definition *machine-code* :: i **where**
  machine-code $\equiv$
    $\{n \in nat.$ *nat-list-decode*$(n) \in$ *list*(*instruction-code*)$\}$

lemma *machine-code-subset-nat*:
  machine-code $\subseteq$ *nat*
  unfolding *machine-code-def* **by** *blast*

lemma *encode-machine-type* [*TC*]:
  assumes *machine-M*: $M \in$ *tm-machine*
  shows *encode-machine*$(M) \in$ *machine-code*
proof −
  from *machine-code-list-type*[*OF machine-M*] **have** *codes*:
    machine-code-list$(M) \in$ *list*(*instruction-code*) **.**
  from *machine-code-list-natural*[*OF machine-M*] **have** *natural-codes*:
    machine-code-list$(M) \in$ *list*(*nat*) **.**
  from *natural-codes*
  have *natural*: *encode-machine*$(M) \in$ *nat*
    unfolding *encode-machine-def* **by** (*rule nat-list-encode-type*)
  from *nat-list-decode-encode*[*OF natural-codes*] **have** *decoded*:
    nat-list-decode(*nat-list-encode*(*machine-code-list*(M))) =
      machine-code-list(M) **.**
  from *codes natural decoded* **show** *?thesis*
    unfolding *machine-code-def encode-machine-def* **by** *simp*
qed

lemma *encode-machine-natural* [*TC*]:
  $P \in$ *tm-machine* $\Longrightarrow$ *encode-machine*$(P) \in$ *nat*
  using *encode-machine-type machine-code-subset-nat* **by** *blast*

lemma *decode-machine-type* [*TC*]:
  assumes *natural*: $n \in$ *nat*
  shows *decode-machine*$(n) \in$ *tm-machine*
proof −
  from *natural* **have** *nat-list-decode*$(n) \in$ *list*(*nat*)
    by (*rule nat-list-decode-type*)
  then show *?thesis*
    unfolding *decode-machine-def* **by** (*rule decode-machine-list-type*)
qed

theorem *decode-encode-machine*:
  assumes *machine-M*: $M \in$ *tm-machine*
  shows *decode-machine*(*encode-machine*$(M)) = M$
proof −
  from *machine-code-list-natural*[*OF machine-M*] **show** *?thesis*

unfolding *decode-machine-def encode-machine-def*
by (*simp add*: *nat-list-decode-encode decode-machine-list-code machine-M*)
qed

theorem *encode-decode-machine*:
assumes *code*: $n \in$ *machine-code*
shows *encode-machine*(*decode-machine*(n)) = n
proof −
from *code* **have** *natural*: $n \in$ *nat*
and *codes*: *nat-list-decode*(n) ∈ *list*(*instruction-code*)
unfolding *machine-code-def* **by** *auto*
show *?thesis*
unfolding *encode-machine-def decode-machine-def*
using *machine-code-list-decode*[*OF codes*]
nat-list-encode-decode[*OF natural*]
by *simp*
qed

theorem *machine-numbering-bij*:
(λ*M*∈*tm-machine*. *encode-machine*(*M*))
∈ *bij*(*tm-machine*,*machine-code*)
proof (*rule lam-bijective*)
fix *M*
assume $M \in$ *tm-machine*
then show *encode-machine*(M) ∈ *machine-code*
by (*rule encode-machine-type*)
next
fix n
assume $n \in$ *machine-code*
with *machine-code-subset-nat* **have** $n \in$ *nat* **by** *blast*
then show *decode-machine*(n) ∈ *tm-machine*
by (*rule decode-machine-type*)
next
fix *M*
assume $M \in$ *tm-machine*
then show *decode-machine*(*encode-machine*(M)) = M
by (*rule decode-encode-machine*)
next
fix n
assume $n \in$ *machine-code*
then show *encode-machine*(*decode-machine*(n)) = n
by (*rule encode-decode-machine*)
qed

theorem *decode-machine-surj*:
(λ*n*∈*nat*. *decode-machine*(n))
∈ *surj*(*nat*,*tm-machine*)
proof (*rule lam-surjective*)

fix n
assume $n \in nat$
then show *decode-machine*$(n) \in$ *tm-machine*
by (*rule decode-machine-type*)
next
fix M
assume $M \in$ *tm-machine*
then have *encode-machine*$(M) \in$ *machine-code*
by (*rule encode-machine-type*)
with *machine-code-subset-nat* **show** *encode-machine*$(M) \in nat$
by *blast*
next
fix M
assume $M \in$ *tm-machine*
then show *decode-machine*(*encode-machine*$(M)) = M$
by (*rule decode-encode-machine*)
qed

end

9 Operations on Machine and List Codes

theory *Turing-Code-Operations*
imports *Turing-Coding*
begin

The operations below act directly on natural-number codes. Their correctness lemmas relate them to pairing and list operations after decoding. The arithmetic definitions are also used by the primitive-recursive certificates.

consts *pair-diagonal* :: $i \Rightarrow i$

primrec
pair-diagonal$(0) = 0$
pair-diagonal(*succ*$(n)) =$
(*if* $n =$ *triangle*(*pair-diagonal*$(n)) \#+$ *pair-diagonal*(n) *then*
succ(*pair-diagonal*$(n))$
else
pair-diagonal$(n))$

lemma *pair-diagonal-type* [*TC*]:
assumes *natural*: $n \in nat$
shows *pair-diagonal*$(n) \in nat$
using *natural* **by** (*induct n rule*: *nat-induct*) *simp-all*

lemma *pair-diagonal-decode*:
assumes *natural*: $n \in nat$
shows *pair-diagonal*$(n) =$

```
    fst(pair-decode(n)) #+ snd(pair-decode(n))
  using natural
proof (induct n rule: nat-induct)
  case 0
  then show ?case by simp
next
  case (succ n)
  from succ.hyps(1) have decoded: pair-decode(n) ∈ nat × nat
    by (rule pair-decode-type)
  from decoded have first: fst(pair-decode(n)) ∈ nat
    and second: snd(pair-decode(n)) ∈ nat
    by typecheck+
  from pair-code-decode[OF succ.hyps(1)] have code:
    pair-code(fst(pair-decode(n)),snd(pair-decode(n))) = n .
  from first show ?case
  proof (cases rule: natE)
    case 0
    from code 0 second have end-of-diagonal:
      n = triangle(snd(pair-decode(n))) #+ snd(pair-decode(n))
      unfolding pair-code-def by simp
    from succ.hyps(2) 0 second have diagonal:
      pair-diagonal(n) = snd(pair-decode(n))
      by simp
    from end-of-diagonal diagonal have at-end:
      n = triangle(pair-diagonal(n)) #+ pair-diagonal(n)
      by simp
    from 0 have next-pair:
      pair-decode(succ(n)) =
        ⟨succ(snd(pair-decode(n))),0⟩
      by (simp add: pair-step-def)
    from at-end have next-diagonal:
      pair-diagonal(succ(n)) = succ(pair-diagonal(n))
      by simp
    from diagonal next-pair next-diagonal second show ?thesis
      by simp
  next
    case (succ m)
    let ?s = fst(pair-decode(n)) #+ snd(pair-decode(n))
    have sum-natural: ?s ∈ nat
      using first second by typecheck
    have not-end:
      n ≠ triangle(?s) #+ ?s
    proof
      assume at-end: n = triangle(?s) #+ ?s
      from code have
        triangle(?s) #+ snd(pair-decode(n)) = n
        unfolding pair-code-def .
      also from at-end have ... = triangle(?s) #+ ?s .
      finally have equal:
```

```
      snd(pair-decode(n)) = ?s
      using second sum-natural
      by (rule add-left-cancel[OF refl])
    from equal succ second show False
      by (simp add: add-ac)
  qed
  from succ.hyps(2) have diagonal:
    pair-diagonal(n) = ?s .
  from not-end diagonal have not-at-end:
    n ≠ triangle(pair-diagonal(n)) #+ pair-diagonal(n)
    by simp
  from succ have next-pair:
    pair-decode(succ(n)) =
      ⟨m,succ(snd(pair-decode(n)))⟩
    by (simp add: pair-step-def)
  from not-at-end have next-diagonal:
    pair-diagonal(succ(n)) = pair-diagonal(n)
    by simp
  from diagonal next-pair next-diagonal succ second show ?thesis
    by (simp add: add-ac)
 qed
qed


definition pair-right-code :: i ⇒ i where
 pair-right-code(n) ≡
   n #− triangle(pair-diagonal(n))

definition pair-left-code :: i ⇒ i where
 pair-left-code(n) ≡
   pair-diagonal(n) #− pair-right-code(n)

lemma pair-right-code-type [TC]:
 pair-right-code(n) ∈ nat
 unfolding pair-right-code-def by typecheck

lemma pair-left-code-type [TC]:
 pair-left-code(n) ∈ nat
 unfolding pair-left-code-def by typecheck

theorem pair-right-code-decode:
 assumes natural: n ∈ nat
 shows pair-right-code(n) = snd(pair-decode(n))
proof −
 from natural have decoded: pair-decode(n) ∈ nat × nat
   by (rule pair-decode-type)
 from decoded have first: fst(pair-decode(n)) ∈ nat
   and second: snd(pair-decode(n)) ∈ nat
   by typecheck+
```

```
  from pair-code-decode[OF natural] have code:
    n = triangle(fst(pair-decode(n)) #+ snd(pair-decode(n))) #+
      snd(pair-decode(n))
    unfolding pair-code-def by (rule sym)
  have pair-right-code(n) =
      (triangle(fst(pair-decode(n)) #+ snd(pair-decode(n))) #+
        snd(pair-decode(n))) #−
      triangle(fst(pair-decode(n)) #+ snd(pair-decode(n)))
    unfolding pair-right-code-def
    using code pair-diagonal-decode[OF natural] by simp
  also from second have ... = snd(pair-decode(n))
    by (simp add: diff-add-inverse)
  finally show ?thesis .
qed

theorem pair-left-code-decode:
  assumes natural: n ∈ nat
  shows pair-left-code(n) = fst(pair-decode(n))
proof −
  from natural have decoded: pair-decode(n) ∈ nat × nat
    by (rule pair-decode-type)
  from decoded have first: fst(pair-decode(n)) ∈ nat
    and second: snd(pair-decode(n)) ∈ nat
    by typecheck+
  show ?thesis
    unfolding pair-left-code-def
    using pair-diagonal-decode[OF natural]
      pair-right-code-decode[OF natural] first second
    by (simp add: diff-add-inverse2 add-ac)
qed

lemma pair-right-code-pair-code:
  assumes first: m ∈ nat
    and second: n ∈ nat
  shows pair-right-code(pair-code(m,n)) = n
  using pair-right-code-decode[OF pair-code-type[OF first second]]
    pair-decode-code[OF first second]
  by simp

lemma pair-left-code-pair-code:
  assumes first: m ∈ nat
    and second: n ∈ nat
  shows pair-left-code(pair-code(m,n)) = m
  using pair-left-code-decode[OF pair-code-type[OF first second]]
    pair-decode-code[OF first second]
  by simp


definition code-cons :: [i,i] ⇒ i where
```

code-cons(*a*,*c*) ≡ *succ*(*pair-code*(*a*,*c*))

definition *code-head* :: *i* ⇒ *i* **where**
code-head(*c*) ≡
if c = *0 then 0 else fst*(*pair-decode*(*pred*(*c*)))

definition *code-tail* :: *i* ⇒ *i* **where**
code-tail(*c*) ≡
if c = *0 then 0 else snd*(*pair-decode*(*pred*(*c*)))

lemma *code-cons-type* [*TC*]:
assumes *element*: *a* ∈ *nat*
and *tail*: *c* ∈ *nat*
shows *code-cons*(*a*,*c*) ∈ *nat*
using *element tail* **unfolding** *code-cons-def* **by** *typecheck*

lemma *code-head-type* [*TC*]:
assumes *natural*: *c* ∈ *nat*
shows *code-head*(*c*) ∈ *nat*
using *natural*
proof (*cases rule*: *natE*)
case *0*
then show *?thesis* **unfolding** *code-head-def* **by** *simp*
next
case (*succ n*)
from *succ*(*1*) **have** *pair-decode*(*n*) ∈ *nat* × *nat*
by (*rule pair-decode-type*)
with *succ natural* **show** *?thesis* **unfolding** *code-head-def* **by** *typecheck*
qed

lemma *code-tail-type* [*TC*]:
assumes *natural*: *c* ∈ *nat*
shows *code-tail*(*c*) ∈ *nat*
using *natural*
proof (*cases rule*: *natE*)
case *0*
then show *?thesis* **unfolding** *code-tail-def* **by** *simp*
next
case (*succ n*)
from *succ*(*1*) **have** *pair-decode*(*n*) ∈ *nat* × *nat*
by (*rule pair-decode-type*)
with *succ natural* **show** *?thesis* **unfolding** *code-tail-def* **by** *typecheck*
qed

lemma *code-cons-encode* [*simp*]:
code-cons(*a*,*nat-list-encode*(*l*)) =
nat-list-encode(*Cons*(*a*,*l*))
unfolding *code-cons-def* **by** *simp*

lemma *code-head-encode*:
assumes *list*: $l \in list(nat)$
shows *code-head*(*nat-list-encode*(l)) = $hd(l)$
using *list*
proof (*induct l rule*: *list.induct*)
case *Nil*
then show *?case* **unfolding** *code-head-def* **by** *simp*
next
case (*Cons a l*)
from *Cons.hyps*(*2*) **have** *tail-code*: *nat-list-encode*(l) $\in$ *nat*
by (*rule nat-list-encode-type*)
from *Cons tail-code* **show** *?case*
unfolding *code-head-def* **by** (*simp add*: *pair-decode-code*)
qed

lemma *code-tail-encode*:
assumes *list*: $l \in list(nat)$
shows *code-tail*(*nat-list-encode*(l)) = *nat-list-encode*(*tl*(l))
using *list*
proof (*induct l rule*: *list.induct*)
case *Nil*
then show *?case* **unfolding** *code-tail-def* **by** *simp*
next
case (*Cons a l*)
from *Cons.hyps*(*2*) **have** *tail-code*: *nat-list-encode*(l) $\in$ *nat*
by (*rule nat-list-encode-type*)
from *Cons tail-code* **show** *?case*
unfolding *code-tail-def* **by** (*simp add*: *pair-decode-code*)
qed

consts *code-drop* :: $[i,i] \Rightarrow i$

primrec
code-drop(0,c) = c
code-drop(*succ*(k),c) = *code-tail*(*code-drop*(k,c))

lemma *code-drop-type* [*TC*]:
assumes *count*: $k \in nat$
and *code*: $c \in nat$
shows *code-drop*(k,c) $\in$ *nat*
using *count code*
by (*induct k rule*: *nat-induct*) (*simp-all add*: *code-tail-type*)

lemma *code-drop-encode*:
assumes *count*: $k \in nat$
and *list*: $l \in list(nat)$
shows *code-drop*(k,*nat-list-encode*(l)) =

```
    nat-list-encode(drop(k,l))
  using count
proof (induct k rule: nat-induct)
  case 0
  then show ?case by simp
next
  case (succ k)
  from succ.hyps(1) list have dropped: drop(k,l) ∈ list(nat)
    by typecheck
  from succ list dropped show ?case
    by (simp add: code-tail-encode drop-succ)
qed

definition code-nth :: [i,i] ⇒ i where
  code-nth(k,c) ≡ code-head(code-drop(k,c))

lemma code-nth-type [TC]:
  assumes count: k ∈ nat
    and code: c ∈ nat
  shows code-nth(k,c) ∈ nat
  using count code unfolding code-nth-def by typecheck


lemma hd-eq-nth-zero:
  assumes list: l ∈ list(A)
  shows hd(l) = nth(0,l)
  using list by (cases rule: list.cases) simp-all

lemma code-nth-encode:
  assumes count: k ∈ nat
    and list: l ∈ list(nat)
  shows code-nth(k,nat-list-encode(l)) = nth(k,l)
proof −
  from count list have dropped: drop(k,l) ∈ list(nat)
    by typecheck
  from nth-drop[OF count] list have nth:
    nth(0,drop(k,l)) = nth(k,l)
    using count by simp
  show ?thesis
    unfolding code-nth-def
    using code-drop-encode[OF count list]
      code-head-encode[OF dropped] hd-eq-nth-zero[OF dropped] nth
    by simp
qed


definition code-has-index :: [i,i] ⇒ i where
  code-has-index(k,c) ≡
    if code-drop(k,c) = 0 then 0 else 1
```

lemma *code-has-index-type* [*TC*]:
code-has-index(*k*,*c*) ∈ *nat*
unfolding *code-has-index-def* **by** *typecheck*

lemma *nat-list-encode-zero-iff*:
assumes *list*: *l* ∈ *list*(*nat*)
shows *nat-list-encode*(*l*) = *0* ⟷ *l* = []
using *list* **by** (*cases rule*: *list.cases*) *simp-all*

lemma *drop-eq-Nil-iff*:
assumes *count*: *k* ∈ *nat*
and *list*: *l* ∈ *list*(*A*)
shows *drop*(*k*,*l*) = [] ⟷ *length*(*l*) ≤ *k*
proof
assume *empty*: *drop*(*k*,*l*) = []
from *length-drop*[*OF count*, *rule-format*, *OF list*] *empty*
have *length*(*l*) #− *k* = *0* **by** *simp*
with *count list* **show** *length*(*l*) ≤ *k*
by (*simp add*: *diff-is-0-lemma*)
next
assume *length*(*l*) ≤ *k*
with *count list* **show** *drop*(*k*,*l*) = []
by (*rule drop-all*)
qed

lemma *code-has-index-encode*:
assumes *count*: *k* ∈ *nat*
and *list*: *l* ∈ *list*(*nat*)
shows *code-has-index*(*k*,*nat-list-encode*(*l*)) =
(*if k* < *length*(*l*) *then 1 else 0*)
proof −
from *list* **have** *length*: *length*(*l*) ∈ *nat*
by *typecheck*
from *count length* **have** *not-le*:
¬ *length*(*l*) ≤ *k* ⟷ *k* < *length*(*l*)
using *not-le-iff-lt*[*OF nat-into-Ord nat-into-Ord*] **by** *simp*
show *?thesis*
unfolding *code-has-index-def*
using *code-drop-encode*[*OF count list*]
nat-list-encode-zero-iff[*OF drop-type*[*OF count list*]]
drop-eq-Nil-iff[*OF count list*] *not-le*
by *auto*
qed

9.1 Instruction Code Normalization

definition *default-instruction-code* :: *i* **where**

default-instruction-code ≡
pair-code(*nop*,*final-state*)

definition *normalize-instruction-code* :: $i \Rightarrow i$ **where**
normalize-instruction-code(n) ≡
if $n \in nat \wedge$ *pair-left-code*(n) ∈ *action then* n
else default-instruction-code

lemma *default-instruction-code-type* [*TC*]:
default-instruction-code ∈ *nat*
unfolding *default-instruction-code-def* **by** *typecheck*

lemma *normalize-instruction-code-type* [*TC*]:
normalize-instruction-code(n) ∈ *nat*
unfolding *normalize-instruction-code-def*
using *default-instruction-code-type* **by** *auto*

theorem *normalize-instruction-code-correct*:
normalize-instruction-code(n) =
encode-instruction(*decode-instruction*(n))
proof (*cases* $n \in nat$)
case *True*
from *True* **have** *decoded*: *pair-decode*(n) ∈ $nat \times nat$
by (*rule pair-decode-type*)
from *decoded* **have** *first*: *fst*(*pair-decode*(n)) ∈ *nat*
and *second*: *snd*(*pair-decode*(n)) ∈ *nat*
by *typecheck+*
from *True* **have** *recover*:
pair-code(*fst*(*pair-decode*(n)),*snd*(*pair-decode*(n))) = n
by (*rule pair-code-decode*)
from *True* **show** *?thesis*
unfolding *normalize-instruction-code-def decode-instruction-def*
encode-instruction-def default-instruction-code-def
using *pair-left-code-decode*[*OF True*] *recover first second*
by *auto*
next
case *False*
then show *?thesis*
unfolding *normalize-instruction-code-def decode-instruction-def*
encode-instruction-def default-instruction-code-def
by *simp*
qed

end

10 Primitive-Recursive Coding

theory *Turing-Primrec*

imports
Turing-Code-Operations
ZF−Induct.Primrec
begin

This theory constructs primitive-recursive numerical operations for pairing and finite-list codes. Each certificate consists of membership in the formal class of primitive-recursive functions and an application theorem identifying its value with the corresponding code operation. Arguments form a list of natural numbers. Application equations retain an arbitrary natural-number tail to state explicitly which leading arguments the operation uses. These results do not construct a Turing machine that implements the operations.

10.1 Composition and Projection

lemma *COMP-2*:
assumes *inputs*: $l \in list(nat)$
shows $COMP(g,[f,h])`l = g`[f`l,h`l]$
using *inputs* **unfolding** *COMP-def* **by** *simp*

lemma *COMP-3*:
assumes *inputs*: $l \in list(nat)$
shows $COMP(g,[f,h,k])`l = g`[f`l,h`l,k`l]$
using *inputs* **unfolding** *COMP-def* **by** *simp*

lemma *PROJ-1* [*simp*]:
assumes *first*: $x \in nat$
and *second*: $y \in nat$
and *tail*: $l \in list(nat)$
shows $PROJ(1)`Cons(x,Cons(y,l)) = y$
using *first second tail* **unfolding** *PROJ-def* **by** *simp*

lemma *PROJ-2* [*simp*]:
assumes *first*: $x \in nat$
and *second*: $y \in nat$
and *third*: $z \in nat$
and *tail*: $l \in list(nat)$
shows $PROJ(2)`Cons(x,Cons(y,Cons(z,l))) = z$
using *first second third tail* **unfolding** *PROJ-def* **by** *simp*

lemma *PROJ-3* [*simp*]:
assumes *first*: $x \in nat$
and *second*: $y \in nat$
and *third*: $z \in nat$
and *fourth*: $w \in nat$
and *tail*: $l \in list(nat)$
shows $PROJ(succ(2))`Cons(x,Cons(y,Cons(z,Cons(w,l)))) = w$
using *first second third fourth tail* **unfolding** *PROJ-def* **by** *simp*

10.2 Arithmetic

definition *pr-pred* :: *i* **where**
  pr-pred ≡ *PREC*(*CONSTANT*(*0*),*PROJ*(*1*))

lemma *pr-pred-in-prim-rec* [*TC*]:
  pr-pred ∈ *prim-rec*
  unfolding *pr-pred-def* **by** *typecheck*

lemma *pr-pred-apply*:
  assumes *natural*: *n* ∈ *nat*
    and *tail*: *l* ∈ *list*(*nat*)
  shows *pr-pred*‘*Cons*(*n*,*l*) = *pred*(*n*)
  using *natural*
proof (*induct n rule*: *nat-induct*)
  case *0*
  with *tail* **show** *?case*
    unfolding *pr-pred-def* **by** (*simp add*: *PREC-0 CONSTANT*)
next
  case (*succ n*)
  with *tail* **show** *?case*
    unfolding *pr-pred-def* **by** (*simp add*: *PREC-succ*)
qed

definition *pr-add* :: *i* **where**
  pr-add ≡ *PREC*(*PROJ*(*0*),*SC*)

lemma *pr-add-in-prim-rec* [*TC*]:
  pr-add ∈ *prim-rec*
  unfolding *pr-add-def* **by** *typecheck*

lemma *pr-add-apply*:
  assumes *first*: *m* ∈ *nat*
    and *second*: *n* ∈ *nat*
    and *tail*: *l* ∈ *list*(*nat*)
  shows *pr-add*‘*Cons*(*m*,*Cons*(*n*,*l*)) = *m* #+ *n*
  using *first*
proof (*induct m rule*: *nat-induct*)
  case *0*
  with *second tail* **show** *?case*
    unfolding *pr-add-def* **by** (*simp add*: *PREC-0 PROJ-0*)
next
  case (*succ m*)
  with *second tail* **show** *?case*
    unfolding *pr-add-def* **by** (*simp add*: *PREC-succ SC*)
qed

definition *pr-double* :: *i* **where**
  pr-double ≡ *COMP*(*pr-add*,[*PROJ*(*0*),*PROJ*(*0*)])

lemma *pr-double-in-prim-rec* [*TC*]:
pr-double ∈ *prim-rec*
unfolding *pr-double-def* **by** *typecheck*

lemma *pr-double-apply*:
assumes *natural*: *n* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *pr-double*‘*Cons*(*n*,*l*) = *n* #+ *n*
unfolding *pr-double-def* **using** *natural tail*
by (*simp add*: *COMP-2 PROJ-0 pr-add-apply*)

definition *pr-reverse-diff* :: *i* **where**
pr-reverse-diff ≡
PREC(*PROJ*(*0*),*COMP*(*pr-pred*,[*PROJ*(*0*)]))

lemma *pr-reverse-diff-in-prim-rec* [*TC*]:
pr-reverse-diff ∈ *prim-rec*
unfolding *pr-reverse-diff-def* **by** *typecheck*

lemma *pr-reverse-diff-apply*:
assumes *subtrahend*: *n* ∈ *nat*
and *minuend*: *m* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *pr-reverse-diff*‘*Cons*(*n*,*Cons*(*m*,*l*)) = *m* #− *n*
using *subtrahend*
proof (*induct n rule*: *nat-induct*)
case *0*
with *minuend tail* **show** *?case*
unfolding *pr-reverse-diff-def*
by (*simp add*: *PREC-0 PROJ-0*)
next
case (*succ n*)
with *minuend tail* **show** *?case*
unfolding *pr-reverse-diff-def*
by (*simp add*: *PREC-succ COMP-1 PROJ-def pr-pred-apply*
diff-succ-eq-pred)
qed

definition *pr-diff* :: *i* **where**
pr-diff ≡
COMP(*pr-reverse-diff*,[*PROJ*(*1*),*PROJ*(*0*)])

lemma *pr-diff-in-prim-rec* [*TC*]:
pr-diff ∈ *prim-rec*
unfolding *pr-diff-def* **by** *typecheck*

lemma *pr-diff-apply*:
assumes *first*: *m* ∈ *nat*
and *second*: *n* ∈ *nat*

```
    and tail: l ∈ list(nat)
  shows pr-diff‘Cons(m,Cons(n,l)) = m #− n
  unfolding pr-diff-def
  using first second tail
  by (simp add: COMP-2 pr-reverse-diff-apply PROJ-0)

lemma succ-diff-succ:
  assumes index: j ∈ nat
    and total: n ∈ nat
    and bounded: succ(j) ≤ n
  shows succ(n #− succ(j)) = n #− j
proof −
  from bounded have less: j < n
    by (rule succ-leE)
  from total index have comparison:
    ¬ n ≤ j ⟷ j < n
    using not-le-iff-lt[OF nat-into-Ord nat-into-Ord] by simp
  from less comparison have not-le: ¬ n ≤ j
    by simp
  from index total not-le have nonzero: n #− j ≠ 0
    by (simp add: diff-is-0-lemma)
  have difference: n #− j ∈ nat
    by typecheck
  from difference show ?thesis
  proof (cases rule: natE)
    case 0
    with nonzero show ?thesis by contradiction
  next
    case (succ d)
    then show ?thesis by (simp add: diff-succ-eq-pred)
  qed
qed
```

10.3 Conditionals

```
definition pr-is-zero :: i where
  pr-is-zero ≡ PREC(CONSTANT(1),CONSTANT(0))

lemma pr-is-zero-in-prim-rec [TC]:
  pr-is-zero ∈ prim-rec
  unfolding pr-is-zero-def by typecheck

lemma pr-is-zero-apply:
  assumes natural: n ∈ nat
    and tail: l ∈ list(nat)
  shows pr-is-zero‘Cons(n,l) = (if n = 0 then 1 else 0)
  using natural
proof (cases rule: natE)
  case 0
```

```
  with tail show ?thesis
    unfolding pr-is-zero-def by (simp add: PREC-0 CONSTANT)
next
  case (succ m)
  with tail show ?thesis
    unfolding pr-is-zero-def by (simp add: PREC-succ CONSTANT)
qed

definition pr-positive :: i where
  pr-positive ≡ PREC(CONSTANT(0),CONSTANT(1))

lemma pr-positive-in-prim-rec [TC]:
  pr-positive ∈ prim-rec
  unfolding pr-positive-def by typecheck

lemma pr-positive-apply:
  assumes natural: n ∈ nat
    and tail: l ∈ list(nat)
  shows pr-positive`Cons(n,l) = (if n = 0 then 0 else 1)
  using natural
proof (cases rule: natE)
  case 0
  with tail show ?thesis
    unfolding pr-positive-def by (simp add: PREC-0 CONSTANT)
next
  case (succ m)
  with tail show ?thesis
    unfolding pr-positive-def by (simp add: PREC-succ CONSTANT)
qed

definition pr-if-zero :: i where
  pr-if-zero ≡ PREC(PROJ(0),PROJ(succ(2)))

lemma pr-if-zero-in-prim-rec [TC]:
  pr-if-zero ∈ prim-rec
  unfolding pr-if-zero-def by typecheck

lemma pr-if-zero-apply:
  assumes condition: c ∈ nat
    and zero: z ∈ nat
    and successor: s ∈ nat
    and tail: l ∈ list(nat)
  shows pr-if-zero`Cons(c,Cons(z,Cons(s,l))) =
    (if c = 0 then z else s)
  using condition
proof (cases rule: natE)
  case 0
  with zero successor tail show ?thesis
    unfolding pr-if-zero-def by (simp add: PREC-0 PROJ-0)
```

next
case (*succ n*)
with *zero successor tail* **show** *?thesis*
unfolding *pr-if-zero-def* **by** (*simp add*: *PREC-succ*)
qed

definition *pr-le* :: *i* **where**
pr-le ≡ *COMP*(*pr-is-zero*,[*pr-diff*])

lemma *pr-le-in-prim-rec* [*TC*]:
pr-le ∈ *prim-rec*
unfolding *pr-le-def* **by** *typecheck*

lemma *pr-le-apply*:
assumes *first*: *m* ∈ *nat*
and *second*: *n* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *pr-le*‘*Cons*(*m*,*Cons*(*n*,*l*)) = (*if m* ≤ *n then 1 else 0*)
unfolding *pr-le-def*
using *first second tail*
by (*simp add*: *COMP-1 pr-diff-apply pr-is-zero-apply diff-is-0-lemma*)

10.4 Cantor Pairing

definition *pr-triangle-step* :: *i* **where**
pr-triangle-step ≡
COMP(*pr-add*,[*PROJ*(*0*),*COMP*(*SC*,[*PROJ*(*1*)])])

lemma *pr-triangle-step-in-prim-rec* [*TC*]:
pr-triangle-step ∈ *prim-rec*
unfolding *pr-triangle-step-def* **by** *typecheck*

definition *pr-triangle* :: *i* **where**
pr-triangle ≡ *PREC*(*CONSTANT*(*0*),*pr-triangle-step*)

lemma *pr-triangle-in-prim-rec* [*TC*]:
pr-triangle ∈ *prim-rec*
unfolding *pr-triangle-def* **by** *typecheck*

lemma *pr-triangle-apply*:
assumes *natural*: *n* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *pr-triangle*‘*Cons*(*n*,*l*) = *triangle*(*n*)
using *natural*
proof (*induct n rule*: *nat-induct*)
case *0*
with *tail* **show** *?case*
unfolding *pr-triangle-def* **by** (*simp add*: *PREC-0 CONSTANT*)
next

case (*succ n*)
from *succ.hyps*(*1*) **have** *triangle-natural*: *triangle*(*n*) ∈ *nat*
by (*rule triangle-type*)
from *succ tail triangle-natural* **show** *?case*
unfolding *pr-triangle-def pr-triangle-step-def*
by (*simp add*: *PREC-succ COMP-2 COMP-1 PROJ-def SC pr-add-apply*)
qed

definition *pr-pair-code* :: *i* **where**
pr-pair-code ≡
COMP(*pr-add*,[*COMP*(*pr-triangle*,[*pr-add*]),*PROJ*(*1*)])

theorem *pr-pair-code-in-prim-rec* [*TC*]:
pr-pair-code ∈ *prim-rec*
unfolding *pr-pair-code-def* **by** *typecheck*

theorem *pr-pair-code-apply*:
assumes *first*: *m* ∈ *nat*
and *second*: *n* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *pr-pair-code*‘*Cons*(*m*,*Cons*(*n*,*l*)) = *pair-code*(*m*,*n*)
unfolding *pr-pair-code-def pair-code-def*
using *first second tail*
by (*simp add*: *COMP-2 COMP-1 pr-add-apply pr-triangle-apply PROJ-0*)

10.5 Cantor Projections

definition *pr-equal-core* :: *i* **where**
pr-equal-core ≡ *COMP*(*pr-add*,[*pr-diff*,*pr-reverse-diff*])

lemma *pr-equal-core-in-prim-rec* [*TC*]:
pr-equal-core ∈ *prim-rec*
unfolding *pr-equal-core-def* **by** *typecheck*

definition *pr-equal* :: *i* **where**
pr-equal ≡ *COMP*(*pr-is-zero*,[*pr-equal-core*])

lemma *pr-equal-in-prim-rec* [*TC*]:
pr-equal ∈ *prim-rec*
unfolding *pr-equal-def* **by** *typecheck*

lemma *pr-equal-apply*:
assumes *first*: *m* ∈ *nat*
and *second*: *n* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *pr-equal*‘*Cons*(*m*,*Cons*(*n*,*l*)) = (*if m* = *n then 1 else 0*)
proof −
have *difference*:
(*m* #− *n*) #+ (*n* #− *m*) = *0* ⟷ *m* = *n*

```
    using first second
    by (auto simp add: diff-is-0-lemma intro: le-anti-sym)
  show ?thesis
    unfolding pr-equal-def pr-equal-core-def
    using first second tail difference
    by (simp add: COMP-2 COMP-1 pr-add-apply pr-diff-apply
        pr-reverse-diff-apply pr-is-zero-apply)
qed

definition pr-diagonal-end :: i where
  pr-diagonal-end ≡
    COMP(pr-add,[COMP(pr-triangle,[PROJ(0)]),PROJ(0)])

lemma pr-diagonal-end-in-prim-rec [TC]:
  pr-diagonal-end ∈ prim-rec
  unfolding pr-diagonal-end-def by typecheck

definition pr-pair-diagonal-step :: i where
  pr-pair-diagonal-step ≡
    COMP(pr-if-zero,
      [COMP(pr-equal,[PROJ(1),pr-diagonal-end]),
       PROJ(0),
       COMP(SC,[PROJ(0)])])

lemma pr-pair-diagonal-step-in-prim-rec [TC]:
  pr-pair-diagonal-step ∈ prim-rec
  unfolding pr-pair-diagonal-step-def by typecheck

definition pr-pair-diagonal :: i where
  pr-pair-diagonal ≡
    PREC(CONSTANT(0),pr-pair-diagonal-step)

lemma pr-pair-diagonal-in-prim-rec [TC]:
  pr-pair-diagonal ∈ prim-rec
  unfolding pr-pair-diagonal-def by typecheck

lemma pr-pair-diagonal-apply:
  assumes natural: n ∈ nat
    and tail: l ∈ list(nat)
  shows pr-pair-diagonal`Cons(n,l) = pair-diagonal(n)
  using natural
proof (induct n rule: nat-induct)
  case 0
  with tail show ?case
    unfolding pr-pair-diagonal-def by (simp add: PREC-0 CONSTANT)
next
  case (succ n)
  from succ.hyps(1) have diagonal-natural: pair-diagonal(n) ∈ nat
    by (rule pair-diagonal-type)
```

```
  from succ tail diagonal-natural show ?case
    unfolding pr-pair-diagonal-def pr-pair-diagonal-step-def
      pr-diagonal-end-def
    by (simp add: PREC-succ COMP-3 COMP-2 COMP-1 PROJ-def SC
        pr-if-zero-apply pr-equal-apply pr-add-apply pr-triangle-apply)
qed

definition pr-pair-right :: i where
  pr-pair-right ≡
    COMP(pr-diff,[PROJ(0),COMP(pr-triangle,[pr-pair-diagonal])])

lemma pr-pair-right-in-prim-rec [TC]:
  pr-pair-right ∈ prim-rec
  unfolding pr-pair-right-def by typecheck

lemma pr-pair-right-apply:
  assumes natural: n ∈ nat
    and tail: l ∈ list(nat)
  shows pr-pair-right`Cons(n,l) = snd(pair-decode(n))
  unfolding pr-pair-right-def
  using natural tail pair-right-code-decode[OF natural]
  unfolding pair-right-code-def
  by (simp add: COMP-2 COMP-1 pr-diff-apply pr-triangle-apply
      pr-pair-diagonal-apply PROJ-0)

definition pr-pair-left :: i where
  pr-pair-left ≡
    COMP(pr-diff,[pr-pair-diagonal,pr-pair-right])

lemma pr-pair-left-in-prim-rec [TC]:
  pr-pair-left ∈ prim-rec
  unfolding pr-pair-left-def by typecheck

lemma pr-pair-left-apply:
  assumes natural: n ∈ nat
    and tail: l ∈ list(nat)
  shows pr-pair-left`Cons(n,l) = fst(pair-decode(n))
  unfolding pr-pair-left-def
  using natural tail pair-left-code-decode[OF natural]
    pair-right-code-decode[OF natural]
  unfolding pair-left-code-def
  by (simp add: COMP-2 pr-diff-apply pr-pair-diagonal-apply
      pr-pair-right-apply)

corollary pr-pair-right-code-apply:
  assumes natural: n ∈ nat
    and tail: l ∈ list(nat)
  shows pr-pair-right`Cons(n,l) = pair-right-code(n)
  using pr-pair-right-apply[OF natural tail]
```

pair-right-code-decode[*OF natural*]
by *simp*

corollary *pr-pair-left-code-apply*:
assumes *natural*: $n \in nat$
and *tail*: $l \in list(nat)$
shows *pr-pair-left*`*Cons*$(n,l) =$ *pair-left-code*(n)
using *pr-pair-left-apply*[*OF natural tail*]
pair-left-code-decode[*OF natural*]
by *simp*

10.6 Coded Natural Lists

definition *pr-list-cons* :: i **where**
pr-list-cons $\equiv$ *COMP*$(SC,[$*pr-pair-code*$])$

lemma *pr-list-cons-in-prim-rec* [*TC*]:
pr-list-cons $\in$ *prim-rec*
unfolding *pr-list-cons-def* **by** *typecheck*

lemma *pr-list-cons-apply*:
assumes *element*: $a \in nat$
and *tail-code*: $c \in nat$
and *tail*: $l \in list(nat)$
shows *pr-list-cons*`*Cons*$(a,$*Cons*$(c,l)) =$ *code-cons*(a,c)
unfolding *pr-list-cons-def code-cons-def*
using *element tail-code tail*
by (*simp add*: *COMP-1 SC pr-pair-code-apply*)

definition *pr-list-head* :: i **where**
pr-list-head $\equiv$
COMP$($*pr-if-zero*$,$
$[PROJ(0),CONSTANT(0),$
COMP$($*pr-pair-left*$,[$*COMP*$($*pr-pred*$,[PROJ(0)])])])$

lemma *pr-list-head-in-prim-rec* [*TC*]:
pr-list-head $\in$ *prim-rec*
unfolding *pr-list-head-def* **by** *typecheck*

lemma *pr-list-head-apply*:
assumes *natural*: $c \in nat$
and *tail*: $l \in list(nat)$
shows *pr-list-head*`*Cons*$(c,l) =$ *code-head*(c)
using *natural*
proof (*cases rule*: *natE*)
case *0*
with *tail* **show** *?thesis*
unfolding *pr-list-head-def code-head-def*
by (*simp add*: *COMP-3 COMP-1 CONSTANT PROJ-0 pr-if-zero-apply*)

```
next
  case (succ n)
  with tail show ?thesis
    unfolding pr-list-head-def code-head-def
    by (simp add: COMP-3 COMP-1 CONSTANT PROJ-0 pr-if-zero-apply
        pr-pred-apply pr-pair-left-apply)
qed

definition pr-list-tail :: i where
  pr-list-tail ≡
    COMP(pr-if-zero,
      [PROJ(0),CONSTANT(0),
       COMP(pr-pair-right,[COMP(pr-pred,[PROJ(0)])])])

lemma pr-list-tail-in-prim-rec [TC]:
  pr-list-tail ∈ prim-rec
  unfolding pr-list-tail-def by typecheck

lemma pr-list-tail-apply:
  assumes natural: c ∈ nat
    and tail: l ∈ list(nat)
  shows pr-list-tail`Cons(c,l) = code-tail(c)
  using natural
proof (cases rule: natE)
  case 0
  with tail show ?thesis
    unfolding pr-list-tail-def code-tail-def
    by (simp add: COMP-3 COMP-1 CONSTANT PROJ-0 pr-if-zero-apply)
next
  case (succ n)
  with tail show ?thesis
    unfolding pr-list-tail-def code-tail-def
    by (simp add: COMP-3 COMP-1 CONSTANT PROJ-0 pr-if-zero-apply
        pr-pred-apply pr-pair-right-apply)
qed

definition pr-list-drop :: i where
  pr-list-drop ≡
    PREC(PROJ(0),COMP(pr-list-tail,[PROJ(0)]))

lemma pr-list-drop-in-prim-rec [TC]:
  pr-list-drop ∈ prim-rec
  unfolding pr-list-drop-def by typecheck

lemma pr-list-drop-apply:
  assumes count: k ∈ nat
    and code: c ∈ nat
    and tail: l ∈ list(nat)
  shows pr-list-drop`Cons(k,Cons(c,l)) = code-drop(k,c)
```

```
  using count
proof (induct k rule: nat-induct)
  case 0
  with code tail show ?case
    unfolding pr-list-drop-def by (simp add: PREC-0 PROJ-0)
next
  case (succ k)
  from succ.hyps(1) code have dropped: code-drop(k,c) ∈ nat
    by (rule code-drop-type)
  from succ code tail dropped show ?case
    unfolding pr-list-drop-def
    by (simp add: PREC-succ COMP-1 PROJ-def pr-list-tail-apply)
qed

definition pr-list-nth :: i where
  pr-list-nth ≡ COMP(pr-list-head,[pr-list-drop])

lemma pr-list-nth-in-prim-rec [TC]:
  pr-list-nth ∈ prim-rec
  unfolding pr-list-nth-def by typecheck

lemma pr-list-nth-apply:
  assumes count: k ∈ nat
    and code: c ∈ nat
    and tail: l ∈ list(nat)
  shows pr-list-nth`Cons(k,Cons(c,l)) = code-nth(k,c)
  unfolding pr-list-nth-def code-nth-def
  using count code tail
  by (simp add: COMP-1 pr-list-drop-apply pr-list-head-apply
      code-drop-type)

definition pr-list-has-index :: i where
  pr-list-has-index ≡
    COMP(pr-positive,[pr-list-drop])

lemma pr-list-has-index-in-prim-rec [TC]:
  pr-list-has-index ∈ prim-rec
  unfolding pr-list-has-index-def by typecheck

lemma pr-list-has-index-apply:
  assumes count: k ∈ nat
    and code: c ∈ nat
    and tail: l ∈ list(nat)
  shows pr-list-has-index`Cons(k,Cons(c,l)) = code-has-index(k,c)
  unfolding pr-list-has-index-def code-has-index-def
  using count code tail
  by (simp add: COMP-1 pr-list-drop-apply pr-positive-apply
      code-drop-type)
```

end

11 Primitive-Recursive Instruction Normalization

theory *Turing-Instruction-Normalization*
imports *Turing-Primrec*
begin

Invalid action codes denote the no-op instruction entering the final state. Normalization and its primitive-recursive certificate depend only on coding operations.

lemma *action-iff-le-nop*:
assumes *natural*: $a \in nat$
shows $a \in action \longleftrightarrow a \leq nop$
using *natural nat-into-Ord*[*OF natural*]
unfolding *action-def*
by (*simp add*: *Ord-mem-iff-lt*)

definition *pr-valid-action* :: i **where**
$pr\text{-}valid\text{-}action \equiv$
$COMP(pr\text{-}le,[PROJ(0),CONSTANT(nop)])$

lemma *pr-valid-action-in-prim-rec* [*TC*]:
pr-valid-action $\in$ *prim-rec*
unfolding *pr-valid-action-def* **by** *typecheck*

lemma *pr-valid-action-apply*:
assumes *action-code*: $a \in nat$
and *tail*: $l \in list(nat)$
shows $pr\text{-}valid\text{-}action`Cons(a,l) =$
(*if* $a \in action$ *then 1 else 0*)
unfolding *pr-valid-action-def*
using *action-code tail action-iff-le-nop*[*OF action-code*]
by (*simp add*: *COMP-2 CONSTANT PROJ-0 pr-le-apply*)

definition *pr-valid-instruction-code* :: i **where**
$pr\text{-}valid\text{-}instruction\text{-}code \equiv$
$COMP(pr\text{-}valid\text{-}action,[pr\text{-}pair\text{-}left])$

lemma *pr-valid-instruction-code-in-prim-rec* [*TC*]:
pr-valid-instruction-code $\in$ *prim-rec*
unfolding *pr-valid-instruction-code-def* **by** *typecheck*

lemma *pr-valid-instruction-code-apply*:
assumes *instruction-number*: $n \in nat$
and *tail*: $l \in list(nat)$
shows $pr\text{-}valid\text{-}instruction\text{-}code`Cons(n,l) =$
(*if* $pair\text{-}left\text{-}code(n) \in action$ *then 1 else 0*)

proof −
from *instruction-number* **have** *action-code*: *pair-left-code*(*n*) ∈ *nat*
by *typecheck*
show *?thesis*
unfolding *pr-valid-instruction-code-def*
using *instruction-number tail action-code*
by (*simp add*: *COMP-1 pr-pair-left-code-apply pr-valid-action-apply*)
qed

definition *pr-normalize-instruction-code* :: *i* **where**
pr-normalize-instruction-code ≡
COMP(*pr-if-zero*,
[*pr-valid-instruction-code*,
CONSTANT(*default-instruction-code*),
PROJ(*0*)])

lemma *pr-normalize-instruction-code-in-prim-rec* [*TC*]:
pr-normalize-instruction-code ∈ *prim-rec*
unfolding *pr-normalize-instruction-code-def* **by** *typecheck*

lemma *pr-normalize-instruction-code-apply*:
assumes *instruction-number*: *n* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *pr-normalize-instruction-code*‘*Cons*(*n*,*l*) =
normalize-instruction-code(*n*)
unfolding *pr-normalize-instruction-code-def*
normalize-instruction-code-def
using *instruction-number tail default-instruction-code-type*
by (*simp add*: *COMP-3 CONSTANT PROJ-0 pr-if-zero-apply*
pr-valid-instruction-code-apply)

end

12 Finite Witnesses in Transitive Models

theory *Turing-Model-Witnesses*
imports
Turing-Machine
Independence-CH.ZF-Trans-Interpretations
begin

definition *transitive-zf-model* :: *i* ⇒ *o* **where**
transitive-zf-model(*M*) ≡ *Transset*(*M*) ∧ *M* ⊨ *ZF*

Finite machines, inputs, configurations, and canonical finite traces belong to each transitive ZF set model. In *transitive-zf-model*, *ZF* is the internalised axiom set from *Independence-CH.Internal-ZFC-Axioms*. The predicate requires satisfaction of those axioms by the given set. Choice is not required for finite object closure or finite runs.

12.1 Finite object closure

context *M-trivial*
begin

lemma *tm-list-closed*:
  assumes *xs*: $xs \in list(A)$ **and** *elements*: $\bigwedge a.\ a \in A \Longrightarrow M(a)$
  shows $M(xs)$
  using *xs* **by** (*induct xs rule*: *list.induct*) (*auto intro*: *elements*)

lemma *tm-symbol-closed*: $a \in symbol \Longrightarrow M(a)$
  unfolding *symbol-def* **by** *auto*

lemma *tm-instruction-closed*: $ins \in instruction \Longrightarrow M(ins)$
  unfolding *instruction-def action-def* **by** *auto*

lemma *tm-machine-closed*: $P \in tm\text{-}machine \Longrightarrow M(P)$
  unfolding *tm-machine-def* **by** (*rule tm-list-closed*) (*auto intro*:
    tm-instruction-closed)

lemma *tm-input-closed*: $x \in list(symbol) \Longrightarrow M(x)$
  by (*rule tm-list-closed*) (*auto intro*: *tm-symbol-closed*)

lemma *tm-configuration-closed*: $c \in configuration \Longrightarrow M(c)$
  unfolding *configuration-def tm-tape-def* **by** (*auto intro*: *tm-input-closed*)

lemma *tm-finite-lambda-closed*:
  assumes *n*: $n \in nat$ **and** *values*: $\bigwedge k.\ k \in n \Longrightarrow M(f(k))$
  shows $M(\lambda k{\in}n.\ f(k))$
  using *n values*
proof (*induct n rule*: *nat-induct*)
  case *0* **show** *?case* **by** *simp*
next
  case (*succ n*)
  have *fn*: $M(f(n))$ **using** *succ.prems* **by** *auto*
  have *rest*: $M(\lambda k{\in}n.\ f(k))$ **by** (*rule succ.hyps*(*2*)) (*auto intro*:
    succ.prems)
  have *split*: $(\lambda k{\in}succ(n).\ f(k)) = cons(\langle n,f(n)\rangle,\lambda k{\in}n.\ f(k))$
    unfolding *lam-def succ-def* **by** *auto*
  from *fn rest nat-into-M*[*OF succ.hyps*(*1*)] *split* **show** *?case* **by** *simp*
qed

Finite traces are built from ordered pairs by finite induction. The closure theorem *tm-finite-lambda-closed* requires only that each value belongs to the model. It does not require the whole function to have been given internally.

lemma *tm-trace-closed*:
  assumes *P*: $P \in tm\text{-}machine$ **and** *c*: $c \in configuration$ **and** *n*: $n \in nat$
  shows $M(\lambda k{\in}succ(n).\ steps(P,c,k))$
proof (*rule tm-finite-lambda-closed*[*OF nat-succI*[*OF n*]])
  fix *k* **assume** *k*: $k \in succ(n)$

have *natural*: $k \in nat$ **using** *k n* **by** (*blast intro*: *Ord-trans*)
show $M(steps(P,c,k))$ **by** (*rule tm-configuration-closed*[*OF steps-type*[*OF P c natural*]])
qed
end

12.2 Transitive model interpretations

lemma *transitive-zf-M-ZF1-trans*:
assumes *model*: *transitive-zf-model*(A)
shows *M-ZF1-trans*(A)
proof −
from *model* **have** *trans*: *Transset*(A) **and** *zf*: $A \models ZF$
unfolding *transitive-zf-model-def* **by** *auto*
from *M-satT-imp-M-ZF2*[*OF zf*] *trans* **show** *?thesis*
unfolding *M-ZF1-trans-def M-Z-trans-def M-transset-def M-ZF1-def* **by** *auto*
qed

lemma *transitive-zf-M-trivial*:
transitive-zf-model(A) $\Longrightarrow$ *M-trivial*($\#\#A$)
proof −
assume *model*: *transitive-zf-model*(A)
interpret *A*: *M-ZF1-trans A* **by** (*rule transitive-zf-M-ZF1-trans*[*OF model*])
show *M-trivial*($\#\#A$) **by** *unfold-locales*
qed

lemma *transitive-zf-M-trancl*:
transitive-zf-model(A) $\Longrightarrow$ *M-trancl*($\#\#A$)
proof −
assume *model*: *transitive-zf-model*(A)
interpret *A*: *M-ZF1-trans A* **by** (*rule transitive-zf-M-ZF1-trans*[*OF model*])
show *M-trancl*($\#\#A$) **by** *unfold-locales*
qed

12.3 Machine objects in a set model

theorem *transitive-zf-contains-machine*:
⟦*transitive-zf-model*(A); $P \in$ *tm-machine*⟧ $\Longrightarrow$ $P \in A$
using *M-trivial.tm-machine-closed*[*OF transitive-zf-M-trivial*] **by** *simp*

theorem *transitive-zf-contains-configuration*:
⟦*transitive-zf-model*(A); $c \in$ *configuration*⟧ $\Longrightarrow$ $c \in A$
using *M-trivial.tm-configuration-closed*[*OF transitive-zf-M-trivial*] **by** *simp*

theorem *transitive-zf-contains-trace*:
⟦*transitive-zf-model*(A); $P \in$ *tm-machine*; $c \in$ *configuration*; $n \in nat$⟧ $\Longrightarrow$
$(\lambda k \in succ(n).\ steps(P,c,k)) \in A$
using *M-trivial.tm-trace-closed*[*OF transitive-zf-M-trivial*] **by** *simp*

end

13 Natural Membership in Transitive Models

theory *Nat-Membership*
  imports *Turing-Model-Witnesses*
begin

The formula expresses membership in the standard natural numbers. Transitivity identifies the model's natural numbers with the external set *nat*.

definition *nat-member-fm* :: $i \Rightarrow i$ **where**
  nat-member-fm(i) $\equiv$ *Exists*(*And*(*omega-fm*(*0*),*Member*(*succ*(i),*0*)))

The existentially bound set at index 0 is the model's omega. The original variable at index i shifts to *succ*(i), and membership in that set expresses membership in the external natural numbers.

lemma *nat-member-fm-type* [*TC*]:
  $i \in nat \Longrightarrow$ *nat-member-fm*(i) $\in$ *formula*
  unfolding *nat-member-fm-def* **by** *typecheck*

theorem *sats-nat-member-fm*:
  assumes *model*: *M-ZF1-trans*(A) **and** *env*: *env* $\in$ *list*(A)
    and i: $i \in nat$
  shows *sats*(A,*nat-member-fm*(i),*env*) $\longleftrightarrow$ *nth*(i,*env*) $\in$ *nat*
proof −
  interpret A: *M-ZF1-trans* A **by** (*rule model*)
  have *natA*: *nat* $\in$ A **using** A.*M-nat* **by** *simp*
  show *?thesis* **unfolding** *nat-member-fm-def*
    using *env* i *natA* **by** *auto*
qed

end

14 Internal Halting Formula

theory *Turing-Halting-Formula*
  imports *Nat-Membership*
begin

This theory constructs an explicit first-order formula *halt-fm* of set theory with two free variables and proves that, in every transitive set model of ZF, it holds of a machine P and an input x exactly when P halts on x (theorem *transitive-model-halt-fm-iff*).

The formulas for list traversal and machine execution constrain finitely many applications of an existential witness set. They do not require every witness to be a finite function. An external run supplies a canonical finite function belonging to the model by *transitive-zf-contains-trace*. Conversely, induction recovers the required finite sequence from any internal witness.

The formulas *drop-fm* and *fetch-fm* are internal components. The public interface consists of the halting satisfaction theorems. The library also supplies *nth-abs* in *M-datatypes*. That locale assumes replacement for list and formula constructions and list indexing. Here finite traversal witnesses are verified in *M-ZF1-trans*, without interpreting *M-datatypes*.

14.1 Names for natural numbers

definition *nat-name-fm* :: $i \Rightarrow i$ **where**
  nat-name-fm(n) $\equiv$ *nat-rec*(n, *empty-fm*(*0*), $\lambda k\ p$. *Exists*(*And*(p,
    succ-fm(*0*,*1*))))

The numeral name defines the value at index 0. Its successor clause binds the predecessor at index 0 and shifts the value being named to index 1. The formula *const-fm*(k,i) below transports this name to the selected environment index i.

lemma *nat-name-fm-0* [*simp*]: *nat-name-fm*(*0*) = *empty-fm*(*0*)
  unfolding *nat-name-fm-def* **by** (*rule nat-rec-0*)

lemma *nat-name-fm-succ* [*simp*]:
  $n \in nat \Longrightarrow$ *nat-name-fm*(*succ*(n)) =
    Exists(*And*(*nat-name-fm*(n), *succ-fm*(*0*,*1*)))
  unfolding *nat-name-fm-def* **by** (*rule nat-rec-succ*)

lemma *nat-name-fm-type* [*TC*]: $n \in nat \Longrightarrow$ *nat-name-fm*(n) $\in$ *formula*
  by (*induct n rule*: *nat-induct*) *simp-all*

lemma *arity-nat-name-fm*: $n \in nat \Longrightarrow$ *arity*(*nat-name-fm*(n)) = *1*
proof (*induct n rule*: *nat-induct*)
  case *0* **show** *?case* **by** (*simp add*: *arity-empty-fm*)
next
  case (*succ n*)
  have *u*: *1* $\cup$ *2* = *2* **by** (*rule subset-Un-iff*[*THEN iffD1*]) (*auto simp*:
    succ-def)
  have *s*: *arity*(*succ-fm*(*0*,*1*)) = *2* **using** *u* **by** (*simp add*: *arity-succ-fm*)
  show *?case* **using** *succ s u* **by** (*simp add*: *arity-Exists arity-And*)
qed

lemma *sats-nat-name-fm*:
  assumes *model*: *M-ZF1-trans*(A) **and** n: $n \in nat$
  shows $\bigwedge x\ env.\ x \in A \Longrightarrow env \in list(A) \Longrightarrow$
    (A, *Cons*(x,*env*)) $\models$ *nat-name-fm*(n)) $\longleftrightarrow$ $x = n$
  using n
proof (*induct n rule*: *nat-induct*)
  case *0*
  interpret A: *M-ZF1-trans A* **by** (*rule model*)
  have *trans*: *Transset*(A) **by** (*rule A.trans-M*)
  from *Transset-sats-empty-fm-eq-0*[*of 0 Cons*(x,*env*) A] *0 trans* **show** *?case* **by**

```
    simp
next
  case (succ n)
  interpret A: M-ZF1-trans A by (rule model)
  have nA: n ∈ A using A.nat-into-M[OF succ.hyps(1)] by simp
  have h1: (A, Cons(x,env) ⊨ nat-name-fm(succ(n))) ⟷
      (∃ y∈A. (A, Cons(y,Cons(x,env)) ⊨ nat-name-fm(n)) ∧ x = succ(y))
    using succ.hyps(1) succ.prems by simp
  have h2: (∃ y∈A. (A, Cons(y,Cons(x,env)) ⊨
    nat-name-fm(n)) ∧ x = succ(y)) ⟷ (∃ y∈A. y = n ∧ x = succ(y))
    using succ.hyps(2) succ.prems by auto
  from h1 h2 nA show ?case by auto
qed

definition const-fm :: [i,i] ⇒ i where
  const-fm(k,i) ≡ Exists(And(nat-name-fm(k), Equal(0,succ(i))))

lemma const-fm-type [TC]: ⟦k ∈ nat; i ∈ nat⟧ ⟹ const-fm(k,i) ∈ formula
  unfolding const-fm-def by simp

lemma sats-const-fm:
  assumes model: M-ZF1-trans(A) and k: k ∈ nat and i: i ∈ nat
    and env: env ∈ list(A)
  shows (A, env ⊨ const-fm(k,i)) ⟷ nth(i,env) = k
proof −
  interpret A: M-ZF1-trans A by (rule model)
  have kA: k ∈ A using A.nat-into-M[OF k] by simp
  have (A, env ⊨ const-fm(k,i)) ⟷ (∃ y∈A. y = k ∧ y = nth(i,env))
    unfolding const-fm-def using env i k sats-nat-name-fm[OF model k] by auto
  then show ?thesis using kA by auto
qed
```

14.2 Model and tape updates

The satisfaction proofs use *M-ZF1-trans*, a transitive model of Zermelo set theory with the four replacement instances specified by the imported locale. A transitive ZF set model interprets this locale. No minimality claim about these assumptions is made.

```
context M-ZF1-trans
begin

lemma tm-const [simp]:
  ⟦k ∈ nat; i ∈ nat; env ∈ list(M)⟧ ⟹
    (M, env ⊨ const-fm(k,i)) ⟷ nth(i,env) = k
  by (rule sats-const-fm[OF M-ZF1-trans-axioms])

lemma list-in-A:
  assumes l: l ∈ list(B) and el: ⋀a. a ∈ B ⟹ a ∈ M
  shows l ∈ M
```

using *tm-list-closed*[*OF l*] *el* **by** *simp*

lemma *nat-in-A* [*simp*]: $n \in nat \Longrightarrow n \in M$
using *nat-into-M* **by** *simp*

lemma *pair-in-A* [*simp*]: $\langle a,b\rangle \in M \longleftrightarrow a \in M \wedge b \in M$
using *pair-in-M-iff* **by** *simp*

lemma *Nil-in-A* [*simp*]: $[] \in M$
using *Nil-in-M* **by** *simp*

lemma *Cons-in-A* [*simp*]: $Cons(a,l) \in M \longleftrightarrow a \in M \wedge l \in M$
using *Cons-in-M-iff* **by** *simp*

lemma *symbols-in-A* [*simp*]: $l \in list(symbol) \Longrightarrow l \in M$
by (*erule list-in-A*) (*simp add*: *symbol-nat*)

lemma *hd'-list* [*simp*]: $l \in list(B) \Longrightarrow hd'(l) = hd(l)$
by (*simp add*: *hd'-def list-imp-quasilist*)

lemma *tl'-list* [*simp*]: $l \in list(B) \Longrightarrow tl'(l) = tl(l)$
by (*simp add*: *tl'-def list-imp-quasilist*)

lemma *hd-symbol-in-A* [*simp*]: $l \in list(symbol) \Longrightarrow hd(l) \in M$
by (*rule nat-in-A*, *rule symbol-nat*, *erule hd-symbol-type*)

lemma *tl-symbols* [*simp*]: $l \in list(symbol) \Longrightarrow tl(l) \in list(symbol)$
by (*rule list.cases*[*of l*]) *auto*

lemma *tl-symbols-in-A* [*simp*]: $l \in list(symbol) \Longrightarrow tl(l) \in M$
by *simp*

end

definition *upd-fm* :: [*i*,*i*,*i*,*i*,*i*] ⇒ *i* **where**
upd-fm(*a*,*l*,*r*,*l2*,*r2*) ≡
Or(*And*(*const-fm*(*0*,*a*), *And*(*Equal*(*l2*,*l*),
Exists(*Exists*(*And*(*const-fm*(*0*,*1*), *And*(*tl-fm*(*r#+2*,*0*),
Cons-fm(*1*,*0*,*r2#+2*)))))),
Or(*And*(*const-fm*(*1*,*a*), *And*(*Equal*(*l2*,*l*),
Exists(*Exists*(*And*(*const-fm*(*1*,*1*), *And*(*tl-fm*(*r#+2*,*0*),
Cons-fm(*1*,*0*,*r2#+2*)))))),
Or(*And*(*const-fm*(*2*,*a*),
Exists(*Exists*(*And*(*tl-fm*(*l#+2*,*0*), *And*(*hd-fm*(*l#+2*,*1*),
And(*Equal*(*l2#+2*,*0*), *Cons-fm*(*1*,*r#+2*,*r2#+2*)))))),
Or(*And*(*const-fm*(*3*,*a*),
Exists(*Exists*(*And*(*hd-fm*(*r#+2*,*1*), *And*(*tl-fm*(*r#+2*,*0*),
And(*Cons-fm*(*1*,*l#+2*,*l2#+2*), *Equal*(*r2#+2*,*0*)))))),
And(*Neg*(*Or*(*const-fm*(*0*,*a*), *Or*(*const-fm*(*1*,*a*), *Or*(*const-fm*(*2*,*a*),

const-fm(3,*a*))))),
And(*Equal*(*l2*,*l*), *Equal*(*r2*,*r*)))))))

lemma *upd-fm-type* [*TC*]:
⟦*a* ∈ *nat*; *l* ∈ *nat*; *r* ∈ *nat*; *l2* ∈ *nat*; *r2* ∈ *nat*⟧
⟹ *upd-fm*(*a*,*l*,*r*,*l2*,*r2*) ∈ *formula*
unfolding *upd-fm-def* **by** *simp*

context *M-ZF1-trans*
begin

lemma *sats-upd-fm*:
assumes *idx*: *ia* ∈ *nat il* ∈ *nat ir* ∈ *nat il2* ∈ *nat ir2* ∈ *nat*
and *env*: *env* ∈ *list*(*M*)
and *a*: *nth*(*ia*,*env*) = *a*
and *l*: *nth*(*il*,*env*) = *l l* ∈ *list*(*symbol*)
and *r*: *nth*(*ir*,*env*) = *r r* ∈ *list*(*symbol*)
and *l2*: *nth*(*il2*,*env*) = *l2 l2* ∈ *M*
and *r2*: *nth*(*ir2*,*env*) = *r2 r2* ∈ *M*
shows (*M*, *env* ⊨ *upd-fm*(*ia*,*il*,*ir*,*il2*,*ir2*)) ⟷ ⟨*l2*,*r2*⟩ = *tm-update*(*a*,⟨*l*,*r*⟩)
using *idx env a l r l2 r2*
by (*simp add*: *upd-fm-def tm-update-def*)

end

14.3 Iterated tails

definition *drop-fm* :: [*i*,*i*,*i*] ⇒ *i* **where**
drop-fm(*n*,*l*,*d*) ≡
Exists(*And*(*Exists*(*And*(*empty-fm*(0), *fun-apply-fm*(1,0,*l*#+2))),
And(*Forall*(*Implies*(*Member*(0,*n*#+2),
Exists(*Exists*(*Exists*(*And*(*succ-fm*(3,2),
And(*fun-apply-fm*(4,3,1), *And*(*fun-apply-fm*(4,2,0),
tl-fm(1,0)))))))),
fun-apply-fm(0,*n*#+1,*d*#+1))))

The witness for *drop-fm* records successive list tails. At the outer existential quantifier its index is 0. Under the bounded universal quantifier and three further existential quantifiers, the environment has the prefix
[*f*‘*succ*(*k*),*f*‘*k*,*succ*(*k*),*k*,*f*].

The recurrence relates entries 1 and 0 by *tl-fm*. The initial and final applications select the original list and the required tail.

lemma *drop-fm-type* [*TC*]:
⟦*n* ∈ *nat*; *l* ∈ *nat*; *d* ∈ *nat*⟧ ⟹ *drop-fm*(*n*,*l*,*d*) ∈ *formula*
unfolding *drop-fm-def* **by** *simp*

definition *drop-witness* :: [*i*,*i*,*i*,*i*] ⇒ *o* **where**
drop-witness(*f*,*n*,*l*,*d*) ≡ *f*‘0 = *l* ∧ (∀ *k*∈*n*. *f*‘*succ*(*k*) =
tl′(*f*‘*k*)) ∧ *f*‘*n* = *d*

definition *tm-drop-tail* :: $[i,i] \Rightarrow i$ **where**
  tm-drop-tail$(n,l) \equiv (\lambda x.\ tl'(x))\hat{}n\ (l)$

lemma *tm-drop-tail-0* [*simp*]: *tm-drop-tail*$(0,l) = l$
  unfolding *tm-drop-tail-def* **by** *simp*

lemma *tm-drop-tail-succ* [*simp*]:
  $n \in nat \Longrightarrow$
    tm-drop-tail$(succ(n),l) =$ *tl'*$($*tm-drop-tail*$(n,l))$
  unfolding *tm-drop-tail-def* **by** *simp*

lemma *tm-drop-tail-list*:
  assumes *l*: $l \in list(B)$ **and** *n*: $n \in nat$
  shows *tm-drop-tail*$(n,l) =$ *drop*(n,l)
  using *n*
proof (*induct n rule*: *nat-induct*)
  case *0* **then show** *?case* **by** *simp*
next
  case (*succ n*)
  have *drop*$(n,l) \in list(B)$ **using** *succ.hyps*(*1*) *l* **by** *simp*
  then show *?case* **using** *succ* **by** (*simp add*: *drop-succ tl'-def list-imp-quasilist*)
qed

context *M-ZF1-trans*
begin

lemma *sats-drop-fm-witness*:
  assumes *idx*: *jn* $\in$ *nat il* $\in$ *nat jd* $\in$ *nat* **and** *env*: *env* $\in$ *list*(M)
    and *n*: *nth*$(jn,env) = n\ n \in nat$
    and *l*: *nth*$(il,env) = l\ l \in M$
    and *d*: *nth*$(jd,env) = d\ d \in M$
  shows $(M,\ env \models$ *drop-fm*$(jn,il,jd)) \longleftrightarrow (\exists f{\in}M.$ *drop-witness*$(f,n,l,d))$
proof −
  have *nA*: $n \in M$ **using** *n* **by** *simp*
  have *k-nat*: $\bigwedge k.\ k \in n \Longrightarrow k \in nat$ **using** *n* **by**
    (*blast intro*: *Ord-trans*)
  have *k-A*: $\bigwedge k.\ k \in n \Longrightarrow k \in M$ **using** *k-nat* **by** *simp*
  have *sk-A*: $\bigwedge k.\ k \in n \Longrightarrow succ(k) \in M$ **using** *k-nat*
    by *simp*
  have *ap-A*: $\bigwedge f\ x.\ f \in M \Longrightarrow x \in M \Longrightarrow f`x \in M$ **using** *apply-closed* **by** *simp*
  show *?thesis*
    unfolding *drop-fm-def drop-witness-def*
    using *idx env n l d nA k-A sk-A ap-A* **by** (*auto simp add*: *zero-in-M*)
qed

lemma *tl'-in-A* [*simp*]: $x \in M \Longrightarrow tl'(x) \in M$
proof (*cases quasilist*(x))
  case *True*

```
  assume xA: x ∈ M
  from True show ?thesis
  proof (unfold quasilist-def, elim disjE exE)
    assume x = [] then show ?thesis using Nil-in-M by (simp add: tl'-def)
  next
    fix a t assume x = Cons(a,t)
    with xA show ?thesis using Cons-in-M-iff by (simp add: tl'-def)
  qed
next
  case False then show ?thesis using zero-in-M by (simp add: tl'-def)
qed

lemma tm-drop-tail-in-A [simp]:
  assumes l: l ∈ M and n: n ∈ nat
  shows tm-drop-tail(n,l) ∈ M
  using n by (induct n rule: nat-induct) (simp-all add: l)

lemma drop-witness-iff:
  assumes n: n ∈ nat and l: l ∈ M and d: d ∈ M
  shows (∃f∈M. drop-witness(f,n,l,d)) ⟷ d = tm-drop-tail(n,l)
proof
  assume ∃f∈M. drop-witness(f,n,l,d)
  then obtain f where f0: f`0 = l and fs: ⋀k. k ∈ n ⟹ f`succ(k) = tl'(f`k)
    and fn: f`n = d unfolding drop-witness-def by blast
  have ⋀k. k ∈ nat ⟹ k ≤ n ⟶ f`k = tm-drop-tail(k,l)
  proof −
    fix k assume k: k ∈ nat
    then show k ≤ n ⟶ f`k = tm-drop-tail(k,l)
    proof (induct k rule: nat-induct)
      case 0 show ?case using f0 by simp
    next
      case (succ k)
      show ?case
      proof
        assume le: succ(k) ≤ n
        then have kn: k ∈ n using succ.hyps(1) n by (simp add: ltD)
        have k ≤ n using le by (blast intro: leI)
        then have f`k = tm-drop-tail(k,l) using succ.hyps(2) by blast
        then show f`succ(k) = tm-drop-tail(succ(k),l)
          using fs[OF kn] succ.hyps(1)
          by simp
      qed
    qed
  qed
  from this[OF n] fn n show d = tm-drop-tail(n,l) by simp
next
  assume dd: d = tm-drop-tail(n,l)
  let ?f = λk∈succ(n). tm-drop-tail(k,l)
  have fA: ?f ∈ M
```

```
  proof −
    have (##M)(?f)
    proof (rule tm-finite-lambda-closed[OF nat-succI[OF n]])
      fix k assume k ∈ succ(n)
      then have k ∈ nat using n by (blast intro: Ord-trans)
      then show (##M)(tm-drop-tail(k,l)) using l by simp
    qed
    then show ?thesis by simp
  qed
  have drop-witness(?f,n,l,d)
    unfolding drop-witness-def
  proof (intro conjI ballI)
    show ?f‘0 = l using ltD[OF nat-0-le[OF n]] by simp
    show ?f‘n = d using dd by simp
  next
    fix k assume k: k ∈ n
    then have knat: k ∈ nat using n by (blast intro: Ord-trans)
    have succ(k) ∈ succ(n) using k n by (simp add: Ord-succ-mem-iff)
    moreover have k ∈ succ(n) using k by (rule succI2)
    ultimately show ?f‘succ(k) = tl'(?f‘k) using knat by simp
  qed
  with fA show ∃f∈M. drop-witness(f,n,l,d) by blast
qed

lemma sats-drop-fm [simp]:
  assumes idx: jn ∈ nat il ∈ nat jd ∈ nat and env: env ∈ list(M)
    and n: nth(jn,env) ∈ nat and l: nth(il,env) ∈ M and d:
      nth(jd,env) ∈ M
  shows (M, env ⊨ drop-fm(jn,il,jd)) ⟷
    nth(jd,env) = tm-drop-tail(nth(jn,env),nth(il,env))
  using sats-drop-fm-witness[OF idx env refl n refl l refl d]
    drop-witness-iff[OF n l d]
  by simp

end
```

14.4 Fetching an instruction

```
lemma nth-via-drop:
  assumes P: P ∈ list(B) and s: s ∈ nat
  shows (if s < length(P) then nth(s,P) else c) =
    (if drop(s,P) = [] then c else hd(drop(s,P)))
proof (cases s < length(P))
  case True
  then have s ∈ length(P) by (rule ltD)
  then obtain z zs where dz: drop(s,P) = Cons(z,zs) using drop-length[OF P]
by
    blast
  have nth(0, drop(s,P)) = nth(s #+ 0, P) using nth-drop[OF s, of 0 P B] P by
```

```
    simp
  then have nth(s,P) = z using dz s by simp
  then show ?thesis using True dz by simp
next
  case False
  then have length(P) ≤ s using s P by (simp add: not-lt-iff-le)
  then have drop(s,P) = [] using drop-all[OF s P] by simp
  then show ?thesis using False by simp
qed

lemma fetch-via-drop:
  assumes P: P ∈ list(B) and q: q ∈ nat and b: b ∈ nat
  shows fetch(P,q,b) = (if q = 0 then ⟨nop,0⟩
     else if drop(b, drop(pred(q), drop(pred(q),P))) = [] then ⟨nop,0⟩
     else hd(drop(b, drop(pred(q), drop(pred(q),P)))))
proof (cases q = 0)
  case True then show ?thesis by (simp add: fetch-def)
next
  case False
  then obtain p where p: p ∈ nat and qp: q = succ(p) using q by (blast
    elim: natE)
  have dd: drop(b, drop(p, drop(p,P))) = drop(p #+ p #+ b, P)
  proof −
    have pp: p #+ p ∈ nat by simp
    have drop(p, drop(p,P)) = drop(p #+ p, P) using drop-drop[OF p] P p by
      simp
    moreover have drop(b, drop(p #+ p, P)) = drop(b #+ (p #+ p), P)
      using drop-drop[OF pp] P b by simp
    ultimately show ?thesis by (simp add: add-commute)
  qed
  have sn: p #+ p #+ b ∈ nat by simp
  show ?thesis using False qp p dd nth-via-drop[OF P sn, of
    ⟨nop,0⟩]
    by (auto simp add: fetch-def split: split-if-asm)
qed

definition nop-instr-fm :: i ⇒ i where
  nop-instr-fm(i) ≡ Exists(Exists(And(const-fm(nop,1),
    And(const-fm(0,0), pair-fm(1,0,i#+2)))))

lemma nop-instr-fm-type [TC]: i ∈ nat ⟹ nop-instr-fm(i) ∈ formula
  unfolding nop-instr-fm-def by simp

definition fetch-fm :: [i,i,i,i] ⇒ i where
  fetch-fm(p,q,b,ins) ≡
    Or(And(const-fm(0,q), nop-instr-fm(ins)),
      And(Neg(const-fm(0,q)),
        Exists(And(succ-fm(0,q#+1),
          Exists(And(drop-fm(1,p#+2,0),
```

Exists(*And*(*drop-fm*(*2*,*1*,*0*),
Exists(*And*(*drop-fm*(*b*#+*4*,*1*,*0*),
Or(*And*(*Nil-fm*(*0*), *nop-instr-fm*(*ins*#+*4*)),
And(*Neg*(*Nil-fm*(*0*)), *hd-fm*(*0*,*ins*#+*4*)))))))))))

State 0 selects the no-op instruction. For a positive state q, list traversal drops $q - 1$ instructions twice and then drops the scanned symbol number. This selects index $2 * (q - 1) + b$ of the instruction list. An exhausted list also selects the no-op instruction. These cases match the total external function *fetch*.

lemma *fetch-fm-type* [*TC*]:
$[\![p \in nat;\ q \in nat;\ b \in nat;\ ins \in nat]\!] \Longrightarrow fetch\text{-}fm(p,q,b,ins) \in formula$
unfolding *fetch-fm-def* **by** *simp*

context *M-ZF1-trans*
begin

lemma *nth-in-A* [*simp*]: $env \in list(M) \Longrightarrow i \in nat \Longrightarrow nth(i,env) \in M$
proof (*cases* $i < length(env)$)
case *True*
assume $env \in list(M)$ **then show** *?thesis* **using** *True* **by** (*rule nth-type*)
next
case *False*
assume $env \in list(M)$ $i \in nat$
then have $nth(i,env) = 0$ **using** *False* **by** (*simp add*: *nth-eq-0 not-lt-iff-le*)
then show *?thesis* **using** *zero-in-M* **by** *simp*
qed

lemma *nop-in-A* [*simp*]: $nop \in M$
by (*rule nat-in-A*) *simp*

lemma *sats-nop-instr-fm* [*simp*]:
$[\![i \in nat;\ env \in list(M)]\!] \Longrightarrow (M,\ env \models nop\text{-}instr\text{-}fm(i)) \longleftrightarrow nth(i,env) = \langle nop,0\rangle$
unfolding *nop-instr-fm-def* **by** *auto*

lemma *machine-in-A* [*simp*]: $P \in tm\text{-}machine \Longrightarrow P \in M$
using *tm-machine-closed* **by** *simp*

lemma *sats-fetch-fm*:
assumes *idx*: $ip \in nat$ $iq \in nat$ $ib \in nat$ $ii \in nat$ **and**
env: $env \in list(M)$
and *P*: $nth(ip,env) = P$ $P \in tm\text{-}machine$
and *q*: $nth(iq,env) = q$ $q \in nat$
and *b*: $nth(ib,env) = b$ $b \in nat$
and *ins*: $nth(ii,env) = ins$ $ins \in M$
shows $(M,\ env \models fetch\text{-}fm(ip,iq,ib,ii)) \longleftrightarrow ins = fetch(P,q,b)$
proof −
have *PL*: $P \in list(instruction)$ **using** $P(2)$ **unfolding** *tm-machine-def* **.**

```
  show ?thesis
  proof (cases q = 0)
    case True
    then show ?thesis using idx env P q b ins PL
      by (simp add: fetch-fm-def fetch-via-drop[OF PL q(2) b(2)])
  next
    case False
    then obtain p where p: p ∈ nat and qp: q = succ(p) using q(2) by
      (blast elim: natE)
    have D1: drop(p,P) ∈ list(instruction) using p PL by simp
    have D2: drop(p,drop(p,P)) ∈ list(instruction) using p D1 by simp
    have D3: drop(b,drop(p,drop(p,P))) ∈ list(instruction) using b D2 by
      simp
    have D1A: drop(p,P) ∈ M
      using tm-drop-tail-in-A[OF machine-in-A[OF P(2)] p]
        tm-drop-tail-list[OF PL p] by simp
    have D2A: drop(p,drop(p,P)) ∈ M
      using tm-drop-tail-in-A[OF D1A p] tm-drop-tail-list[OF D1 p] by simp
    have D3A: drop(b,drop(p,drop(p,P))) ∈ M
     using tm-drop-tail-in-A[OF D2A b(2)] tm-drop-tail-list[OF D2 b(2)] by simp
    show ?thesis using idx env P q b ins PL qp p D1 D2 D3 D1A D2A D3A
      by (simp add: fetch-fm-def fetch-via-drop[OF PL nat-succI[OF p] b(2)]
        tm-drop-tail-list)
  qed
qed

lemma sats-upd-fm-simp [simp]:
  ⟦ia ∈ nat; il ∈ nat; ir ∈ nat; il2 ∈ nat; ir2 ∈ nat; env ∈ list(M);
    nth(il,env) ∈ list(symbol); nth(ir,env) ∈ list(symbol)⟧ ⟹
   (M, env ⊨ upd-fm(ia,il,ir,il2,ir2)) ⟷ ⟨nth(il2,env),nth(ir2,env)⟩ =
      tm-update(nth(ia,env),⟨nth(il,env),nth(ir,env)⟩)
  by (rule sats-upd-fm) simp-all

lemma sats-fetch-fm-simp [simp]:
  ⟦ip ∈ nat; iq ∈ nat; ib ∈ nat; ii ∈ nat; env ∈ list(M);
    nth(ip,env) ∈ tm-machine; nth(iq,env) ∈ nat; nth(ib,env) ∈ nat⟧ ⟹
   (M, env ⊨ fetch-fm(ip,iq,ib,ii)) ⟷
     nth(ii,env) = fetch(nth(ip,env),nth(iq,env),nth(ib,env))
  by (rule sats-fetch-fm) simp-all

end
```

14.5 One step

```
definition step-fm :: [i,i,i] ⇒ i where
  step-fm(p,c,d) ≡
    Exists(Exists(Exists(Exists(Exists(Exists(
      Exists(Exists(Exists(Exists(Exists(
      And(pair-fm(10,9,c#+11),
```

And(*pair-fm*(*8*,*7*,*9*),
And(*hd-fm*(*7*,*6*),
And(*fetch-fm*(*p*#+*11*,*10*,*6*,*5*),
And(*pair-fm*(*4*,*3*,*5*),
And(*upd-fm*(*4*,*8*,*7*,*2*,*1*),
And(*pair-fm*(*2*,*1*,*0*),
pair-fm(*3*,*0*,*d*#+*11*)))))))))))))))))

At the innermost conjunction, the eleven existentially bound variables form the environment prefix $[t',r',l',q',a,i,b,r,l,t,q]$, in index order from 0 to 10. The original configuration is $\langle q,t\rangle$ with $t = \langle l,r\rangle$. The scanned symbol is $b = hd(r)$, the fetched instruction is $i = \langle a,q'\rangle$, and the updated tape is $t' = \langle l',r'\rangle$. The free indices for the machine and the two configurations shift by eleven. The final pair clause assembles the output configuration $\langle q',t'\rangle$. Each conjunction therefore corresponds to one component of the external *tm-step* definition.

lemma *step-fm-type* [*TC*]:
$[\![p \in nat;\ c \in nat;\ d \in nat]\!] \Longrightarrow step\text{-}fm(p,c,d) \in formula$
unfolding *step-fm-def* **by** *simp*

context *M-ZF1-trans*
begin

lemma *configuration-in-A* [*simp*]: $c \in configuration \Longrightarrow c \in M$
using *tm-configuration-closed* **by** *simp*

lemma *sats-step-fm*:
assumes *idx*: $ip \in nat$ $ic \in nat$ $jd \in nat$ **and** *env*: $env \in list(M)$
and *P*: $nth(ip,env) = P$ $P \in tm\text{-}machine$
and *c*: $nth(ic,env) = c$ $c \in configuration$
and *d*: $nth(jd,env) = d$ $d \in M$
shows $(M,\ env \models step\text{-}fm(ip,ic,jd)) \longleftrightarrow d = tm\text{-}step(P,c)$
proof −
from *c*(*2*) **obtain** *q l r* **where** *cq*: $c = \langle q,\langle l,r\rangle\rangle$ **and** *q*: $q \in nat$
and *l*: $l \in list(symbol)$ **and** *r*: $r \in list(symbol)$
unfolding *configuration-def tm-tape-def* **by** *blast*
have *b*: $hd(r) \in nat$ **using** *r* **by** (*blast intro*: *symbol-nat hd-symbol-type*)
have *lrA*: $\langle l,r\rangle \in M$ **using** *pair-in-M-iff l r* **by** *simp*
have $fetch(P,q,hd(r)) \in instruction$ **using** *P*(*2*) **by** (*rule fetch-type*)
then obtain *fa fq* **where** *fe*: $fetch(P,q,hd(r)) = \langle fa,fq\rangle$ **and**
fa: $fa \in action$
and *fq*: $fq \in nat$ **unfolding** *instruction-def* **by** *blast*
have *faA*: $fa \in M$ **using** *fa action-in-nat Ord-nat* **by** (*blast dest*: *OrdmemD*
intro: *nat-in-A*)
have $tm\text{-}update(fa,\langle l,r\rangle) \in tm\text{-}tape$ **by** (*rule tm-update-type*)
(*simp add*: *l r tm-tape-def*)
then obtain *u1 u2* **where** *ue*: $tm\text{-}update(fa,\langle l,r\rangle) = \langle u1,u2\rangle$
and *u1*: $u1 \in list(symbol)$ **and** *u2*: $u2 \in list(symbol)$ **unfolding**
tm-tape-def **by** *blast*

```
  have feA: ⟨fa,fq⟩ ∈ M using pair-in-M-iff faA fq by simp
  have ueA: ⟨u1,u2⟩ ∈ M using pair-in-M-iff u1 u2 by simp
  show ?thesis
    using idx env P c d cq q l r b lrA fe fa fq faA ue u1 u2 feA ueA
    by (simp add: step-fm-def)
qed

end
```

14.6 Run formulas

```
definition init-fm :: [i,i] ⇒ i where
  init-fm(r,x) ≡
    Exists(Exists(Exists(Exists(Exists(
      And(empty-fm(4), And(const-fm(1,3), And(Nil-fm(2),
      And(pair-fm(2,x#+5,1), And(pair-fm(3,1,0),
        fun-apply-fm(r#+5,4,0)))))))))))

definition trans-fm :: [i,i,i] ⇒ i where
  trans-fm(p,r,n) ≡
    Forall(Implies(Member(0,n#+1),
      Exists(Exists(Exists(
        And(succ-fm(3,2), And(fun-apply-fm(r#+4,3,1),
        And(fun-apply-fm(r#+4,2,0), step-fm(p#+4,1,0)))))))))

definition final-fm :: [i,i] ⇒ i where
  final-fm(r,n) ≡
    Exists(Exists(Exists(
      And(empty-fm(2), And(fun-apply-fm(r#+3,n#+3,1), pair-fm(2,0,1)))))))

definition halt-fm :: i where
  halt-fm ≡
    Exists(And(nat-member-fm(0),
      Exists(And(init-fm(0,3), And(trans-fm(2,0,1), final-fm(0,1)))))))
```

In *init-fm*, the five bound entries, in index order, are the initial configuration, its tape, the empty left list, state 1, and the time 0. In *trans-fm*, the bounded variable k and three existential witnesses give the environment prefix $[r`succ(k),r`k,succ(k),k]$. The call to *step-fm* relates indices 1 and 0 and shifts the machine index by four. In *final-fm*, the three witnesses are the final tape, the final configuration, and state 0, in index order.

The free environment of *halt-fm* is $[P,x]$. The two outer existential quantifiers bind a natural time and a run witness, producing $[r,n,P,x]$. Consequently the initial clause uses indices $(0,3)$, the transition clause uses $(2,0,1)$, and the final clause uses $(0,1)$. The run witness need only satisfy these finite application constraints. The external halting run supplies a finite function in the model, and induction identifies the constrained values of any internal witness with the external execution.

lemma *init-fm-type* [*TC*]: ⟦*r* ∈ *nat*; *x* ∈ *nat*⟧ ⟹ *init-fm*(*r*,*x*) ∈ *formula*
unfolding *init-fm-def* **by** *simp*

lemma *trans-fm-type* [*TC*]:
⟦*p* ∈ *nat*; *r* ∈ *nat*; *n* ∈ *nat*⟧ ⟹ *trans-fm*(*p*,*r*,*n*) ∈ *formula*
unfolding *trans-fm-def* **by** *simp*

lemma *final-fm-type* [*TC*]: ⟦*r* ∈ *nat*; *n* ∈ *nat*⟧ ⟹ *final-fm*(*r*,*n*) ∈ *formula*
unfolding *final-fm-def* **by** *simp*

lemma *halt-fm-type* [*TC*]: *halt-fm* ∈ *formula*
unfolding *halt-fm-def* **by** *simp*

context *M-ZF1-trans*
begin

lemma *ap-in-A* [*simp*]: *f* ∈ *M* ⟹ *a* ∈ *M* ⟹ *f*‘*a* ∈ *M*
using *apply-closed* **by** *simp*

lemma *sats-init-fm* [*simp*]:
⟦*ir* ∈ *nat*; *ix* ∈ *nat*; *env* ∈ *list*(*M*)⟧ ⟹
(*M*, *env* ⊨ *init-fm*(*ir*,*ix*)) ⟷ *nth*(*ir*,*env*)‘0 = *initial-config*(*nth*(*ix*,*env*))
unfolding *init-fm-def* **by** (*auto simp add*: *zero-in-M*)

lemma *sats-final-fm* [*simp*]:
⟦*ir* ∈ *nat*; *jn* ∈ *nat*; *env* ∈ *list*(*M*)⟧ ⟹
(*M*, *env* ⊨ *final-fm*(*ir*,*jn*)) ⟷
(∃ *t*∈*M*. *nth*(*ir*,*env*)‘*nth*(*jn*,*env*) = ⟨0,*t*⟩)
unfolding *final-fm-def* **by** (*auto simp add*: *zero-in-M*)

lemma *sats-trans-fm* [*simp*]:
assumes *ip* ∈ *nat ir* ∈ *nat jn* ∈ *nat env* ∈ *list*(*M*)
nth(*jn*,*env*) ∈ *nat*
shows (*M*, *env* ⊨ *trans-fm*(*ip*,*ir*,*jn*)) ⟷
(∀ *k*∈*nth*(*jn*,*env*). (*M*, *Cons*(*nth*(*ir*,*env*)‘*succ*(*k*),
Cons(*nth*(*ir*,*env*)‘*k*,
Cons(*succ*(*k*), *Cons*(*k*, *env*)))) ⊨ *step-fm*(*ip*#+4,1,0)))
proof −
have *kA*: ⋀*k*. *k* ∈ *nth*(*jn*,*env*) ⟹ *k* ∈ *M* ∧ *succ*(*k*) ∈ *M*
using *assms*(5) **by** (*blast intro*: *Ord-trans nat-in-A*)
show *?thesis* **unfolding** *trans-fm-def* **using** *assms kA* **by** *auto*
qed

14.7 Run correspondence

The following two lemmas separate finite execution from the layout of a surrounding formula. The reconstruction lemma accepts any initial configuration and any outer environment. Its induction establishes configuration validity along the run before applying the one-step theorem. The converse

lemma supplies a transition from the canonical finite trace. These same interfaces support the halting proof and applications that constrain a run's output.

```
lemma trans-run-steps:
  assumes env: env ∈ list(M) and p: p ∈ nat
    and P: nth(p,env) = P P ∈ tm-machine
    and c0: c0 ∈ configuration and n: n ∈ nat
    and rA: r ∈ M and r0: r`0 = c0
    and rs: ⋀k. k ∈ n ⟹
      (M, Cons(r`succ(k), Cons(r`k, Cons(succ(k), Cons(k, env))))
        ⊨ step-fm(p#+4,1,0))
  shows r`n = steps(P,c0,n)
proof −
  have ⋀k. k ∈ nat ⟹ k ≤ n ⟶ r`k = steps(P,c0,k)
  proof −
    fix k assume k: k ∈ nat
    then show k ≤ n ⟶ r`k = steps(P,c0,k)
    proof (induct k rule: nat-induct)
      case 0 show ?case using r0 by simp
    next
      case (succ k)
      show ?case
      proof
        assume le: succ(k) ≤ n
        then have kn: k ∈ n
          using succ.hyps(1) n by (simp add: succ-le-iff ltD)
        have k ≤ n using le by (blast intro: leI succ-leE)
        then have rk: r`k = steps(P,c0,k) using succ.hyps(2) by blast
        have conf: r`k ∈ configuration
          using rk steps-type[OF P(2) c0 succ.hyps(1)] by simp
        have kA: k ∈ M succ(k) ∈ M using succ.hyps(1) by simp-all
        have envA:
          Cons(r`succ(k), Cons(r`k, Cons(succ(k), Cons(k, env)))) ∈ list(M)
          using rA kA env by simp
        have r`succ(k) = tm-step(P,r`k)
          using sats-step-fm[OF - - - envA, of p#+4 1 0 P r`k r`succ(k)]
            rs[OF kn] P p conf rA kA
          by simp
        then show r`succ(k) = steps(P,c0,succ(k))
          using rk succ.hyps(1) by simp
      qed
    qed
  qed
  from this[OF n] n show ?thesis by (simp add: nat-into-Ord)
qed

lemma trace-trans:
  assumes env: env ∈ list(M) and p: p ∈ nat
    and P: nth(p,env) = P P ∈ tm-machine
```

and *c0*: $c0 \in configuration$ **and** *n*: $n \in nat$ **and** *k*: $k \in n$
shows $(M,\ Cons((\lambda j{\in}succ(n).\ steps(P,c0,j))`succ(k),$
$Cons((\lambda j{\in}succ(n).\ steps(P,c0,j))`k,$
$Cons(succ(k),\ Cons(k,\ env)))) \models step\text{-}fm(p\#{+}4,1,0))$
proof −
have *knat*: $k \in nat$ **using** *k n* **by** (*blast intro*: *Ord-trans Ord-nat*)
have *sk*: $succ(k) \in succ(n)$
using *k n* **by** (*simp add*: *Ord-succ-mem-iff nat-into-Ord*)
have *k1*: $k \in succ(n)$ **using** *k* **by** (*rule succI2*)
have *conf*: $steps(P,c0,k) \in configuration$ **using** *steps-type*[*OF P*(*2*) *c0 knat*] **.**
have *conf2*: $steps(P,c0,succ(k)) \in configuration$
using *steps-type*[*OF P*(*2*) *c0 nat-succI*[*OF knat*]] **.**
have *kA*: $k \in M\ succ(k) \in M$ **using** *knat* **by** *simp-all*
have *envA*: $Cons(steps(P,c0,succ(k)),$
$Cons(steps(P,c0,k),\ Cons(succ(k),\ Cons(k,\ env)))) \in list(M)$
using *kA env conf conf2* **by** *simp*
show *?thesis*
using *sats-step-fm*[*OF - - - envA*, *of p*#+*4 1 0 P*
$steps(P,c0,k)\ steps(P,c0,succ(k))$]
P p conf conf2 sk k1 knat
by *simp*
qed

14.8 Halting satisfaction

lemma *halting-witness-iff*:
assumes *P*: $P \in tm\text{-}machine$ **and** *x*: $x \in list(symbol)$
shows $(\exists\, n{\in}M.\ n \in nat \wedge (\exists\, r{\in}M.\ r`0 = initial\text{-}config(x) \wedge$
$(\forall\, k{\in}n.\ (M,\ Cons(r`succ(k),\ Cons(r`k,\ Cons(succ(k),\ Cons(k,$
$[r,n,P,x])))) \models step\text{-}fm(6,1,0))) \wedge (\exists\, t{\in}M.\ r`n = \langle 0,t\rangle))) \longleftrightarrow$
$halts\text{-}on(P,x)$
proof
assume $\exists\, n{\in}M.\ n \in nat \wedge (\exists\, r{\in}M.\ r`0 = initial\text{-}config(x) \wedge$
$(\forall\, k{\in}n.\ (M,\ Cons(r`succ(k),\ Cons(r`k,\ Cons(succ(k),\ Cons(k,$
$[r,n,P,x])))) \models step\text{-}fm(6,1,0))) \wedge (\exists\, t{\in}M.\ r`n = \langle 0,t\rangle))$
then obtain *n r t* **where** *n*: $n \in nat\ n \in M$ **and** *rA*: $r \in M$
and *r0*: $r`0 = initial\text{-}config(x)$
and *rs*: $\bigwedge k.\ k \in n \Longrightarrow (M,\ Cons(r`succ(k),\ Cons(r`k,$
$Cons(succ(k),\ Cons(k,\ [r,n,P,x])))) \models step\text{-}fm(6,1,0))$
and *tA*: $t \in M$ **and** *rn*: $r`n = \langle 0,t\rangle$
by *blast*
have *init*: $initial\text{-}config(x) \in configuration$ **using** *x* **by** (*rule*
initial-config-type)
have *env*: $[r,n,P,x] \in list(M)$ **using** *rA n P x* **by** *simp*
have *run*: $r`n = steps(P,initial\text{-}config(x),n)$
by (*rule trans-run-steps*[*OF env*, *of 2 P initial-config*(*x*) *n r*])
(*use P init n rA r0 rs* **in** *auto*)
from *rn* **have** $fst(r`n) = final\text{-}state$ **by** *simp*
with *run* **have** $fst(steps(P,initial\text{-}config(x),n)) = final\text{-}state$ **by** *simp*

with *P x n(1)* **show** *halts-on(P,x)* **unfolding** *halts-on-def* **by** *blast*
next
assume *h*: *halts-on(P,x)*
then obtain *n* **where** *n*: *n* ∈ *nat* **and** *fin*:
fst(steps(P,initial-config(x),n)) = final-state
unfolding *halts-on-def* **by** *blast*
have *init*: *initial-config(x)* ∈ *configuration* **using** *x* **by** (*rule initial-config-type*)
let *?r* = λ*k*∈*succ(n)*. *steps(P,initial-config(x),k)*
have *rA*: *?r* ∈ *M* **using** *tm-trace-closed*[*OF P init n*] **by** *simp*
have *nA*: *n* ∈ *M* **using** *n* **by** *simp*
have *xA*: *x* ∈ *M* **using** *x* **by** *simp*
have *r0*: *?r‘0 = initial-config(x)* **using** *ltD*[*OF nat-0-le*[*OF n*]] **by** *simp*
have *env*: [*?r*,*n*,*P*,*x*] ∈ *list(M)* **using** *rA nA P xA* **by** *simp*
have *rs*: ⋀*k*. *k* ∈ *n* ⟹ (*M*, *Cons(?r‘succ(k)*, *Cons(?r‘k*, *Cons(succ(k)*, *Cons(k*, [*?r*,*n*,*P*,*x*])))) ⊨ *step-fm(6*,*1*,*0)*)
using *trace-trans*[*OF env - - P init n*, **where** *p=2*] **by** *simp*
have *steps(P,initial-config(x),n)* ∈ *configuration* **using** *steps-type*[*OF P init n*] **.**
then obtain *t* **where** *st*: *steps(P,initial-config(x),n)* = ⟨*0*,*t*⟩
and *t*: *t* ∈ *tm-tape*
using *fin* **unfolding** *configuration-def* **by** *auto*
have *tA*: *t* ∈ *M* **using** *t pair-in-M-iff* **unfolding** *tm-tape-def* **by** *auto*
have *rn*: *?r‘n* = ⟨*0*,*t*⟩ **using** *st* **by** *simp*
show ∃ *n*∈*M*. *n* ∈ *nat* ∧ (∃ *r*∈*M*. *r‘0 = initial-config(x)* ∧
(∀ *k*∈*n*. (*M*, *Cons(r‘succ(k)*, *Cons(r‘k*, *Cons(succ(k)*, *Cons(k*, [*r*,*n*,*P*,*x*])))) ⊨ *step-fm(6*,*1*,*0)*)) ∧ (∃ *t*∈*M*. *r‘n* = ⟨*0*,*t*⟩))
using *nA n rA r0 rs tA rn* **by** *blast*
qed

theorem *sats-halt-fm*:
assumes *P*: *P* ∈ *tm-machine* **and** *x*: *x* ∈ *list(symbol)*
shows (*M*, [*P*,*x*] ⊨ *halt-fm*) ⟷ *halts-on(P,x)*
proof −
have *xA*: *x* ∈ *M* **using** *x* **by** *simp*
have (*M*, [*P*,*x*] ⊨ *halt-fm*) ⟷
(∃ *n*∈*M*. *n* ∈ *nat* ∧ (∃ *r*∈*M*. *r‘0 = initial-config(x)* ∧
(∀ *k*∈*n*. (*M*, *Cons(r‘succ(k)*, *Cons(r‘k*, *Cons(succ(k)*, *Cons(k*, [*r*,*n*,*P*,*x*])))) ⊨ *step-fm(6*,*1*,*0)*)) ∧ (∃ *t*∈*M*. *r‘n* = ⟨*0*,*t*⟩)))
using *P xA sats-nat-member-fm*[*OF M-ZF1-trans-axioms*]
by (*auto simp add*: *halt-fm-def*)
then show *?thesis* **using** *halting-witness-iff*[*OF P x*] **by** *simp*
qed

end

theorem *transitive-model-halt-fm-iff*:
assumes *transitive-zf-model(A) P* ∈ *tm-machine x* ∈ *list(symbol)*
shows (*A*, [*P*,*x*] ⊨ *halt-fm*) ⟷ *halts-on(P,x)*

by (*rule M-ZF1-trans.sats-halt-fm*[*OF transitive-zf-M-ZF1-trans*[*OF assms*(*1*)] *assms*(*2*,*3*)])

14.9 Arities

context
notes *Un-assoc*[*symmetric*,*simp*] *Un-trasposition-aux1*[*simp*]
begin
arity-theorem for *hd-fm*
end

declare *arity-nat-name-fm*[*arity*]

context
notes *Un-assoc*[*symmetric*,*simp*] *Un-trasposition-aux1*[*simp*]
begin
arity-theorem for *const-fm*
arity-theorem for *upd-fm*
arity-theorem for *drop-fm*
arity-theorem for *nop-instr-fm*
arity-theorem for *fetch-fm*
arity-theorem for *step-fm*
arity-theorem for *init-fm*
arity-theorem for *trans-fm*
arity-theorem for *final-fm*
arity-theorem for *nat-member-fm*
arity-theorem for *halt-fm*
end

lemma *arity-halt-fm-eq*: *arity*(*halt-fm*) = *2*
using *arity-halt-fm* **by** (*simp add*: *ord-simp-union*)
end

15 Closed Halting Sentences

theory *Turing-Halting-Sentences*
imports *Turing-Halting-Formula Turing-Tape*
begin

Closed sentences are obtained from *halt-fm* by naming the machine and the input with formulas that define them in every transitive model. The resulting satisfaction equation applies to any finite machine and symbol input in a transitive ZF set model.

15.1 Names for finite lists

lemma *nat-name-sym-type*: $a \in \mathit{symbol} \Longrightarrow \mathit{nat\text{-}name\text{-}fm}(a) \in \mathit{formula}$
by (*rule nat-name-fm-type*, *erule symbol-nat*)

lemma *arity-nat-name-sym*: $a \in \mathit{symbol} \Longrightarrow \mathit{arity}(\mathit{nat\text{-}name\text{-}fm}(a)) = 1$
by (*rule arity-nat-name-fm*, *erule symbol-nat*)

lemma *sats-nat-name-sym*:
$[\![\mathit{M\text{-}ZF1\text{-}trans}(A);\ a \in \mathit{symbol};\ y \in A;\ \mathit{env} \in \mathit{list}(A)]\!] \Longrightarrow$
$(A,\ \mathit{Cons}(y,\mathit{env}) \models \mathit{nat\text{-}name\text{-}fm}(a)) \longleftrightarrow y = a$
by (*rule sats-nat-name-fm*, *assumption*, *erule symbol-nat*)

definition *list-name-fm* :: $[i \Rightarrow i,\ i] \Rightarrow i$ **where**
list-name-fm$(N,l) \equiv$
list-rec(*Nil-fm*(*0*), $\lambda a\ t\ p$. *Exists*(*And*($N(a)$, *Exists*(*And*(p,
Cons-fm(*1*,*0*,*2*))))), l)

A list name has one free variable, at index 0. In the nonempty case it binds the head and then the tail. At *Cons-fm*(*1*,*0*,*2*), the environment prefix consists of the tail, the head, and the list being named. An instruction name uses the same binding pattern for its action and next-state fields, joined by *pair-fm*(*1*,*0*,*2*).

lemma *list-name-fm-Nil* [*simp*]: *list-name-fm*$(N,[]) =$ *Nil-fm*(*0*)
unfolding *list-name-fm-def* **by** *simp*

lemma *list-name-fm-Cons* [*simp*]:
list-name-fm$(N,\mathit{Cons}(a,t)) =$ *Exists*(*And*($N(a)$, *Exists*(*And*(*list-name-fm*(N,t),
Cons-fm(*1*,*0*,*2*)))))
unfolding *list-name-fm-def* **by** *simp*

lemma *list-name-fm-type* [*TC*]:
assumes l: $l \in \mathit{list}(B)$ **and** N: $\bigwedge a.\ a \in B \Longrightarrow N(a) \in \mathit{formula}$
shows *list-name-fm*$(N,l) \in \mathit{formula}$
proof (*rule list.induct*[*OF l*])
show *list-name-fm*$(N,[]) \in \mathit{formula}$ **by** *simp*
next
fix $a\ t$ **assume** a: $a \in B$ **and** t: *list-name-fm*$(N,t) \in \mathit{formula}$
have $N(a) \in \mathit{formula}$ **using** N[*OF a*] **.**
then show *list-name-fm*$(N,\mathit{Cons}(a,t)) \in \mathit{formula}$ **using** t **by** *simp*
qed

lemma *arity-list-name-fm*:
assumes l: $l \in \mathit{list}(B)$ **and** N: $\bigwedge a.\ a \in B \Longrightarrow N(a) \in \mathit{formula}$
and aN: $\bigwedge a.\ a \in B \Longrightarrow \mathit{arity}(N(a)) = 1$
shows *arity*(*list-name-fm*$(N,l)) = 1$
proof (*rule list.induct*[*OF l*])
show *arity*(*list-name-fm*$(N,[])) = 1$ **by** (*simp add*: *arity-Nil-fm*
ord-simp-union)
next
fix $a\ t$ **assume** a: $a \in B$ **and** tl: $t \in \mathit{list}(B)$ **and** ih:
arity(*list-name-fm*$(N,t)) = 1$
have t: *list-name-fm*$(N,t) \in \mathit{formula}$ **using** $tl\ N$ **by** (*rule*
list-name-fm-type)

show *arity*(*list-name-fm*(*N*,*Cons*(*a*,*t*))) = *1* **using** *ih aN*[*OF a*] *N*[*OF a*] *t*
by (*simp add*: *arity-Exists arity-And arity-Cons-fm ord-simp-union*)
qed

lemma *sats-list-name-fm*:
assumes *model*: *M-ZF1-trans*(*A*) **and** *l*: *l* ∈ *list*(*B*)
and *BA*: *B* ⊆ *A*
and *N*: ⋀*a y env*. *a* ∈ *B* ⟹ *y* ∈ *A* ⟹ *env* ∈ *list*(*A*) ⟹
(*A*, *Cons*(*y*,*env*) ⊨ *N*(*a*)) ⟷ *y* = *a*
shows ⋀*y env*. *y* ∈ *A* ⟹ *env* ∈ *list*(*A*) ⟹
(*A*, *Cons*(*y*,*env*) ⊨ *list-name-fm*(*N*,*l*)) ⟷ *y* = *l*
using *l*
proof (*induct l rule*: *list.induct*)
case *Nil*
interpret *M-ZF1-trans A* **by** (*rule model*)
show *?case* **using** *Nil* **by** *simp*
next
case (*Cons a t*)
interpret *M-ZF1-trans A* **by** (*rule model*)
have *aA*: *a* ∈ *A* **using** *Cons.hyps*(*1*) *BA* **by** *blast*
have *tA*: *t* ∈ *A* **using** *list-in-A*[*OF Cons.hyps*(*2*)] *BA* **by** *blast*
show *?case* **using** *Cons.prems Cons.hyps*(*3*) *N*[*OF Cons.hyps*(*1*)] *aA tA* **by** *auto*
qed

definition *instr-name-fm* :: *i* ⇒ *i* **where**
instr-name-fm(*i*) ≡
Exists(*And*(*nat-name-fm*(*fst*(*i*)), *Exists*(*And*(*nat-name-fm*(*snd*(*i*)),
pair-fm(*1*,*0*,*2*)))))

lemma *instr-name-fm-type* [*TC*]: *i* ∈ *instruction* ⟹ *instr-name-fm*(*i*) ∈ *formula*
unfolding *instr-name-fm-def instruction-def* **using** *action-subset-nat* **by** *auto*

lemma *arity-instr-name-fm*: *i* ∈ *instruction* ⟹ *arity*(*instr-name-fm*(*i*)) = *1*
unfolding *instr-name-fm-def instruction-def* **using** *action-subset-nat*
by (*auto simp add*: *arity-Exists arity-And arity-nat-name-fm arity-pair-fm*
ord-simp-union)

lemma *sats-instr-name-fm*:
assumes *model*: *M-ZF1-trans*(*A*) **and** *i*: *i* ∈ *instruction*
and *y*: *y* ∈ *A* **and** *env*: *env* ∈ *list*(*A*)
shows (*A*, *Cons*(*y*,*env*) ⊨ *instr-name-fm*(*i*)) ⟷ *y* = *i*
proof −
interpret *M-ZF1-trans A* **by** (*rule model*)
from *i* **obtain** *a q* **where** *iq*: *i* = ⟨*a*,*q*⟩ **and** *a*: *a* ∈ *nat*
and *q*: *q* ∈ *nat*
unfolding *instruction-def* **using** *action-subset-nat* **by** *blast*
show *?thesis*
using *iq a q y env sats-nat-name-fm*[*OF model a*] *sats-nat-name-fm*[*OF model*
q]

by (*auto simp add*: *instr-name-fm-def*)
qed

15.2 Closed halting sentences

definition *halt-sentence* :: [*i*,*i*] ⇒ *i* **where**
halt-sentence(*P*,*x*) ≡
Exists(*And*(*list-name-fm*(*nat-name-fm*,*x*),
Exists(*And*(*list-name-fm*(*instr-name-fm*,*P*), *halt-fm*))))

The outer quantifier names the input and the inner quantifier names the machine. The environment passed to *halt-fm* is therefore [*P*,*x*]. Both free variables of the halting formula are bound, and the resulting sentence has arity 0.

lemma *halt-sentence-type* [*TC*]:
assumes *P*: *P* ∈ *tm-machine* **and** *x*: *x* ∈ *list*(*symbol*)
shows *halt-sentence*(*P*,*x*) ∈ *formula*
proof −
have *list-name-fm*(*nat-name-fm*,*x*) ∈ *formula*
using *x* **by** (*rule list-name-fm-type*) (*rule nat-name-sym-type*)
moreover have *list-name-fm*(*instr-name-fm*,*P*) ∈ *formula*
using *P* **unfolding** *tm-machine-def* **by** (*rule list-name-fm-type*) *simp*
ultimately show *?thesis* **unfolding** *halt-sentence-def* **by** *simp*
qed

lemma *arity-halt-sentence*:
assumes *P*: *P* ∈ *tm-machine* **and** *x*: *x* ∈ *list*(*symbol*)
shows *arity*(*halt-sentence*(*P*,*x*)) = *0*
proof −
have *arity*(*list-name-fm*(*nat-name-fm*,*x*)) = *1*
using *x* **by** (*rule arity-list-name-fm*) (*erule nat-name-sym-type*, *erule arity-nat-name-sym*)
moreover have *arity*(*list-name-fm*(*instr-name-fm*,*P*)) = *1*
using *P* **unfolding** *tm-machine-def* **by** (*rule arity-list-name-fm*) (*auto intro*: *arity-instr-name-fm*)
moreover have *list-name-fm*(*nat-name-fm*,*x*) ∈ *formula*
using *x* **by** (*rule list-name-fm-type*) (*rule nat-name-sym-type*)
moreover have *list-name-fm*(*instr-name-fm*,*P*) ∈ *formula*
using *P* **unfolding** *tm-machine-def* **by** (*rule list-name-fm-type*) *simp*
ultimately show *?thesis* **unfolding** *halt-sentence-def*
by (*simp add*: *arity-Exists arity-And arity-halt-fm-eq ord-simp-union*)
qed

context *M-ZF1-trans*
begin

theorem *sats-halt-sentence-iff*:
assumes *P*: *P* ∈ *tm-machine* **and** *x*: *x* ∈ *list*(*symbol*)
shows (*M*, [] ⊨ *halt-sentence*(*P*,*x*)) ⟷ *halts-on*(*P*,*x*)

proof −
have *symA*: *symbol* ⊆ *M* **using** *symbol-nat* **by** *auto*
have *insA*: *instruction* ⊆ *M*
unfolding *instruction-def* **using** *action-subset-nat* **by** *auto*
have *PL*: *P* ∈ *list*(*instruction*) **using** *P* **unfolding** *tm-machine-def* **.**
have *nx*: ⋀*y env*. *y* ∈ *M* ⟹ *env* ∈ *list*(*M*) ⟹
(*M*, *Cons*(*y*,*env*) ⊨ *list-name-fm*(*nat-name-fm*,*x*)) ⟷ *y* = *x*
by (*rule sats-list-name-fm*[*OF M-ZF1-trans-axioms x symA*])
(*rule sats-nat-name-sym*[*OF M-ZF1-trans-axioms*])
have *nP*: ⋀*y env*. *y* ∈ *M* ⟹ *env* ∈ *list*(*M*) ⟹
(*M*, *Cons*(*y*,*env*) ⊨ *list-name-fm*(*instr-name-fm*,*P*)) ⟷ *y* = *P*
by (*rule sats-list-name-fm*[*OF M-ZF1-trans-axioms PL insA*])
(*rule sats-instr-name-fm*[*OF M-ZF1-trans-axioms*])
have *xA*: *x* ∈ *M* **using** *x* **by** *simp*
have *PA*: *P* ∈ *M* **using** *P* **by** *simp*
have (*M*, [] ⊨ *halt-sentence*(*P*,*x*)) ⟷ (*M*, [*P*,*x*] ⊨ *halt-fm*)
unfolding *halt-sentence-def* **using** *nx nP xA PA* **by** *auto*
then show *?thesis* **using** *sats-halt-fm*[*OF P x*] **by** *simp*
qed

end

theorem *sats-halt-sentence-iff*:
assumes *transitive-zf-model*(*A*) *P* ∈ *tm-machine x* ∈ *list*(*symbol*)
shows (*A*, [] ⊨ *halt-sentence*(*P*,*x*)) ⟷ *halts-on*(*P*,*x*)
by (*rule M-ZF1-trans.sats-halt-sentence-iff*
[*OF transitive-zf-M-ZF1-trans*[*OF assms*(*1*)] *assms*(*2*,*3*)])

The theorem *sats-halt-sentence-iff* explicitly assumes a transitive ZF set model. The preceding result in *M-ZF1-trans* uses weaker assumptions. Neither theorem asserts model existence.

15.3 Model-theoretic consequences

theorem *sats-halt-sentence-half-tape-eq-iff*:
assumes *model*: *transitive-zf-model*(*A*) **and** *P*: *P* ∈ *tm-machine*
and *x*: *x* ∈ *list*(*symbol*) **and** *y*: *y* ∈ *list*(*symbol*)
and *eq*: *half-tape-eq*(*x*,*y*)
shows (*A*, [] ⊨ *halt-sentence*(*P*,*x*)) ⟷
(*A*, [] ⊨ *halt-sentence*(*P*,*y*))
using *sats-halt-sentence-iff*[*OF model P x*]
sats-halt-sentence-iff[*OF model P y*]
halts-on-half-tape-eq-iff[*OF P x y eq*] **by** *simp*

corollary *transitive-models-agree-on-halting*:
assumes *transitive-zf-model*(*A*) *transitive-zf-model*(*B*)
and *P* ∈ *tm-machine x* ∈ *list*(*symbol*)
shows (*A*, [] ⊨ *halt-sentence*(*P*,*x*)) ⟷ (*B*, [] ⊨ *halt-sentence*(*P*,*x*))
using *sats-halt-sentence-iff assms* **by** *blast*

end

16 Numerical Coding of Formulas

theory *Formula-Coding*
  imports
    Turing-Coding
    ZF−Constructible.Formula
begin

The primitive constructors are membership, equality, NAND, and universal quantification. Their tags are 0, 1, 2, and 3. Atomic formulas encode their two variable indices. NAND encodes the codes of its two subformulas, and universal quantification encodes the code of its body. Pairing makes the constructor tag and its arguments recoverable, so structural induction establishes injectivity.

16.1 Constructor codes

consts *formula-code* :: $i \Rightarrow i$

primrec
  formula-code(*Member*(*x*,*y*)) = *pair-code*(*0*,*pair-code*(*x*,*y*))
  formula-code(*Equal*(*x*,*y*)) = *pair-code*(*1*,*pair-code*(*x*,*y*))
  formula-code(*Nand*(*p*,*q*)) =
    pair-code(*2*,*pair-code*(*formula-code*(*p*),*formula-code*(*q*)))
  formula-code(*Forall*(*p*)) = *pair-code*(*succ*(*2*),*formula-code*(*p*))

lemma *formula-code-type* [*TC*]:
  $p \in$ *formula* $\Longrightarrow$ *formula-code*(*p*) $\in$ *nat*
  by (*induct set*: *formula*) (*auto intro*: *pair-code-type*)

16.2 Injectivity

lemma *pair-code-inject*:
  assumes *a*: $a \in$ *nat* **and** *b*: $b \in$ *nat*
    and *c*: $c \in$ *nat* **and** *d*: $d \in$ *nat*
    and *same*: *pair-code*(*a*,*b*) = *pair-code*(*c*,*d*)
  shows $a = c \wedge b = d$
proof −
  from *same* **have** *decoded*:
    pair-decode(*pair-code*(*a*,*b*)) = *pair-decode*(*pair-code*(*c*,*d*))
    by (*simp only*: *same*)
  from *decoded* **have** *pairs*: $\langle a,b\rangle = \langle c,d\rangle$
    by (*simp only*: *pair-decode-code*[*OF a b*] *pair-decode-code*[*OF c d*])
  from *pairs* **show** *?thesis* **by** *auto*
qed

lemma *pair-code-eq-iff*:
assumes $a \in nat$ $b \in nat$ $c \in nat$ $d \in nat$
shows *pair-code*$(a,b) =$ *pair-code*$(c,d) \longleftrightarrow a = c \wedge b = d$
using *pair-code-inject*[*OF assms*] **by** *auto*

lemma *formula-code-injective*:
assumes $p \in formula$ $q \in formula$
formula-code$(p) =$ *formula-code*(q)
shows $p = q$
using *assms*
proof (*induct p arbitrary*: *q rule*: *formula.induct*)
case (*Member x y*)
from *Member.prems*(*1*) **show** *?case*
by *cases* (*use Member* **in** ‹*simp-all add*:
pair-code-eq-iff pair-code-type formula-code-type›)
next
case (*Equal x y*)
from *Equal.prems*(*1*) **show** *?case*
by *cases* (*use Equal* **in** ‹*simp-all add*:
pair-code-eq-iff pair-code-type formula-code-type›)
next
case (*Nand p r*)
from *Nand.prems*(*1*) *Nand.prems*(*2*) *Nand.hyps*(*1*,*3*) **show** *?case*
by *cases* (*auto simp add*: *pair-code-eq-iff pair-code-type*
formula-code-type intro: *Nand.hyps*(*2*,*4*))
next
case (*Forall p*)
from *Forall.prems*(*1*) *Forall.prems*(*2*) *Forall.hyps*(*1*) **show** *?case*
by *cases* (*auto simp add*: *pair-code-eq-iff pair-code-type*
formula-code-type intro: *Forall.hyps*(*2*))
qed

end

17 Primitive-Recursive Formula Constructors

theory *Formula-Primrec*
imports *Formula-Coding Turing-Primrec*
begin

Primitive-recursive numerical functions implement the tagged formula constructors. Their application equations identify the output on formula codes with the code of the corresponding formula.

17.1 Tagged constructors

definition *pr-formula-binary* :: $i \Rightarrow i$ **where**
pr-formula-binary$(k) \equiv$

COMP(*pr-pair-code*,
[*CONSTANT*(*k*),*COMP*(*pr-pair-code*,[*PROJ*(*0*),*PROJ*(*1*)])])

lemma *pr-formula-binary-in-prim-rec* [*TC*]:
k ∈ *nat* ⟹ *pr-formula-binary*(*k*) ∈ *prim-rec*
unfolding *pr-formula-binary-def* **by** *typecheck*

lemma *pr-formula-binary-apply*:
assumes *k*: *k* ∈ *nat* **and** *x*: *x* ∈ *nat* **and** *y*: *y* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *pr-formula-binary*(*k*)‘*Cons*(*x*,*Cons*(*y*,*l*)) = *pair-code*(*k*,*pair-code*(*x*,*y*))
proof −
from *x y* **have** *inner*: *pair-code*(*x*,*y*) ∈ *nat* **by** (*rule pair-code-type*)
from *k x y tail inner* **show** *?thesis*
unfolding *pr-formula-binary-def*
by (*simp add*: *COMP-2 CONSTANT PROJ-0 pr-pair-code-apply*)
qed

definition *pr-formula-unary* :: *i* ⇒ *i* **where**
pr-formula-unary(*k*) ≡
COMP(*pr-pair-code*,[*CONSTANT*(*k*),*PROJ*(*0*)])

lemma *pr-formula-unary-in-prim-rec* [*TC*]:
k ∈ *nat* ⟹ *pr-formula-unary*(*k*) ∈ *prim-rec*
unfolding *pr-formula-unary-def* **by** *typecheck*

lemma *pr-formula-unary-apply*:
assumes *k*: *k* ∈ *nat* **and** *x*: *x* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *pr-formula-unary*(*k*)‘*Cons*(*x*,*l*) = *pair-code*(*k*,*x*)
using *k x tail* **unfolding** *pr-formula-unary-def*
by (*simp add*: *COMP-2 CONSTANT PROJ-0 pr-pair-code-apply*)

17.2 Logical connectives

definition *pr-formula-neg* :: *i* **where**
pr-formula-neg ≡
COMP(*pr-formula-binary*(*2*),[*PROJ*(*0*),*PROJ*(*0*)])

lemma *pr-formula-neg-in-prim-rec* [*TC*]:
pr-formula-neg ∈ *prim-rec*
unfolding *pr-formula-neg-def* **by** *typecheck*

lemma *pr-formula-neg-raw-apply*:
assumes *c*: *c* ∈ *nat* **and** *tail*: *l* ∈ *list*(*nat*)
shows *pr-formula-neg*‘*Cons*(*c*,*l*) = *pair-code*(*2*,*pair-code*(*c*,*c*))
using *c tail* **unfolding** *pr-formula-neg-def*
by (*simp add*: *COMP-2 PROJ-0 pr-formula-binary-apply*)

lemma *pr-formula-neg-apply*:
assumes *p*: $p \in formula$ **and** *tail*: $l \in list(nat)$
shows *pr-formula-neg*`*Cons*(*formula-code*(*p*),*l*) = *formula-code*(*Neg*(*p*))
using *p tail* **unfolding** *Neg-def*
by (*simp add*: *pr-formula-neg-raw-apply formula-code-type*)

definition *pr-formula-and* :: *i* **where**
pr-formula-and ≡
COMP(*pr-formula-neg*,
[*COMP*(*pr-formula-binary*(*2*),[*PROJ*(*0*),*PROJ*(*1*)])])

lemma *pr-formula-and-in-prim-rec* [*TC*]:
pr-formula-and ∈ *prim-rec*
unfolding *pr-formula-and-def* **by** *typecheck*

lemma *pr-formula-and-apply*:
assumes *p*: $p \in formula$ **and** *q*: $q \in formula$
and *tail*: $l \in list(nat)$
shows *pr-formula-and*`*Cons*(*formula-code*(*p*),*Cons*(*formula-code*(*q*),*l*)) =
formula-code(*And*(*p*,*q*))
proof −
from *p q* **have** *pq*: *Nand*(*p*,*q*) ∈ *formula* **by** *typecheck*
from *p q tail pq* **show** *?thesis* **unfolding** *pr-formula-and-def And-def Neg-def*
by (*simp add*: *COMP-2 COMP-1 PROJ-0 pr-formula-binary-apply*
pr-formula-neg-raw-apply formula-code-type)
qed

definition *pr-formula-or* :: *i* **where**
pr-formula-or ≡
COMP(*pr-formula-binary*(*2*),
[*COMP*(*pr-formula-neg*,[*PROJ*(*0*)]),
COMP(*pr-formula-neg*,[*PROJ*(*1*)])])

lemma *pr-formula-or-in-prim-rec* [*TC*]:
pr-formula-or ∈ *prim-rec*
unfolding *pr-formula-or-def* **by** *typecheck*

lemma *pr-formula-or-apply*:
assumes *p*: $p \in formula$ **and** *q*: $q \in formula$
and *tail*: $l \in list(nat)$
shows *pr-formula-or*`*Cons*(*formula-code*(*p*),*Cons*(*formula-code*(*q*),*l*)) =
formula-code(*Or*(*p*,*q*))
using *p q tail* **unfolding** *pr-formula-or-def Or-def Neg-def*
by (*simp add*: *COMP-2 COMP-1 PROJ-0 pr-formula-neg-apply*
pr-formula-binary-apply formula-code-type Neg-def)

definition *pr-formula-exists* :: *i* **where**
pr-formula-exists ≡
COMP(*pr-formula-neg*,

```
    [COMP(pr-formula-unary(succ(2)),
      [COMP(pr-formula-neg,[PROJ(0)])])])

lemma pr-formula-exists-in-prim-rec [TC]:
  pr-formula-exists ∈ prim-rec
  unfolding pr-formula-exists-def by typecheck

lemma pr-formula-exists-apply:
  assumes p: p ∈ formula and tail: l ∈ list(nat)
  shows pr-formula-exists`Cons(formula-code(p),l) = formula-code(Exists(p))
proof −
  from p have neg: Neg(p) ∈ formula by typecheck
  from neg have forall: Forall(Neg(p)) ∈ formula by typecheck
  from p tail neg forall show ?thesis
    unfolding pr-formula-exists-def Exists-def Neg-def
    by (simp add: COMP-1 PROJ-0 pr-formula-neg-apply
        pr-formula-neg-raw-apply pr-formula-unary-apply formula-code-type)
qed

end
```

18 Primitive-Recursive Names for Numerals

```
theory Turing-Name-Primrec
  imports
    Formula-Primrec
    Turing-Halting-Sentences
begin
```

Iteration constructs the code of a formula naming a natural number. A second iteration names the unary input with that many one symbols. The application equations use the finite-object names whose satisfaction properties were established in *Turing-Halting-Sentences*.

18.1 Natural number names

```
definition pr-nat-name-step :: i where
  pr-nat-name-step ≡
    COMP(pr-formula-exists,
      [COMP(pr-formula-and,
        [PROJ(0),CONSTANT(formula-code(succ-fm(0,1)))])])

lemma pr-nat-name-step-in-prim-rec [TC]:
  pr-nat-name-step ∈ prim-rec
  unfolding pr-nat-name-step-def
  by typecheck

lemma pr-nat-name-step-apply:
  assumes k: k ∈ nat and tail: l ∈ list(nat)
```

shows *pr-nat-name-step'Cons(formula-code(nat-name-fm(k)),l) =*
formula-code(nat-name-fm(succ(k)))
proof −
from *k* **have** *name*: *nat-name-fm(k) ∈ formula* **by** (*rule nat-name-fm-type*)
have *successor*: *succ-fm(0,1) ∈ formula* **by** *typecheck*
from *name successor* **have** *conjunction*:
And(nat-name-fm(k),succ-fm(0,1)) ∈ formula **by** *typecheck*
from *k tail name successor conjunction* **show** *?thesis*
unfolding *pr-nat-name-step-def*
by (*simp add*: *COMP-1 COMP-2 PROJ-0 CONSTANT*
pr-formula-and-apply pr-formula-exists-apply formula-code-type)
qed

definition *pr-nat-name-code* :: *i* **where**
pr-nat-name-code ≡
PREC(CONSTANT(formula-code(empty-fm(0))),pr-nat-name-step)

lemma *pr-nat-name-code-in-prim-rec* [*TC*]:
pr-nat-name-code ∈ prim-rec
unfolding *pr-nat-name-code-def*
by *typecheck*

theorem *pr-nat-name-code-apply*:
assumes *n*: *n ∈ nat* **and** *tail*: *l ∈ list(nat)*
shows *pr-nat-name-code'Cons(n,l) = formula-code(nat-name-fm(n))*
using *n*
proof (*induct n rule*: *nat-induct*)
case *0*
with *tail* **show** *?case*
unfolding *pr-nat-name-code-def*
by (*simp add*: *PREC-0 CONSTANT*)
next
case (*succ n*)
from *succ.hyps(1)* **have** *name*: *nat-name-fm(n) ∈ formula*
by (*rule nat-name-fm-type*)
from *name* **have** *code*: *formula-code(nat-name-fm(n)) ∈ nat*
by (*rule formula-code-type*)
from *succ tail code* **show** *?case*
unfolding *pr-nat-name-code-def*
by (*simp add*: *PREC-succ pr-nat-name-step-apply*)
qed

18.2 Finite list names

definition *pr-list-name-cons* :: *i* **where**
pr-list-name-cons ≡
COMP(pr-formula-exists,
[COMP(pr-formula-and,
[PROJ(0),

```
      COMP(pr-formula-exists,
       [COMP(pr-formula-and,
         [PROJ(1),CONSTANT(formula-code(Cons-fm(1,0,2)))])])])
```

```
lemma pr-list-name-cons-in-prim-rec [TC]:
  pr-list-name-cons ∈ prim-rec
  unfolding pr-list-name-cons-def
  by typecheck

lemma pr-list-name-cons-apply:
  assumes p: p ∈ formula and q: q ∈ formula
    and tail: l ∈ list(nat)
  shows pr-list-name-cons‘Cons(formula-code(p),Cons(formula-code(q),l)) =
    formula-code(Exists(And(p,Exists(And(q,Cons-fm(1,0,2))))))
proof −
  have cons: Cons-fm(1,0,2) ∈ formula by typecheck
  from q cons have inner: And(q,Cons-fm(1,0,2)) ∈ formula by typecheck
  from inner have exists: Exists(And(q,Cons-fm(1,0,2))) ∈ formula by
    typecheck
  from p exists have outer: And(p,Exists(And(q,Cons-fm(1,0,2)))) ∈ formula
    by typecheck
  from p q tail cons inner exists outer show ?thesis
    unfolding pr-list-name-cons-def
    by (simp add: COMP-1 COMP-2 PROJ-0 CONSTANT
       pr-formula-and-apply pr-formula-exists-apply formula-code-type)
qed
```

18.3 Unary input names

```
definition pr-input-name-step :: i where
  pr-input-name-step ≡
    COMP(pr-list-name-cons,
     [CONSTANT(formula-code(nat-name-fm(one-symbol))),PROJ(0)])

lemma pr-input-name-step-in-prim-rec [TC]:
  pr-input-name-step ∈ prim-rec
  unfolding pr-input-name-step-def
  by typecheck

lemma pr-input-name-step-apply:
  assumes n: n ∈ nat and tail: l ∈ list(nat)
  shows pr-input-name-step‘Cons(
      formula-code(list-name-fm(nat-name-fm,numeral-input(n))),l) =
    formula-code(list-name-fm(nat-name-fm,numeral-input(succ(n))))
proof −
  have one: one-symbol ∈ symbol by simp
  from one have item: nat-name-fm(one-symbol) ∈ formula
    by (rule nat-name-sym-type)
  from n have input: numeral-input(n) ∈ list(symbol)
```

by (*rule numeral-input-type*)
from *input* **have** *named*:
list-name-fm(*nat-name-fm*,*numeral-input*(*n*)) ∈ *formula*
by (*rule list-name-fm-type*) (*rule nat-name-sym-type*)
from *n tail item named* **show** *?thesis*
unfolding *pr-input-name-step-def numeral-input-def*
by (*simp add*: *COMP-2 CONSTANT PROJ-0*
pr-list-name-cons-apply formula-code-type)
qed

definition *pr-input-name-code* :: *i* **where**
pr-input-name-code ≡
PREC(*CONSTANT*(*formula-code*(*Nil-fm*(*0*))),*pr-input-name-step*)

lemma *pr-input-name-code-in-prim-rec* [*TC*]:
pr-input-name-code ∈ *prim-rec*
unfolding *pr-input-name-code-def*
by *typecheck*

theorem *pr-input-name-code-apply*:
assumes *n*: *n* ∈ *nat* **and** *tail*: *l* ∈ *list*(*nat*)
shows *pr-input-name-code*‘*Cons*(*n*,*l*) =
formula-code(*list-name-fm*(*nat-name-fm*,*numeral-input*(*n*)))
using *n*
proof (*induct n rule*: *nat-induct*)
case *0*
with *tail* **show** *?case*
unfolding *pr-input-name-code-def numeral-input-def*
by (*simp add*: *PREC-0 CONSTANT*)
next
case (*succ n*)
from *succ.hyps*(*1*) **have** *input*: *numeral-input*(*n*) ∈ *list*(*symbol*)
by (*rule numeral-input-type*)
from *input* **have** *named*:
list-name-fm(*nat-name-fm*,*numeral-input*(*n*)) ∈ *formula*
by (*rule list-name-fm-type*) (*rule nat-name-sym-type*)
from *named* **have** *code*:
formula-code(*list-name-fm*(*nat-name-fm*,*numeral-input*(*n*))) ∈ *nat*
by (*rule formula-code-type*)
from *succ tail code* **show** *?case*
unfolding *pr-input-name-code-def*
by (*simp add*: *PREC-succ pr-input-name-step-apply*)
qed

end

19 Primitive-Recursive Machine Names

theory *Turing-Machine-Name-Primrec*

imports *Turing-Name-Primrec*
Turing-Instruction-Normalization
begin

The machine decoder accepts every natural number. Instruction normalization supplies valid action and state fields before their names are constructed. The decoded instruction list has length at most its numeric code e. A primitive recursion of length e traverses the indices in reverse order and constructs the code of the complete list name.

19.1 Instruction names

definition *pr-instr-name-from-fields* :: *i* **where**
pr-instr-name-from-fields ≡
COMP(*pr-formula-exists*,
[*COMP*(*pr-formula-and*,
[*COMP*(*pr-nat-name-code*,[*PROJ*(*0*)]),
COMP(*pr-formula-exists*,
[*COMP*(*pr-formula-and*,
[*COMP*(*pr-nat-name-code*,[*PROJ*(*1*)]),
CONSTANT(*formula-code*(*pair-fm*(*1*,*0*,*2*)))])])])])

lemma *pr-instr-name-from-fields-in-prim-rec* [*TC*]:
pr-instr-name-from-fields ∈ *prim-rec*
unfolding *pr-instr-name-from-fields-def*
by *typecheck*

lemma *pr-instr-name-from-fields-apply*:
assumes *a*: *a* ∈ *nat* **and** *b*: *b* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *pr-instr-name-from-fields*‘*Cons*(*a*,*Cons*(*b*,*l*)) =
formula-code(*instr-name-fm*(⟨*a*,*b*⟩))
proof −
have *pair*: *pair-fm*(*1*,*0*,*2*) ∈ *formula* **by** *typecheck*
from *a b* **have** *left*: *nat-name-fm*(*a*) ∈ *formula*
and *right*: *nat-name-fm*(*b*) ∈ *formula* **by** (*auto intro*: *nat-name-fm-type*)
from *right pair* **have** *inner*: *And*(*nat-name-fm*(*b*),*pair-fm*(*1*,*0*,*2*)) ∈ *formula*
by *typecheck*
from *inner* **have** *inner-ex*:
Exists(*And*(*nat-name-fm*(*b*),*pair-fm*(*1*,*0*,*2*))) ∈ *formula*
by *typecheck*
from *left inner-ex* **have** *outer*:
And(*nat-name-fm*(*a*),*Exists*(*And*(*nat-name-fm*(*b*),*pair-fm*(*1*,*0*,*2*)))) ∈ *formula*
by *typecheck*
from *a b tail pair left right inner inner-ex outer* **show** *?thesis*
unfolding *pr-instr-name-from-fields-def instr-name-fm-def*
by (*simp add*: *COMP-1 COMP-2 PROJ-0 CONSTANT*
pr-nat-name-code-apply pr-formula-and-apply
pr-formula-exists-apply formula-code-type)

qed

definition *pr-instr-name-code* :: *i* **where**
pr-instr-name-code ≡
COMP(*pr-instr-name-from-fields*,
[*COMP*(*pr-pair-left*,[*pr-normalize-instruction-code*]),
COMP(*pr-pair-right*,[*pr-normalize-instruction-code*])])

lemma *pr-instr-name-code-in-prim-rec* [*TC*]:
pr-instr-name-code ∈ *prim-rec*
unfolding *pr-instr-name-code-def* **by** *typecheck*

lemma *pr-instr-name-code-apply*:
assumes *n*: *n* ∈ *nat* **and** *tail*: *l* ∈ *list*(*nat*)
shows *pr-instr-name-code*‘*Cons*(*n*,*l*) =
formula-code(*instr-name-fm*(*decode-instruction*(*n*)))
proof −
from *n* **have** *ins*: *decode-instruction*(*n*) ∈ *instruction*
by (*rule decode-instruction-type*)
from *ins* **have** *a*: *fst*(*decode-instruction*(*n*)) ∈ *nat*
and *b*: *snd*(*decode-instruction*(*n*)) ∈ *nat*
unfolding *instruction-def* **using** *action-subset-nat* **by** *auto*
have *normalized*:
normalize-instruction-code(*n*) =
pair-code(*fst*(*decode-instruction*(*n*)),*snd*(*decode-instruction*(*n*)))
unfolding *normalize-instruction-code-correct encode-instruction-def* **by** *simp*
from *a b normalized* **have** *left*:
pair-left-code(*normalize-instruction-code*(*n*)) = *fst*(*decode-instruction*(*n*))
and *right*:
pair-right-code(*normalize-instruction-code*(*n*)) =
snd(*decode-instruction*(*n*))
by (*simp-all add*: *pair-left-code-pair-code pair-right-code-pair-code*)
from *ins* **have** *pair*:
⟨*fst*(*decode-instruction*(*n*)),*snd*(*decode-instruction*(*n*))⟩ =
decode-instruction(*n*)
unfolding *instruction-def* **by** (*rule Pair-fst-snd-eq*)
from *n tail a b left right pair* **show** *?thesis*
unfolding *pr-instr-name-code-def*
by (*simp add*: *COMP-2 COMP-1 pr-normalize-instruction-code-apply*
pr-pair-left-code-apply pr-pair-right-code-apply
pr-instr-name-from-fields-apply normalize-instruction-code-type)
qed

19.2 Decoded list indices

lemma *drop-at-index*:
assumes *k*: *k* ∈ *nat* **and** *l*: *l* ∈ *list*(*A*)
shows *drop*(*k*,*l*) =
(*if k* < *length*(*l*) *then Cons*(*nth*(*k*,*l*),*drop*(*succ*(*k*),*l*)) *else* [])

proof (*cases* $k < length(l)$)
  case *True*
  from *l k* **have** *dropped*: $drop(k,l) \in list(A)$ **by** *typecheck*
  from *True k l* **have** *nonempty*: $drop(k,l) \neq []$
    using *drop-eq-Nil-iff*[*OF k l*]
    by (*simp add*: *not-le-iff-lt*)
  from *dropped nonempty* **obtain** *a t* **where** *d*: $drop(k,l) = Cons(a,t)$
    by (*cases rule*: *list.cases*) *auto*
  from *k l* **have** *nth*: $nth(0,drop(k,l)) = nth(k,l)$
    using *nth-drop*[*OF k*] **by** *simp*
  from *d nth* **have** *a*: $a = nth(k,l)$ **by** *simp*
  from *d* **have** *t*: $t = drop(succ(k),l)$ **by** (*simp add*: *drop-succ*)
  from *True d a t* **show** *?thesis* **by** *simp*
next
  case *False*
  from *k l False* **have** $length(l) \leq k$
    by (*simp add*: *not-lt-iff-le*)
  with *k l* **have** $drop(k,l) = []$ **by** (*rule drop-all*)
  with *False* **show** *?thesis* **by** *simp*
qed

lemma *code-nth-decode*:
  assumes *e*: $e \in nat$ **and** *k*: $k \in nat$
  shows $code\text{-}nth(k,e) = nth(k,nat\text{-}list\text{-}decode(e))$
proof −
  from *e* **have** *l*: $nat\text{-}list\text{-}decode(e) \in list(nat)$
    by (*rule nat-list-decode-type*)
  from *code-nth-encode*[*OF k l*] *nat-list-encode-decode*[*OF e*]
  show *?thesis* **by** *simp*
qed

lemma *code-has-index-decode*:
  assumes *e*: $e \in nat$ **and** *k*: $k \in nat$
  shows $code\text{-}has\text{-}index(k,e) =$
    (*if* $k < length(nat\text{-}list\text{-}decode(e))$ *then 1 else 0*)
proof −
  from *e* **have** *l*: $nat\text{-}list\text{-}decode(e) \in list(nat)$
    by (*rule nat-list-decode-type*)
  from *code-has-index-encode*[*OF k l*] *nat-list-encode-decode*[*OF e*]
  show *?thesis* **by** *simp*
qed

lemma *decode-machine-length*:
  assumes *e*: $e \in nat$
  shows $length(decode\text{-}machine(e)) = length(nat\text{-}list\text{-}decode(e))$
  unfolding *decode-machine-def decode-machine-list-def*
  using *nat-list-decode-type*[*OF e*] **by** *simp*

lemma *decode-machine-nth*:

```
  assumes e: e ∈ nat and k: k ∈ nat
    and index: k < length(decode-machine(e))
  shows nth(k,decode-machine(e)) = decode-instruction(code-nth(k,e))
proof −
  from e have l: nat-list-decode(e) ∈ list(nat)
    by (rule nat-list-decode-type)
  from index decode-machine-length[OF e] have bound:
    k < length(nat-list-decode(e)) by simp
  from e k index l bound show ?thesis
    unfolding decode-machine-def decode-machine-list-def
    by (simp add: code-nth-decode[OF e k])
qed
```

19.3 Bounded reverse traversal

At stage $k < e$, the selected index is $e \#- succ(k)$. The accumulator initially names the empty list. An absent index leaves it unchanged, and a present index prepends the name of the decoded instruction. After n stages, it names $drop(e \#- n, decode\text{-}machine(e))$. The decoded list has length at most e, so e stages construct its complete name without an unbounded search.

```
definition pr-machine-reverse-index :: i where
  pr-machine-reverse-index ≡
    COMP(pr-diff,[PROJ(2),COMP(SC,[PROJ(1)])])

lemma pr-machine-reverse-index-in-prim-rec [TC]:
  pr-machine-reverse-index ∈ prim-rec
  unfolding pr-machine-reverse-index-def by typecheck

lemma pr-machine-reverse-index-apply:
  assumes acc: r ∈ nat and count: k ∈ nat
    and tm-machine: e ∈ nat and tail: l ∈ list(nat)
  shows pr-machine-reverse-index‘Cons(r,Cons(k,Cons(e,l))) = e #− succ(k)
  using acc count tm-machine tail unfolding pr-machine-reverse-index-def
  by (simp add: COMP-2 COMP-1 PROJ-0 SC pr-diff-apply)

definition pr-machine-present :: i where
  pr-machine-present ≡
    COMP(pr-list-has-index,[pr-machine-reverse-index,PROJ(2)])

lemma pr-machine-present-in-prim-rec [TC]:
  pr-machine-present ∈ prim-rec
  unfolding pr-machine-present-def by typecheck

lemma pr-machine-present-apply:
  assumes acc: r ∈ nat and count: k ∈ nat
    and tm-machine: e ∈ nat and tail: l ∈ list(nat)
  shows pr-machine-present‘Cons(r,Cons(k,Cons(e,l))) =
    code-has-index(e #− succ(k),e)
```

using *acc count tm-machine tail* **unfolding** *pr-machine-present-def*
by (*simp add*: *COMP-2 pr-machine-reverse-index-apply*
pr-list-has-index-apply)

definition *pr-machine-element* :: *i* **where**
pr-machine-element ≡
COMP(*pr-list-nth*,[*pr-machine-reverse-index*,*PROJ*(*2*)])

lemma *pr-machine-element-in-prim-rec* [*TC*]:
pr-machine-element ∈ *prim-rec*
unfolding *pr-machine-element-def* **by** *typecheck*

lemma *pr-machine-element-apply*:
assumes *acc*: *r* ∈ *nat* **and** *count*: *k* ∈ *nat*
and *tm-machine*: *e* ∈ *nat* **and** *tail*: *l* ∈ *list*(*nat*)
shows *pr-machine-element*‘*Cons*(*r*,*Cons*(*k*,*Cons*(*e*,*l*))) =
code-nth(*e* #− *succ*(*k*),*e*)
using *acc count tm-machine tail* **unfolding** *pr-machine-element-def*
by (*simp add*: *COMP-2 pr-machine-reverse-index-apply*
pr-list-nth-apply)

definition *pr-machine-name-step* :: *i* **where**
pr-machine-name-step ≡
COMP(*pr-if-zero*,
[*pr-machine-present*,*PROJ*(*0*),
COMP(*pr-list-name-cons*,
[*COMP*(*pr-instr-name-code*,[*pr-machine-element*]),*PROJ*(*0*)])])

lemma *pr-machine-name-step-in-prim-rec* [*TC*]:
pr-machine-name-step ∈ *prim-rec*
unfolding *pr-machine-name-step-def* **by** *typecheck*

lemma *pr-machine-name-step-apply*:
assumes *q*: *q* ∈ *formula* **and** *k*: *k* ∈ *nat*
and *e*: *e* ∈ *nat* **and** *tail*: *l* ∈ *list*(*nat*)
shows *pr-machine-name-step*‘*Cons*(*formula-code*(*q*),*Cons*(*k*,*Cons*(*e*,*l*))) =
(*if code-has-index*(*e* #− *succ*(*k*),*e*) = *0 then formula-code*(*q*)
else formula-code(*Exists*(*And*(
instr-name-fm(*decode-instruction*(*code-nth*(*e* #− *succ*(*k*),*e*))),
Exists(*And*(*q*,*Cons-fm*(*1*,*0*,*2*)))))))
proof −
let *?j* = *e* #− *succ*(*k*)
from *e k* **have** *j*: *?j* ∈ *nat* **by** *typecheck*
from *j e* **have** *selected*: *code-nth*(*?j*,*e*) ∈ *nat*
by (*rule code-nth-type*)
from *selected* **have** *ins*: *decode-instruction*(*code-nth*(*?j*,*e*)) ∈ *instruction*
by (*rule decode-instruction-type*)
from *ins* **have** *item*:
instr-name-fm(*decode-instruction*(*code-nth*(*?j*,*e*))) ∈ *formula*

```
    by (rule instr-name-fm-type)
  have cons: Cons-fm(1,0,2) ∈ formula by typecheck
  from q cons have inner: Exists(And(q,Cons-fm(1,0,2))) ∈ formula
    by typecheck
  from item inner have combined:
    Exists(And(instr-name-fm(decode-instruction(code-nth(?j,e))),
      Exists(And(q,Cons-fm(1,0,2))))) ∈ formula
    by typecheck
  from q k e tail j selected item inner combined show ?thesis
    unfolding pr-machine-name-step-def
    by (simp add: COMP-3 COMP-2 COMP-1 PROJ-0 CONSTANT
        pr-machine-present-apply pr-machine-element-apply
        pr-instr-name-code-apply pr-list-name-cons-apply
        pr-if-zero-apply formula-code-type code-has-index-type)
qed

definition pr-machine-name-iter :: i where
  pr-machine-name-iter ≡
    PREC(CONSTANT(formula-code(Nil-fm(0))),pr-machine-name-step)

lemma pr-machine-name-iter-in-prim-rec [TC]:
  pr-machine-name-iter ∈ prim-rec
  unfolding pr-machine-name-iter-def by typecheck

theorem pr-machine-name-iter-apply:
  assumes e: e ∈ nat and n: n ∈ nat
    and bound: n ≤ e and tail: l ∈ list(nat)
  shows pr-machine-name-iter`Cons(n,Cons(e,l)) =
    formula-code(list-name-fm(instr-name-fm,
      drop(e #− n,decode-machine(e))))
  using n bound
proof (induct n rule: nat-induct)
  case 0
  from e have tm-machine: decode-machine(e) ∈ list(instruction)
    using decode-machine-type[OF e] unfolding tm-machine-def by simp
  from decode-machine-length[OF e] length-nat-list-decode-le[OF e]
  have length: length(decode-machine(e)) ≤ e by simp
  from drop-all[OF e tm-machine length] have empty:
    drop(e,decode-machine(e)) = [] .
  from e tail empty show ?case
    unfolding pr-machine-name-iter-def
    by (simp add: PREC-0 CONSTANT)
next
  case (succ k)
  let ?P = decode-machine(e)
  let ?j = e #− succ(k)
  let ?t = drop(succ(?j),?P)
  let ?Q = list-name-fm(instr-name-fm,?t)
  from succ.prems have kbound: k ≤ e
```

```
    by (blast intro: leI)
  from succ.hyps(2)[OF kbound] have previous:
    pr-machine-name-iter'Cons(k,Cons(e,l)) = formula-code(?Q)
    using succ-diff-succ[OF succ.hyps(1) e succ.prems] by simp
  from e have tm-machine: ?P ∈ list(instruction)
    using decode-machine-type[OF e] unfolding tm-machine-def by simp
  from succ.hyps(1) e have j: ?j ∈ nat by typecheck
  from j tm-machine have suffix: ?t ∈ list(instruction) by typecheck
  from suffix have named: ?Q ∈ formula
    by (rule list-name-fm-type) (rule instr-name-fm-type)
  from code-has-index-decode[OF e j] decode-machine-length[OF e]
  have present:
    code-has-index(?j,e) = (if ?j < length(?P) then 1 else 0)
    by simp
  have iteration:
    pr-machine-name-iter'Cons(succ(k),Cons(e,l)) = pr-machine-name-step'Cons(
        pr-machine-name-iter'Cons(k,Cons(e,l)),Cons(k,Cons(e,l)))
    unfolding pr-machine-name-iter-def
    using succ.hyps(1) e tail by (simp add: PREC-succ)
  from pr-machine-name-step-apply[OF named succ.hyps(1) e tail]
  have tm-step:
    pr-machine-name-step'Cons(formula-code(?Q),Cons(k,Cons(e,l))) =
      (if code-has-index(?j,e) = 0 then formula-code(?Q)
       else formula-code(Exists(And(
         instr-name-fm(decode-instruction(code-nth(?j,e))),
         Exists(And(?Q,Cons-fm(1,0,2))))))) .
  show ?case
  proof (cases ?j < length(?P))
    case True
    from drop-at-index[OF j tm-machine] True have current:
      drop(?j,?P) = Cons(nth(?j,?P),?t) by simp
    from decode-machine-nth[OF e j True] have head:
      decode-instruction(code-nth(?j,e)) = nth(?j,?P) by simp
    from iteration previous tm-step present current head True show ?thesis
      by simp
  next
    case False
    from drop-at-index[OF j tm-machine] False have current:
      drop(?j,?P) = [] by simp
    from current have tail-empty: ?t = [] by (simp add: drop-succ)
    from iteration previous tm-step present current tail-empty False show ?thesis
      by simp
  qed
qed
```

19.4 Complete machine names

```
definition pr-machine-name-code :: i where
  pr-machine-name-code ≡
```

COMP(*pr-machine-name-iter*,[*PROJ*(*0*),*PROJ*(*0*)])

lemma *pr-machine-name-code-in-prim-rec* [*TC*]:
pr-machine-name-code ∈ *prim-rec*
unfolding *pr-machine-name-code-def* **by** *typecheck*

theorem *pr-machine-name-code-apply*:
assumes *e*: *e* ∈ *nat* **and** *tail*: *l* ∈ *list*(*nat*)
shows *pr-machine-name-code*‘*Cons*(*e*,*l*) =
formula-code(*list-name-fm*(*instr-name-fm*,*decode-machine*(*e*)))
proof −
from *e* **have** *iter*:
pr-machine-name-iter‘[*e*,*e*] =
formula-code(*list-name-fm*(*instr-name-fm*,*decode-machine*(*e*)))
using *pr-machine-name-iter-apply*[*OF e e*] **by** *simp*
from *e tail iter* **show** *?thesis*
unfolding *pr-machine-name-code-def* **by** (*simp add*: *COMP-2 PROJ-0*)
qed

end

20 Effective Halting Sentences

theory *Turing-Halting-Codes*
imports *Turing-Machine-Name-Primrec*
begin

The machine code and unary input are independent arguments of a primitive-recursive sentence constructor. The resulting code determines a closed formula whose satisfaction is equivalent to the specified execution halting. The self-input constructor is the diagonal instance. Membership in *prim-rec* certifies numerical construction independently of a machine implementation of that function.

20.1 Closed halting sentence code

definition *pr-machine-input-sentence-code* :: *i* **where**
pr-machine-input-sentence-code ≡
COMP(*pr-formula-exists*,
[*COMP*(*pr-formula-and*,
[*COMP*(*pr-input-name-code*,[*PROJ*(*1*)]),
COMP(*pr-formula-exists*,
[*COMP*(*pr-formula-and*,
[*pr-machine-name-code*,*CONSTANT*(*formula-code*(*halt-fm*))])])])])

lemma *pr-machine-input-sentence-code-in-prim-rec* [*TC*]:
pr-machine-input-sentence-code ∈ *prim-rec*
unfolding *pr-machine-input-sentence-code-def*

by *typecheck*

theorem *pr-machine-input-sentence-code-apply*:
assumes *e*: $e \in nat$ **and** *n*: $n \in nat$ **and** *tail*: $l \in list(nat)$
shows *pr-machine-input-sentence-code*`*Cons*(e,*Cons*(n,l)) =
formula-code(*halt-sentence*(*decode-machine*(e),*numeral-input*(n)))
proof −
have *halt*: *halt-fm* ∈ *formula* **by** (*rule halt-fm-type*)
from *n* **have** *input*: *numeral-input*(n) ∈ *list*(*symbol*)
by (*rule numeral-input-type*)
from *e* **have** *tm-machine*: *decode-machine*(e) ∈ *tm-machine*
by (*rule decode-machine-type*)
from *input* **have** *input-name*:
list-name-fm(*nat-name-fm*,*numeral-input*(n)) ∈ *formula*
by (*rule list-name-fm-type*) (*rule nat-name-sym-type*)
from *tm-machine* **have** *machine-name*:
list-name-fm(*instr-name-fm*,*decode-machine*(e)) ∈ *formula*
unfolding *tm-machine-def*
by (*rule list-name-fm-type*) (*rule instr-name-fm-type*)
from *machine-name halt* **have** *inner*:
Exists(*And*(*list-name-fm*(*instr-name-fm*,*decode-machine*(e)),*halt-fm*))
∈ *formula* **by** *typecheck*
from *input-name inner* **have** *outer*:
And(*list-name-fm*(*nat-name-fm*,*numeral-input*(n)),
Exists(*And*(*list-name-fm*(*instr-name-fm*,*decode-machine*(e)),*halt-fm*)))
∈ *formula* **by** *typecheck*
from *e n tail halt input-name machine-name inner outer* **show** *?thesis*
unfolding *pr-machine-input-sentence-code-def halt-sentence-def*
by (*simp add*: *COMP-1 COMP-2 CONSTANT PROJ-0*
pr-input-name-code-apply pr-machine-name-code-apply
pr-formula-and-apply pr-formula-exists-apply formula-code-type)
qed

The equation *pr-machine-input-sentence-code-apply* supplies a particular closed formula with the constructed code. The code interface below identifies any formula with that code. Injectivity removes the need for a separate formula decoder. Formula membership is a premise because the coding function is specified on formulas.

theorem *pr-machine-input-sentence-code-iff*:
assumes *e*: $e \in nat$ **and** *n*: $n \in nat$ **and** *p*: $p \in formula$
shows *formula-code*(p) = *pr-machine-input-sentence-code*`[e,n] ⟷
p = *halt-sentence*(*decode-machine*(e),*numeral-input*(n))
using *pr-machine-input-sentence-code-apply*[*OF e n*, *of* []] *p*
halt-sentence-type[*OF decode-machine-type*[*OF e*] *numeral-input-type*[*OF n*]]
formula-code-injective **by** *auto*

theorem *sats-machine-input-sentence-code-iff*:
assumes *model*: *transitive-zf-model*(A)
and *e*: $e \in nat$ **and** *n*: $n \in nat$ **and** *p*: $p \in formula$

and *code*: *formula-code*(p) = *pr-machine-input-sentence-code*‘[e,n]
shows (A, [] $\models$ p) $\longleftrightarrow$
halts-on(*decode-machine*(e),*numeral-input*(n))
using *pr-machine-input-sentence-code-iff*[*OF e n p*] *code*
sats-halt-sentence-iff[*OF model decode-machine-type*[*OF e*]
numeral-input-type[*OF n*]] **by** *simp*

20.2 Self-input instance

definition *pr-halt-sentence-code* :: i **where**
pr-halt-sentence-code $\equiv$
COMP(*pr-machine-input-sentence-code*,[*PROJ*(*0*),*PROJ*(*0*)])

lemma *pr-halt-sentence-code-in-prim-rec* [*TC*]:
pr-halt-sentence-code $\in$ *prim-rec*
unfolding *pr-halt-sentence-code-def* **by** *typecheck*

theorem *pr-halt-sentence-code-apply*:
assumes *e*: $e \in nat$ **and** *tail*: $l \in list(nat)$
shows *pr-halt-sentence-code*‘*Cons*(e,l) =
formula-code(*halt-sentence*(*decode-machine*(e),*numeral-input*(e)))
using *e tail* **unfolding** *pr-halt-sentence-code-def*
by (*simp add*: *COMP-2 PROJ-0 pr-machine-input-sentence-code-apply*)

corollary *transitive-zf-contains-decoded-machine*:
⟦*transitive-zf-model*(A); $e \in nat$⟧ $\Longrightarrow$ *decode-machine*(e) $\in A$
by (*rule transitive-zf-contains-machine*) (*auto intro*: *decode-machine-type*)

end

21 Using the Halting Formula Interface

theory *Turing-Halting-Examples*
imports *Turing-Examples Turing-Halting-Codes*
begin

The operational proof from the machine example supplies an internal theorem in every transitive ZF set model. The same result applies to a formula identified only by its numerical code. Both clients use the public interfaces without unfolding the halting formula, its finite-object names, or the primitive-recursive constructor.

21.1 From execution to satisfaction

theorem *scan-append-sentence*:
assumes *model*: *transitive-zf-model*(A) **and** n: $n \in nat$
shows (A, [] $\models$ *halt-sentence*(*scan-append-machine*,*numeral-input*(n)))
using *sats-halt-sentence-iff*[*OF model scan-append-machine-type*
numeral-input-type[*OF n*]] *scan-append-halts*[*OF n*] **by** *simp*

21.2 From a sentence code to satisfaction

theorem *scan-append-sentence-code*:
assumes *model*: *transitive-zf-model*(A) **and** n: $n \in nat$
and p: $p \in formula$
and *code*: *formula-code*(p) =
pr-machine-input-sentence-code`[*encode-machine*(*scan-append-machine*),n]
shows (A, [] $\models$ p)
using *sats-machine-input-sentence-code-iff*[*OF model*
encode-machine-natural[*OF scan-append-machine-type*] n p *code*]
decode-encode-machine[*OF scan-append-machine-type*] *scan-append-halts*[*OF* n]
by *simp*

end

References

[1] Jian Xu, Xingyuan Zhang, and Christian Urban. *Mechanising Turing Machines and Computability Theory in Isabelle/HOL*. In *Interactive Theorem Proving*, LNCS 7998, 147–162. Springer, 2013. https://doi.org/10.1007/978-3-642-39634-2_13.

[2] Emmanuel Gunther, Miguel Pagano, Pedro Sánchez Terraf, and Matías Steinberg. *The Independence of the Continuum Hypothesis in Isabelle/ZF*. Archive of Formal Proofs, 2022. https://isa-afp.org/entries/Independence_CH.html.

[3] Jian Xu, Xingyuan Zhang, Christian Urban, Sebastiaan J. C. Joosten, and Franz Regensburger. *Universal Turing Machine*. Archive of Formal Proofs, 2019, revised 2022. https://isa-afp.org/entries/Universal_Turing_Machine.html.

[4] Emmanuel Gunther, Miguel Pagano, Pedro Sánchez Terraf, and Matías Steinberg. *Transitive Models of Fragments of ZFC*. Archive of Formal Proofs, 2022. https://isa-afp.org/entries/Transitive_Models.html.

[5] Thomas Jech. *Set Theory: The Third Millennium Edition, Revised and Expanded*. Springer, 2003. https://doi.org/10.1007/3-540-44761-X.

[6] Lawrence C. Paulson. *The Relative Consistency of the Axiom of Choice, Mechanized Using Isabelle/ZF*. LMS Journal of Computation and Mathematics, 6, 198–248, 2003.

[7] Nora Szasz. *A Machine Checked Proof that Ackermann's Function is not Primitive Recursive*. In Gérard Huet and Gordon Plotkin, editors, *Logical Environments*, 317–338. Cambridge University Press, 1993.